

PROPOSAL PENELITIAN SKRIPSI

RANCANG BANGUN APLIKASI *POINT OF SALE* DENGAN FITUR PREDIKSI PENJUALAN HARIAN MENGUNAKAN METODE *LONG SHORT-TERM MEMORY* (LSTM)



Disusun oleh:

MUHAMMAD KHOLIS

2022573010098

**TEKNOLOGI INFORMASI KOMPUTER
TEKNIK INFORMATIKA
POLITEKNIK NEGERI LHOKSEUMAWE
TAHUN 2026**

HALAMAN PERSETUJUAN

RANCANG BANGUN APLIKASI POINT OF SALE DENGAN FITUR PREDIKSI PENJUALAN HARIAN MENGGUNAKAN METODE LONG SHORT-TERM MEMORY (LSTM)

Dipersiapkan dan Disusun oleh

MUHAMMAD KHOLIS

2022573010098

Telah disetujui oleh Tim Dosen Pembimbing Penelitian Skripsi
pada tanggal 10 Juni 2026

Mengetahui:
Ketua Prodi Teknik Informatika



M. Khadafi, S.T., M.T
NIP. 197507182002121004

Buketrata, 12 Mei 2026
Ketua Pengusul,



Muhammad Kholis
2022573010098

Menyetujui,
Ketua Jurusan Teknologi Informasi & Komputer



Salahuddin, S.T., M.Cs.
NIP. 197404242002121001

DAFTAR ISI

DAFTAR ISI	i
DAFTAR GAMBAR	iv
DAFTAR TABEL	v
BAB 1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	3
1.3 Tujuan Penelitian	4
1.4 Batasan Masalah	4
1.5 Manfaat Penelitian	5
1.6 Sistematika Penulisan	5
BAB 2 TINJAUAN PUSTAKA	7
2.1 <i>State Of The Art</i>	7
2.2 Sistem <i>Point of Sale</i> (POS)	12
2.3 Peramalan Penjualan (<i>Forecasting</i>)	12
2.4 Arsitektur <i>Long Short-Term Memory</i> (LSTM)	13
2.5 Model Pembanding Peramalan (<i>Baseline</i>)	14
2.6 Metrik Evaluasi Peramalan (sMAPE dan MASE)	15
2.7 <i>Generative Adversarial Network</i> (GAN) dan TimeGAN	16
2.8 <i>Framework</i> Flutter dan Bahasa Dart	16
2.9 Layanan Supabase dan Integrasi <i>Cloud</i>	17
2.10 <i>Isar</i> Database (<i>Local Storage</i>)	18
2.11 Python, TensorFlow, PyTorch, dan Flask	18
2.12 Layanan Firebase	19
2.13 Metode Pengembangan Perangkat Lunak <i>Agile</i>	19
2.14 Pemodelan Sistem (UML dan ERD)	20
2.15 Pengujian <i>Black Box</i>	20
BAB 3 METODOLOGI PENELITIAN	21
3.1 Data dan Pengumpulan Data	21
3.1.1 Tempat Penelitian	21
3.1.2 Waktu Penelitian	21
3.1.3 Alat dan Bahan	22
3.1.4 Variabel Data	24
3.1.5 Sumber Data	24
3.1.5.1 Ekstraksi dan Agregasi Data (Format CSV)	27

3.1. Analisis Kebutuhan Data	29
3.1. Preprocessing Data	30
3.2 Rancangan Sistem	31
3.2. Metode Penelitian	31
3.2.1.1 Alur Penelitian	32
3.2. Diagram Use Case	33
3.2. Arsitektur Sistem	36
3.2. Class Diagram	39
3.2. Activity Diagram	40
3.2. Sequence Diagram	43
3.2. Entity Relationship Diagram (ERD)	44
3.2. Rancangan Algoritma Long Short-Term Memory (LSTM)	47
3.2. Perancangan Wireframe	52
3.2. Teknik Pengujian	65
3.2.10.1 Rancangan Pengujian Evaluasi Model Prediksi	66
3.2.10.2 Rancangan Analisis Pengujian <i>Black Box</i>	67
3.2.10.3 Rancangan Analisis Pengujian <i>White Box</i>	69
3.2. Hasil yang Diharapkan	70
BAB 4 HASIL DAN PEMBAHASAN	71
4.1 Hasil Implementasi Sistem	71
4.1. Implementasi Antarmuka (UI)	71
4.1. Implementasi Backend dan Basis Data Cloud	73
4.1. Implementasi Layanan Prediksi (API)	74
4.2 Preprocessing Data	76
4.2. Deskripsi Data dan Eksplorasi Awal	76
4.2. Pra-pemrosesan dan Pembentukan Sequence	77
4.3 Implementasi dan Pelatihan Model Peramalan	79
4.3. Arsitektur Model dan Skenario Eksperimen	79
4.3. Hasil Pelatihan Model	81
4.3. Implementasi Augmentasi Data Sintetis TimeGAN	81
4.3. Implementasi Peramalan Rekursif ke Depan	83
4.4 Evaluasi Model	85
4.4. Evaluasi Model dan Perbandingan dengan Baseline	86
4.4. Evaluasi Multi-Seed dan Analisis Kestabilan	87
4.4. Evaluasi Augmentasi Data Sintetis TimeGAN	89
4.5 Hasil Pengujian Sistem	90
4.5. Skenario Pengujian	90

4.5. Hasil Black Box Testing	91
4.5. Hasil White Box Testing	93
4.5. Evaluasi Kinerja Model	95
4.6 Pembahasan	95
4.6. Pembahasan Pengujian Fungsional dan Arsitektur Offline-First .	95
4.6. Pembahasan Hasil Prediksi Penjualan LSTM	96
4.7 Keterbatasan Sistem dan Penelitian	98
4.7. Keterbatasan Sistem POS	98
4.7. Keterbatasan Penelitian Model Prediksi	99
BAB 5 KESIMPULAN DAN SARAN	101
5.1 Kesimpulan	101
5.2 Saran	102

DAFTAR GAMBAR

3.1	Alur Penelitian	32
3.2	<i>Use Case Diagram</i> Sistem POS	34
3.3	Arsitektur Sistem POS	37
3.4	Class Diagram Sistem POS	39
3.5	Activity Diagram Alur Kerja Transaksi Kasir	41
3.6	Activity Diagram Sinkronisasi Latar Belakang	42
3.7	Activity Diagram Proses AI Smart Analytics	43
3.8	Sequence Diagram Alur Transaksi Kasir	44
3.9	<i>Entity Relationship Diagram</i> (ERD) Sistem POS	45
3.10	Alur Kerja Algoritma <i>Long Short-Term Memory</i> (LSTM)	47
3.11	<i>Wireframe</i> Halaman Registrasi Toko Baru	53
3.12	<i>Wireframe</i> Halaman <i>Login</i>	54
3.13	<i>Wireframe</i> Halaman Pemilihan Toko	55
3.14	<i>Wireframe</i> Halaman Dasbor Owner	56
3.15	<i>Wireframe</i> Halaman POS (Pembayaran)	57
3.16	<i>Wireframe</i> Halaman Riwayat Transaksi	58
3.17	<i>Wireframe</i> Halaman Kelola Produk	59
3.18	<i>Wireframe</i> Halaman Kelola Kategori	60
3.19	<i>Wireframe</i> Halaman Laporan	61
3.20	<i>Wireframe</i> Halaman Pengaturan	62
3.21	<i>Wireframe</i> Halaman Informasi Toko	63
3.22	<i>Wireframe</i> Halaman Struk Digital	64
3.23	<i>Wireframe</i> Halaman Katalog Produk	65
4.1	Kurva Kerugian Pelatihan (<i>Training Loss Curve</i>) Model LSTM	81
4.2	Grafik Pendapatan Aktual vs. Prediksi Model LSTM Hybrid	87
4.3	Boxplot Distribusi MAE Hasil Pengujian <i>Multi-Seed</i>	89

DAFTAR TABEL

2.1	<i>State Of The Art</i>	8
3.1	Jadwal Pelaksanaan Penelitian	22
3.2	Daftar Alat dan Bahan Penelitian	23
3.3	Sebaran Penjualan Berdasarkan Kategori Produk	26
3.4	Daftar 10 Produk Terlaris EatsTEDI UGM	26
3.5	Struktur Skema Dataset <code>daily_sales.csv</code>	28
3.6	Struktur Skema Dataset <code>weekly_sales.csv</code>	28
3.7	Struktur Skema Dataset <code>monthly_sales.csv</code>	29
3.8	Definisi Aktor	35
3.9	Definisi <i>Use Case</i>	35
3.10	Daftar Tabel Basis Data POS	46
3.11	Skenario Pengujian <i>Black Box</i>	68
4.1	Statistik Ringkas Dataset Pendapatan Harian	77
4.2	Pembagian Dataset dan Dimensi Sequence	78
4.3	Perbandingan Performa pada Data Uji (<i>One-Step-Ahead</i>)	86
4.4	Statistik Performa LSTM Hybrid pada 10 Seed	88
4.5	Perbandingan Performa Eksperimen Augmentasi TimeGAN (<i>Multi-Step</i> , Horizon 7)	89
4.6	Hasil Pengujian Black Box	91
4.7	Hasil Pengujian Mekanisme Sinkronisasi	93
4.8	Hasil Pengujian White Box	94

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi dan komunikasi telah mendorong transformasi digital di berbagai sektor bisnis, termasuk pada pengelolaan transaksi dan operasional penjualan usaha mikro, kecil, dan menengah (UMKM). Pemanfaatan sistem *Point of Sale* (POS) berbasis aplikasi kini menjadi salah satu solusi yang banyak digunakan untuk membantu pelaku usaha mencatat transaksi, mengelola stok, serta menyusun laporan penjualan secara lebih terstruktur dan efisien [1]. Namun, Pratama dan Handayani [2] mencatat bahwa adopsi teknologi kasir digital di Indonesia masih terbatas, di mana minimnya fitur analitik menjadi penyebab utama UMKM gagal mengantisipasi siklus musiman penjualan.

Pada praktiknya, sebagian besar aplikasi POS yang digunakan UMKM masih bersifat deskriptif, yaitu hanya menampilkan data historis penjualan tanpa kemampuan analisis prediktif. Pelaku usaha umumnya harus melakukan estimasi penjualan secara manual berdasarkan pengalaman atau intuisi, sehingga rentan menimbulkan kesalahan dalam perencanaan stok, penentuan target, maupun pengelolaan arus kas. Ketidakmampuan sistem dalam memprediksi penjualan harian secara akurat dapat memicu berbagai permasalahan, seperti kelebihan stok (*overstock*), kekurangan persediaan (*stockout*), serta strategi penjualan yang tidak optimal [3].

Perkembangan teknologi kecerdasan buatan, khususnya bidang *deep learning*, membuka peluang untuk menghadirkan fitur prediksi penjualan yang lebih akurat dan adaptif terhadap pola data historis. Salah satu metode *deep learning* yang banyak digunakan dalam analisis data runtun waktu (*time series*) adalah *Long Short-Term Memory* (LSTM). Abbas dkk. [4] membuktikan bahwa LSTM mengungguli model ARIMA konvensional dalam memprediksi data penjualan ritel harian berfluktuasi non-linier tinggi, dengan penurunan *error* hingga 15%. Wilson dan Davies [5] menjelaskan bahwa mekanisme *forget gate*

pada arsitektur LSTM mampu mempertahankan memori pola penjualan jangka panjang, seperti tren musiman akibat periode Ramadan atau Tahun Baru, yang sering menjadi titik kritis operasional UMKM. Choi dan Lee [6] lebih lanjut menunjukkan bahwa penambahan variabel kontekstual seperti jumlah transaksi harian pada model LSTM *multivariate* dapat meningkatkan akurasi prediksi secara signifikan.

Efektivitas penerapan model prediksi tersebut akan semakin optimal apabila didukung oleh arsitektur sistem yang andal pada kondisi lapangan UMKM. Tantangan infrastruktur menjadi hambatan nyata, khususnya koneksi internet yang tidak stabil di daerah. Martinez dan Silva [7] menunjukkan bahwa arsitektur *offline-first* dengan basis data lokal *embedded* merupakan solusi efektif untuk menjamin kelancaran operasional sistem pada jaringan yang tidak andal. Novikov dan Petrov [8] membuktikan bahwa Isar DB unggul sebagai basis data lokal pada aplikasi Flutter berkat kecepatan operasi baca-tulis yang optimal dibandingkan alternatif seperti Hive dan ObjectBox. Untuk mendukung ekosistem *cloud backend* yang andal, Fischer dan Weber [9] mengevaluasi Supabase sebagai platform *Backend-as-a-Service (BaaS) open-source* dengan fitur *Row Level Security (RLS)* dan *real-time subscription* yang sesuai untuk pengelolaan data transaksi multi-tenant UMKM.

Meskipun riset terdahulu telah membuktikan keunggulan masing-masing komponen secara terpisah, belum ada penelitian yang mengintegrasikan keseluruhan komponen tersebut menjadi satu sistem yang utuh. Kumar dan Sharma [10] menyoroti tantangan latensi ketika modul AI di-*deploy* sebagai layanan terpisah yang dipanggil aplikasi klien kasir. Nguyen dan Tran [11] merekomendasikan pemisahan beban kerja transaksional dari beban kerja analitik melalui arsitektur *microservices* agar sistem kasir tidak terganggu selama proses pelatihan model. Roberts dan Thomas [12] membahas standarisasi pengiriman data dari aplikasi *mobile* ke server AI berbasis *REST API*, yang menjadi jembatan integrasi antara sistem POS dan modul prediksi.

Dalam konteks operasional UMKM di Lhokseumawe, kebutuhan akan

sistem kasir yang tidak hanya mampu mencatat transaksi, tetapi juga memprediksi penjualan harian secara akurat sekaligus tetap berfungsi dalam kondisi jaringan terbatas menjadi semakin penting. Kompleksitas pengambilan keputusan stok dan tingginya dinamika pola penjualan menuntut solusi yang memadukan keandalan operasional *offline* dengan kecerdasan analitik prediktif dalam satu kesatuan sistem.

Berdasarkan permasalahan tersebut, penelitian ini bertujuan merancang dan membangun aplikasi *Point of Sale* berbasis *mobile* Android dengan arsitektur *offline-first* menggunakan *framework* Flutter dan Isar DB, yang secara bersamaan dilengkapi fitur prediksi penjualan harian berbasis metode LSTM yang dieksekusi di sisi server melalui *REST API*. Sistem yang dikembangkan diharapkan mampu berfungsi sebagai *Decision Support System* bagi pelaku UMKM di Lhokseumawe dalam mengambil keputusan manajemen stok dan strategi penjualan secara lebih akurat dan berbasis data.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, maka rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana merancang dan membangun aplikasi *Point of Sale* (POS) dengan fitur prediksi penjualan harian berbasis *mobile* yang dapat digunakan oleh pelaku usaha secara efektif?
2. Bagaimana menerapkan metode *Long Short-Term Memory* (LSTM) pada aplikasi *Point of Sale* untuk memprediksi penjualan harian berdasarkan data transaksi historis?
3. Bagaimana tingkat akurasi hasil prediksi penjualan harian yang dihasilkan oleh metode *Long Short-Term Memory* (LSTM) berdasarkan evaluasi menggunakan metrik kesalahan prediksi?

1.3 Tujuan Penelitian

Berdasarkan rumusan masalah yang telah diuraikan, maka tujuan dalam penelitian ini adalah sebagai berikut:

1. Merancang dan membangun aplikasi *Point of Sale* (POS) berbasis *mobile* yang dilengkapi dengan fitur prediksi penjualan harian sebagai alat bantu pengelolaan dan analisis penjualan.
2. Mengimplementasikan metode *Long Short-Term Memory* (LSTM) pada aplikasi *Point of Sale* untuk memprediksi penjualan harian berdasarkan data transaksi historis.
3. Memanfaatkan data transaksi penjualan harian, seperti jumlah transaksi, total penjualan, dan jumlah produk terjual, sebagai data latih utama dalam model prediksi penjualan berbasis metode LSTM.
4. Mengevaluasi kinerja dan tingkat akurasi metode *Long Short-Term Memory* (LSTM) dalam menghasilkan prediksi penjualan harian menggunakan metrik evaluasi kesalahan prediksi.

1.4 Batasan Masalah

Untuk menghindari meluasnya materi pembahasan, maka penelitian ini dibatasi pada hal-hal berikut:

1. Aplikasi dikembangkan menggunakan *framework* Flutter dan difokuskan untuk sistem operasi Android, mengingat dominasi pasar Android pada segmen UMKM lokal.
2. Basis data yang digunakan adalah Supabase dengan skema relasional untuk menyimpan data produk, transaksi, dan akun pengguna.
3. Algoritma yang digunakan adalah *Long Short-Term Memory* (LSTM) dengan model *univariate* atau *multivariate* terbatas, dan prediksi dilakukan untuk rentang waktu harian.
4. Data uji bersumber dari basis data transaksi riil platform EatsTEDI DTEDI Universitas Gadjah Mada dengan periode lebih dari satu tahun; pola musiman

yang diobservasi bersifat kalender akademis (mingguan dan semester), karena cakupan data yang tersedia belum cukup untuk mengamati siklus musiman tahunan penuh secara statistik.

5. Proses pelatihan model dan prediksi dilakukan di sisi *server/cloud* menggunakan bahasa Python, sedangkan aplikasi *mobile* bertugas mengirim data transaksi dan menerima hasil prediksi melalui *REST API*.

1.5 Manfaat Penelitian

Manfaat yang diharapkan dari penelitian ini adalah sebagai berikut:

1. **Bagi Pelaku UMKM:** Memberikan aplikasi *Point of Sale* (POS) berbasis *mobile* yang dapat digunakan untuk pencatatan transaksi penjualan dan pengelolaan stok secara digital, serta menyediakan fitur prediksi penjualan harian berbasis algoritma LSTM sebagai alat bantu dalam pengambilan keputusan *restock* dan pengelolaan persediaan barang.
2. **Bagi Akademisi:** Menjadi referensi dan bahan kajian dalam pengembangan penelitian di bidang sistem informasi dan *machine learning*, khususnya penerapan metode LSTM untuk prediksi penjualan harian pada sistem POS berbasis *mobile* di lingkungan UMKM.
3. **Bagi Peneliti:** Menambah pengetahuan dan pengalaman dalam merancang dan membangun aplikasi POS berbasis *mobile* menggunakan *framework* Flutter serta mengimplementasikan algoritma LSTM untuk menyelesaikan permasalahan prediksi penjualan harian berbasis data transaksi UMKM pada studi kasus nyata.

1.6 Sistematika Penulisan

Sistematika penulisan proposal penelitian skripsi ini disusun untuk memberikan gambaran secara menyeluruh mengenai alur pembahasan yang dilakukan. Sistematika penulisan dibagi menjadi tiga bab utama yang saling berkaitan, dengan rincian sebagai berikut:

BAB I PENDAHULUAN

Bab ini menguraikan gambaran umum penelitian yang meliputi latar belakang masalah, rumusan masalah, tujuan penelitian, batasan masalah, manfaat penelitian, serta sistematika penulisan. Bab ini menjadi dasar dalam memahami alasan dan arah penelitian yang dilakukan.

BAB II TINJAUAN PUSTAKA

Bab ini berisi kajian literatur dan landasan teori yang relevan dengan penelitian. Kajian mencakup perbandingan metode peramalan penjualan dan implementasi sistem POS berbasis *mobile*. Landasan teori menyajikan konsep peramalan, arsitektur LSTM, *framework* Flutter, bahasa Dart, serta layanan Supabase yang digunakan sebagai infrastruktur sistem.

BAB III METODOLOGI PENELITIAN

Bab ini menjelaskan tahapan metodologi penelitian yang dilakukan, meliputi jenis dan pendekatan penelitian, teknik pengumpulan data, teknik analisis data, alur penelitian, rancangan sistem menggunakan UML, teknik pengujian, serta hasil yang diharapkan.

BAB II

TINJAUAN PUSTAKA

2.1 *State Of The Art*

Penelitian terkait prediksi penjualan dan pengembangan sistem *Point of Sale* berbasis *mobile* telah mengalami perkembangan signifikan seiring meningkatnya pemanfaatan teknologi *deep learning* dan komputasi awan. Abbas dkk. [4] dan Zhang dan Liu [13] membuktikan keunggulan LSTM dibandingkan model statistik konvensional maupun arsitektur *Gated Recurrent Unit* (GRU) dalam menangkap pola non-linier pada data penjualan ritel. Choi dan Lee [6] memperluas pendekatan ini dengan menggunakan model multivariate yang menggabungkan variabel kontekstual seperti frekuensi transaksi harian. Di sisi pengembangan sistem, Setiawan dan Nugroho [1] serta Pratama dan Handayani [2] mengembangkan aplikasi POS berbasis Android untuk UMKM, namun keduanya bersifat murni deskriptif tanpa komponen prediksi analitik.

Meskipun demikian, penelitian yang secara spesifik mengintegrasikan modul prediksi penjualan berbasis LSTM langsung ke dalam sistem POS *mobile* yang beroperasi secara *offline-first* untuk konteks UMKM masih sangat terbatas. Ringkasan penelitian terdahulu tersebut disajikan dalam Tabel 2.1.

Tabel 2.1 *State Of The Art*

No	Penulis, Tahun dan Judul Artikel	Metode yang Digunakan	Hasil yang Diperoleh	Persamaan	Perbedaan
1	Abbas dkk. (2021) [4]. <i>Sales Forecasting Using LSTM Networks in Retail Industry.</i>	LSTM, ARIMA	LSTM mengungguli ARIMA pada data penjualan non-linier dengan penurunan <i>error</i> hingga 15%.	Sama-sama menggunakan LSTM untuk prediksi data penjualan berbasis deret waktu.	Model bersifat <i>standalone</i> tanpa integrasi sistem kasir nyata; penelitian ini mengintegrasikan LSTM ke dalam aplikasi POS.
2	Setiawan & Nugroho (2023) [1]. <i>Development of Cross-Platform POS Application using Flutter for SMEs.</i>	Flutter, Firebase	Aplikasi kasir stabil dan mampu mencatat transaksi secara <i>real-time</i> .	Sama-sama mengembangkan aplikasi POS berbasis Android untuk UMKM.	Aplikasi murni deskriptif tanpa fungsi prediksi analitik; penelitian ini menambahkan modul LSTM terintegrasi.
3	Martinez & Silva (2022) [7]. <i>Architecture Design of Offline-First Mobile Application using Local Embedded Database.</i>	<i>Offline-First</i> , SQLite	Data tersinkronisasi aman tanpa tumpang tindih saat kembali <i>online</i> .	Sama-sama menerapkan arsitektur <i>offline-first</i> pada aplikasi <i>mobile</i> .	Tidak menguji volume data besar dan tidak memiliki komponen AI; penelitian ini mengintegrasikan keduanya.

No	Penulis, Tahun dan Judul Artikel	Metode yang Digunakan	Hasil yang Diperoleh	Persamaan	Perbedaan
4	Zhang & Liu (2022) [13]. <i>Comparative Analysis of LSTM and GRU for Time-Series Sales Forecasting.</i>	LSTM, GRU	LSTM memberikan akurasi lebih stabil pada data tren penjualan multi-tahun.	Sama-sama membandingkan model <i>deep learning</i> untuk prediksi deret waktu penjualan.	Memerlukan daya komputasi tinggi; penelitian ini men- <i>deploy</i> model di server <i>cloud</i> terpisah agar tidak membebani perangkat.
5	Fischer & Weber (2024) [9]. <i>Evaluating Open-Source BaaS Platforms: A Case Study on Supabase and PostgreSQL.</i>	Supabase, PostgreSQL	RLS Supabase memangkas waktu pengembangan <i>backend</i> hingga 40%.	Sama-sama menggunakan Supabase sebagai infrastruktur <i>backend cloud</i> .	Belum diuji untuk performa <i>mobile dynamic sync</i> berskala harian; penelitian ini mengujinya dalam konteks UMKM.
6	Choi & Lee (2022) [6]. <i>Multivariate Time-Series Forecasting for Retail Sales Using LSTM with Economic Indicators.</i>	Multivariate LSTM	Akurasi model naik dengan menambahkan variabel kontekstual seperti hari libur.	Sama-sama menggunakan variabel transaksi harian sebagai input model LSTM.	Bergantung pada data eksternal yang sulit diakses pelaku UMKM kecil; penelitian ini menggunakan data transaksi internal saja.

No	Penulis, Tahun dan Judul Artikel	Metode yang Digunakan	Hasil yang Diperoleh	Persamaan	Perbedaan
7	Novikov & Petrov (2023) [8]. <i>Performance Comparison of Local NoSQL Embedded Databases in Flutter: Hive, ObjectBox, and Isar.</i>	Isar DB, Hive, ObjectBox	Isar DB tercepat dalam operasi <i>batch insert</i> transaksi pada perangkat Android.	Sama-sama menggunakan Isar DB sebagai <i>local storage</i> pada aplikasi Flutter.	Belum diintegrasikan ke arsitektur sinkronisasi <i>cloud</i> ; penelitian ini mengintegrasikan keduanya secara penuh.
8	Brown & Green (2021) [3]. <i>Demand Forecasting for Inventory Management in Small Retail Stores Using Neural Networks.</i>	Neural Networks	Mampu memprediksi <i>demand</i> mingguan dengan MAPE 12%.	Sama-sama menghubungkan prediksi penjualan ke manajemen stok barang.	Rentang prediksi mingguan terlalu luas; penelitian ini berfokus pada prediksi harian yang lebih granular.
9	Guntara (2022). Pengembangan Aplikasi POS <i>Mobile Berbasis Cloud Computing</i> untuk UMKM.	Android, Firebase	Meningkatkan fleksibilitas pemantauan bisnis secara <i>real-time</i> .	Sama-sama mengembangkan aplikasi POS berbasis Android untuk UMKM lokal.	Sangat bergantung koneksi internet tanpa dukungan <i>offline-first</i> dan tanpa modul prediksi penjualan.

No	Penulis, Tahun dan Judul Artikel	Metode yang Digunakan	Hasil yang Diperoleh	Persamaan	Perbedaan
10	Penelitian Ini. Rancang Bangun Aplikasi <i>Point of Sale</i> dengan Fitur Prediksi Penjualan Harian Menggunakan Metode LSTM.	LSTM, Flutter, Supabase, Isar DB	Aplikasi POS <i>mobile offline-first</i> terintegrasi dengan prediksi penjualan harian berbasis LSTM menggunakan dataset transaksi riil EatsTEDI UGM.	–	–

Berdasarkan kajian terhadap penelitian terdahulu pada Tabel 2.1, dapat disimpulkan bahwa sebagian besar penelitian prediksi penjualan menggunakan LSTM masih bersifat *standalone*, yakni tidak terintegrasi dengan sistem pencatatan transaksi yang aktif digunakan. Sebaliknya, penelitian pengembangan sistem POS *mobile* yang ada belum dilengkapi kemampuan analitik prediktif berbasis *machine learning*. Selain itu, belum ada sistem POS untuk UMKM yang secara bersamaan menerapkan arsitektur *offline-first* sekaligus menyertakan modul prediksi LSTM terintegrasi.

Oleh karena itu, penelitian ini hadir untuk mengisi celah tersebut dengan mengusulkan sistem yang menggabungkan ketiga aspek sekaligus, yaitu: (1) aplikasi POS *mobile* berbasis Flutter dengan kemampuan operasional *offline-first* menggunakan Isar DB dan sinkronisasi otomatis ke Supabase [9],[8]; (2) modul prediksi penjualan harian berbasis LSTM yang dilatih dari data transaksi riil platform EatsTEDI DTEDI Universitas Gadjah Mada [4],[6],[5]; dan (3) penyajian hasil prediksi secara langsung kepada pengguna melalui dasbor analitik di dalam aplikasi yang sama [12].

2.2 Sistem *Point of Sale* (POS)

Sistem *Point of Sale* (POS) merupakan kombinasi perangkat keras dan perangkat lunak yang digunakan untuk memproses transaksi penjualan pada titik pembelian. Dalam konteks UMKM, sistem POS berfungsi sebagai alat pencatatan transaksi, pengelolaan stok barang, dan pelaporan keuangan yang menggantikan proses manual berbasis buku catatan. Perkembangan teknologi *mobile* telah mendorong lahirnya aplikasi POS berbasis Android yang lebih fleksibel, murah, dan mudah dioperasikan oleh pelaku usaha kecil tanpa keahlian teknis khusus [2],[1].

Sistem POS modern yang dirancang untuk UMKM memerlukan pendekatan yang berbeda dibandingkan sistem ritel skala besar. Fokus utama harus terletak pada kesederhanaan alur kerja kasir, kecepatan proses transaksi, dan kemampuan beroperasi dalam kondisi konektivitas internet yang tidak stabil. Arsitektur *offline-first* menjadi solusi utama untuk memastikan kelancaran operasional, di mana data transaksi disimpan terlebih dahulu secara lokal dan disinkronkan ke *cloud* secara otomatis ketika koneksi tersedia [7].

2.3 Peramalan Penjualan (*Forecasting*)

Peramalan penjualan merupakan proyeksi teknis dari permintaan pelanggan potensial untuk periode waktu tertentu berdasarkan asumsi-asumsi yang ditetapkan. Menurut Heizer dan Render [14], peramalan yang akurat sangat krusial dalam manajemen operasi karena mempengaruhi keputusan strategis di bidang pemasaran, keuangan, dan sumber daya manusia. Terdapat empat prinsip utama dalam peramalan: peramalan jarang sekali sempurna, peramalan jangka pendek biasanya lebih akurat daripada jangka panjang, peramalan agregat lebih akurat daripada peramalan item individu, dan pemilihan metode harus disesuaikan dengan pola data (horizontal, tren, musiman, atau siklus).

Metode kuantitatif dalam peramalan mengandalkan model matematika untuk memproyeksikan data masa lalu ke masa depan. Dalam konteks ritel, tantangan

utama adalah menangani variasi musiman, yaitu fluktuasi yang berulang dalam jangka waktu kurang dari satu tahun, seperti lonjakan penjualan menjelang hari raya atau awal bulan. Pendekatan berbasis *deep learning*, khususnya LSTM, terbukti lebih unggul dibandingkan metode statistik konvensional seperti ARIMA dalam menangkap pola jangka panjang pada data penjualan harian yang bersifat nonlinier [4],[13]. Evaluasi akurasi model peramalan umumnya menggunakan kombinasi metrik MAE (*Mean Absolute Error*), RMSE (*Root Mean Square Error*), dan MAPE (*Mean Absolute Percentage Error*) secara bersamaan untuk menghindari bias pembacaan akurasi akibat lonjakan data [15]. Khusus pada data yang memuat nilai aktual mendekati nol, metrik persentase simetris sMAPE dan metrik galat terskala MASE lebih dianjurkan karena tahan terhadap pembagian dengan nol [16]. Brown dan Green [3] menunjukkan bahwa prediksi berbasis jaringan saraf tiruan dapat mencapai MAPE 12% pada data permintaan ritel kecil, sementara penggunaan pendekatan multivariate dapat meningkatkan presisi tersebut lebih jauh [6].

2.4 Arsitektur Long Short-Term Memory (LSTM)

LSTM adalah jenis khusus dari *Recurrent Neural Network* (RNN) yang pertama kali diperkenalkan oleh Hochreiter dan Schmidhuber [17] dan memiliki kemampuan untuk mempelajari ketergantungan jangka panjang dalam urutan data [4],[5]. Arsitektur LSTM dirancang untuk mengatasi masalah *vanishing gradient* yang menghambat RNN standar dalam memproses informasi yang terpisah jauh dalam satu urutan [5]. Komponen inti dari unit LSTM adalah *cell state* (keadaan sel) yang bertindak sebagai jalur informasi utama, serta tiga gerbang fungsional:

1. **Forget Gate:** Menggunakan fungsi aktivasi sigmoid untuk menentukan bagian informasi dari status sel sebelumnya yang harus dibuang.
2. **Input Gate:** Memutuskan informasi baru mana yang akan diperbarui ke dalam status sel melalui kombinasi fungsi sigmoid dan tanh.
3. **Output Gate:** Menentukan bagian dari status sel yang akan dikeluarkan

sebagai *hidden state* untuk langkah waktu berikutnya.

Persamaan matematis untuk pembaruan status sel C_t dapat dinyatakan sebagai berikut:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad 2.1$$

Di mana f_t adalah *output* dari *forget gate*, i_t adalah *output* dari *input gate*, dan $\tilde{C}_t$ adalah kandidat nilai baru yang akan ditambahkan ke dalam status sel.

Dalam praktiknya, pelatihan jaringan LSTM umumnya menggunakan algoritma optimasi **Adam** (*Adaptive Moment Estimation*), yaitu metode optimasi berbasis gradien stokastik yang menyesuaikan *learning rate* secara adaptif untuk setiap parameter dan terbukti efisien pada berbagai permasalahan *deep learning* [18]. Untuk mencegah *overfitting* pada data latih berukuran kecil, diterapkan teknik regularisasi **Dropout**, yaitu menonaktifkan sebagian neuron secara acak selama pelatihan agar jaringan tidak bergantung berlebihan pada neuron tertentu [19].

2.5 Model Pembanding Peramalan (*Baseline*)

Dalam penelitian peramalan deret waktu, kinerja sebuah model kompleks harus selalu disandingkan dengan model pembanding sederhana (*baseline*) untuk memvalidasi apakah kompleksitas tambahan benar-benar memberikan nilai tambah akurasi [20]. Beberapa *baseline* standar yang digunakan dalam penelitian ini adalah:

1. **Naive**: Nilai peramalan disamakan dengan nilai observasi terakhir yang tersedia. Metode ini menjadi tolok ukur minimum yang wajib dikalahkan oleh model yang lebih kompleks [20].
2. **Seasonal-Naive**: Nilai peramalan diambil dari observasi terakhir pada musim yang sama — misalnya penjualan hari Senin diprediksi dari rata-rata beberapa hari Senin sebelumnya — sehingga efektif menangkap pola musiman mingguan [20].
3. **Moving Average (Rata-rata Bergerak)**: Nilai peramalan berupa rata-rata

dari k observasi terakhir, berguna untuk meredam fluktuasi acak jangka pendek [14].

4. **SARIMA** (*Seasonal Autoregressive Integrated Moving Average*): Model statistik klasik dari keluarga Box–Jenkins yang memodelkan komponen autoregresif, diferensiasi, rata-rata bergerak, beserta pola musimannya secara eksplisit [21].

Temuan pada kompetisi peramalan berskala global M3 dan M4 menunjukkan bahwa metode statistik klasik dan kombinasinya kerap mengungguli metode *machine learning* murni pada deret waktu bisnis, terutama ketika data historis yang tersedia terbatas [22],[23]. Temuan tersebut menjadi dasar metodologis dalam penelitian ini untuk mengevaluasi model LSTM secara jujur terhadap *baseline* sederhana sebelum diputuskan metode mana yang layak disajikan kepada pengguna akhir.

2.6 Metrik Evaluasi Peramalan (sMAPE dan MASE)

Selain MAE dan RMSE, penelitian ini menggunakan dua metrik evaluasi tambahan yang direkomendasikan untuk deret waktu bisnis dengan skala nilai yang bervariasi [16]:

1. **sMAPE** (*Symmetric Mean Absolute Percentage Error*): Varian MAPE yang membagi galat absolut terhadap rata-rata nilai aktual dan prediksi, sehingga terhindar dari ledakan nilai persentase ketika nilai aktual mendekati nol dan memiliki batas atas galat sebesar 200%.
2. **MASE** (*Mean Absolute Scaled Error*): Metrik yang diusulkan Hyndman dan Koehler [16] untuk menormalisasi galat model terhadap galat metode naif *in-sample* pada data latih. Nilai $MASE < 1$ menandakan model lebih baik daripada metode naif, sehingga metrik ini dapat dibandingkan lintas dataset tanpa terpengaruh satuan nominal.

2.7 *Generative Adversarial Network (GAN) dan TimeGAN*

Generative Adversarial Network (GAN) adalah kerangka pembelajaran generatif yang diperkenalkan oleh Goodfellow dkk. [24], terdiri atas dua jaringan saraf yang saling berkompetisi: *generator* yang berusaha menghasilkan data tiruan semirip mungkin dengan data asli, dan *discriminator* yang berusaha membedakan data asli dari data tiruan. Melalui proses pelatihan *adversarial* ini, *generator* secara bertahap belajar mereproduksi distribusi data asli, sehingga GAN banyak dimanfaatkan untuk augmentasi data (*data augmentation*) ketika jumlah data riil terbatas.

TimeGAN (*Time-series Generative Adversarial Networks*) merupakan pengembangan GAN yang dirancang khusus untuk data deret waktu, diusulkan oleh Yoon dkk. [25] pada konferensi NeurIPS 2019. Berbeda dengan GAN konvensional yang hanya menangkap distribusi statis, TimeGAN menggabungkan kerangka *adversarial (unsupervised)* dengan pelatihan *supervised* pada ruang laten agar dinamika temporal antar-langkah waktu pada data asli ikut dipelajari. Arsitektur TimeGAN terdiri atas lima komponen jaringan: *embedder* dan *recovery* yang memetakan data ke/dari ruang laten (*autoencoder*), serta *generator*, *supervisor*, dan *discriminator* yang dilatih dalam tiga fase (pelatihan *autoencoder*, pelatihan *supervised*, dan pelatihan gabungan *adversarial*) [25]. Dalam penelitian ini, TimeGAN digunakan untuk mensintesis sekuens data penjualan harian tambahan sebagai upaya mitigasi keterbatasan ukuran dataset pada pelatihan model LSTM.

2.8 *Framework Flutter dan Bahasa Dart*

Flutter merupakan *framework* pengembangan aplikasi *cross-platform* yang dikembangkan oleh Google dan menggunakan mesin grafis Skia untuk merender antarmuka pengguna secara langsung, menghasilkan performa tinggi mendekati aplikasi *native* [1]. Bahasa pemrograman Dart yang menyertai Flutter mendukung fitur *Just-In-Time (JIT)* untuk pengembangan cepat dan *Ahead-Of-Time (AOT)*

untuk rilis produksi yang efisien. Arsitektur *widget* pada Flutter memungkinkan pengembang membangun komponen antarmuka yang modular dan dapat digunakan kembali, sangat cocok untuk aplikasi POS yang memiliki banyak elemen interaktif.

Dalam penelitian ini, Flutter dipilih sebagai *framework* utama pengembangan aplikasi *mobile* karena kemampuannya menghasilkan aplikasi Android berkinerja tinggi dari satu basis kode, serta dukungan ekosistem *package* yang kaya untuk integrasi fitur seperti koneksi Bluetooth, pemindaian barcode, dan komunikasi *REST API* [12],[1].

2.9 Layanan Supabase dan Integrasi Cloud

Supabase merupakan platform *Backend-as-a-Service* (BaaS) *open-source* yang menyediakan lapisan infrastruktur matang di atas basis data PostgreSQL [9]. Supabase mencakup fitur autentikasi pengguna, *real-time subscriptions*, penyimpanan file, dan *Row Level Security* (RLS) yang memungkinkan pengelolaan hak akses data secara granular. Fischer dan Weber [9] membuktikan bahwa penggunaan RLS pada Supabase dapat memangkas waktu pengembangan *backend* hingga 40% sekaligus mengamankan data transaksi multi-tenant. Fitur *real-time subscriptions* pada Supabase memungkinkan aplikasi *mobile* menerima pembaruan data secara instan tanpa perlu melakukan *polling* ke server.

Penggunaan Supabase dalam penelitian ini memfasilitasi penyimpanan data transaksi terpusat dalam jumlah besar sebagai muara sinkronisasi data penjualan dari perangkat kasir. Supabase juga menangani autentikasi multi-pengguna (Owner dan Kasir) dengan mekanisme *Role-Based Access Control* (RBAC) yang terimplementasi langsung di lapisan basis data, memastikan privasi data antar-merchant terjaga [26].

2.10 Isar Database (*Local Storage*)

Isar DB adalah basis data *embedded* berkinerja tinggi yang dirancang khusus untuk Flutter dan mendukung operasi baca/tulis secara *asynchronous* tanpa memerlukan koneksi internet. Novikov dan Petrov [8] membuktikan bahwa Isar DB memiliki kecepatan *batch insert* dan pembacaan data sekuensial yang paling optimal dibandingkan basis data lokal lain yang umum digunakan pada Flutter, yakni Hive dan ObjectBox. Isar DB beroperasi langsung di perangkat pengguna menggunakan format penyimpanan *binary* yang dioptimalkan untuk kecepatan tinggi, serta mendukung kueri kompleks dengan antarmuka yang intuitif.

Dalam penelitian ini, Isar DB berperan sebagai penyimpanan lokal utama pada perangkat *mobile* untuk menjamin kelancaran operasional kasir dalam kondisi *offline* [7],[8]. Seluruh transaksi yang dilakukan tanpa koneksi akan disimpan terlebih dahulu di Isar DB, kemudian disinkronkan secara otomatis ke Supabase oleh mesin *SyncNotifier* ketika koneksi internet tersedia kembali.

2.11 Python, TensorFlow, PyTorch, dan Flask

Python dipilih sebagai bahasa pemrograman untuk pengembangan modul prediksi di sisi server (*server-side*) karena ekosistemnya yang kaya dalam bidang *machine learning* dan analisis data. **TensorFlow** [27] dengan antarmuka tingkat tinggi Keras digunakan untuk membangun, melatih, dan menyimpan model LSTM [4],[5], sedangkan **PyTorch** [28] — pustaka *deep learning* imperatif dengan komputasi graf dinamis — digunakan untuk mengimplementasikan model augmentasi data sintetis TimeGAN. Proses pelatihan model dilakukan di sisi server menggunakan data transaksi historis yang telah diekstraksi, kemudian layanan prediksi di-*deploy* agar dapat dipanggil oleh aplikasi *mobile* melalui *REST API* [12], sehingga proses komputasi berat tidak membebani perangkat pengguna. Nguyen dan Tran [11] merekomendasikan pendekatan ini dalam arsitektur *microservices* untuk memastikan sistem kasir tetap stabil meskipun modul AI sedang melakukan proses pelatihan ulang di sisi server.

Layanan prediksi tersebut dibangun menggunakan **Flask**, yaitu *micro-framework* web Python yang ringan dan modular untuk membangun aplikasi web dan layanan API [29]. Komunikasi antara aplikasi *mobile* dan layanan prediksi mengikuti gaya arsitektur **REST** (*Representational State Transfer*) yang diformulasikan oleh Fielding [30], di mana setiap sumber daya diakses melalui *endpoint* HTTP yang seragam (*uniform interface*) tanpa menyimpan status sesi di server (*stateless*), dengan format pertukaran data JSON.

2.12 Layanan Firebase

Firebase merupakan platform pengembangan aplikasi dari Google yang menyediakan kumpulan layanan siap pakai di sisi *backend* untuk aplikasi *mobile* [31]. Dua layanan Firebase yang dimanfaatkan pada penelitian ini adalah **Firestore Cloud Messaging** (FCM) sebagai infrastruktur pengiriman *push notification* lintas perangkat secara *real-time* — dimanfaatkan untuk pemberitahuan seperti peringatan stok menipis dan *broadcast* pesan antar staf toko — serta **Analytics** untuk pencatatan telemetri perilaku penggunaan aplikasi (seperti volume transaksi dan pemakaian fitur) sebagai bahan evaluasi produk [31].

2.13 Metode Pengembangan Perangkat Lunak Agile

Agile adalah paradigma pengembangan perangkat lunak yang menekankan iterasi pendek, penyerahan produk fungsional secara bertahap (*incremental*), dan adaptasi cepat terhadap perubahan kebutuhan [32]. Pendekatan *iterative and incremental* membagi pengembangan menjadi siklus-siklus kecil di mana setiap siklus menghasilkan pertambahan fungsionalitas yang dapat diuji. Selanjutnya, **Lean Software Development** yang diadaptasi Poppendieck dari prinsip *lean manufacturing* menambahkan tujuh prinsip utama, antara lain menghilangkan pemborosan (*eliminate waste*), memperkuat pembelajaran, menunda keputusan hingga saat paling akhir yang bertanggung jawab, dan menyerahkan produk secepat mungkin [33]. Kombinasi pendekatan ini melandasi strategi

pengembangan sistem pada penelitian ini yang memprioritaskan *Minimum Viable Product* (MVP) fitur inti kasir sebelum fitur analitik lanjutan.

2.14 Pemodelan Sistem (UML dan ERD)

Unified Modeling Language (UML) adalah bahasa pemodelan visual standar untuk menspesifikasikan, memvisualisasikan, dan mendokumentasikan artefak sistem perangkat lunak [34]. Diagram UML yang digunakan pada penelitian ini meliputi *Use Case Diagram* (memetakan interaksi aktor terhadap fungsi sistem beserta relasi *include/extend*), *Activity Diagram* (memvisualisasikan aliran kerja dan titik keputusan proses), *Sequence Diagram* (menggambarkan urutan pertukaran pesan antar-objek berdasarkan waktu), dan *Class Diagram* (menggambarkan struktur kelas beserta relasinya) [34].

Adapun *Entity Relationship Diagram* (ERD) adalah model konseptual basis data yang diperkenalkan oleh Chen [35], yang merepresentasikan data sebagai entitas, atribut, dan relasi antar-entitas beserta kardinalitasnya. ERD digunakan dalam penelitian ini untuk merancang skema basis data relasional PostgreSQL di sisi *cloud* beserta padanan koleksi datanya di basis data lokal Isar DB.

2.15 Pengujian *Black Box*

Pengujian *black box* adalah teknik pengujian perangkat lunak yang memvalidasi fungsionalitas sistem murni dari sisi masukan dan keluaran yang teramati, tanpa memeriksa struktur internal kode program [32]. Penguji menyusun skenario uji berdasarkan spesifikasi kebutuhan fungsional, kemudian membandingkan hasil aktual sistem terhadap hasil yang diharapkan. Teknik ini efektif untuk memverifikasi kesesuaian perilaku fitur aplikasi terhadap kebutuhan pengguna akhir, dan pada penelitian ini diterapkan terhadap skenario-skenario penggunaan utama aplikasi POS oleh peran Owner dan Kasir [32].

BAB III

METODOLOGI PENELITIAN

3.1 Data dan Pengumpulan Data

3.1.1 Tempat Penelitian

Penelitian dan pengembangan sistem *Point of Sale* (POS) dengan fitur prediksi penjualan harian menggunakan metode LSTM ini dilaksanakan pada beberapa lokasi yang mendukung proses perancangan, implementasi, dan pengujian, yaitu:

1. **Jurusan Teknologi Informasi dan Komputer, Politeknik Negeri Lhokseumawe:** Menjadi lokasi utama dalam perancangan arsitektur sistem, penulisan kode program (aplikasi mobile Flutter dan REST API Python), pelatihan model LSTM, serta penyusunan laporan skripsi.
2. **Departemen Teknik Elektro dan Informatika (DTEDI), Universitas Gadjah Mada:** Sebagai penyedia dataset transaksi riil platform EatsTEDI UGM yang digunakan sebagai data latih dan data uji utama untuk mengevaluasi performa model prediksi LSTM.
3. **Pelaku UMKM di Kota Lhokseumawe:** Sebagai subjek studi kasus untuk melakukan analisis kebutuhan pengguna secara langsung dan menguji kelayakan operasional aplikasi kasir di lapangan secara nyata.

3.1.2 Waktu Penelitian

Penelitian ini dilaksanakan selama 4 (empat) bulan, terhitung mulai bulan Maret 2026 sampai dengan bulan Juni 2026. Alokasi waktu pelaksanaan untuk setiap tahapan penelitian disajikan pada Tabel 3.1.

Tabel 3.1 Jadwal Pelaksanaan Penelitian

No	Kegiatan	Maret 2026	April 2026	Mei 2026	Juni 2026
1	Persiapan dan Studi Literatur	✓			
2	Pengumpulan dan Analisis Data	✓	✓		
3	Perancangan Sistem dan UI/UX		✓		
4	Pengembangan Aplikasi POS Mobile		✓	✓	
5	Pelatihan dan Optimasi Model LSTM			✓	
6	Integrasi Sistem dan Pengujian			✓	✓
7	Evaluasi dan Penyusunan Laporan				✓

3.1.3 Alat dan Bahan

Untuk mendukung kelancaran pelaksanaan penelitian dan pengembangan sistem POS mobile, dibutuhkan alat (perangkat keras dan perangkat lunak) serta bahan penelitian dengan rincian yang disajikan pada Tabel 3.2.

Tabel 3.2 Daftar Alat dan Bahan Penelitian

No	Kategori	Nama Alat/Bahan	Spesifikasi / Deskripsi
1	Perangkat Keras (<i>Hardware</i>)	Laptop Acer Nitro 5	Intel Core i5 Gen 12, RAM 16 GB, SSD 512 GB, NVIDIA GeForce RTX 3050. Digunakan sebagai perangkat utama pengembangan aplikasi dan pelatihan model.
		Smartphone Android	OS Android 11. Digunakan untuk pengujian operasional aplikasi POS secara langsung di perangkat mobile.
		Printer Thermal	Bluetooth 58mm. Digunakan untuk menguji fungsionalitas cetak struk transaksi fisik.
2	Perangkat Lunak (<i>Software</i>)	Windows 11	Sistem Operasi Laptop (Home Single Language).
		VS Code & Android Studio	<i>Integrated Development Environment</i> (IDE) utama untuk penulisan kode program.
		Flutter SDK (Dart)	<i>Framework</i> lintas platform untuk membangun aplikasi kasir mobile.
		Python 3.10	Bahasa pemrograman dengan TensorFlow, Keras, PyTorch, Pandas, NumPy, Scikit-Learn, dan Flask untuk melatih model LSTM, membangun modul augmentasi data sintetis TimeGAN, dan membuat REST API prediksi.
		Supabase Cloud	Basis data <i>cloud</i> terpusat (PostgreSQL) untuk sinkronisasi data transaksi.
		Isar Database	Basis data lokal terbenam (<i>embedded</i>) pada perangkat Android (<i>offline-first</i>).
		Draw.io & Mermaid.js	Alat pemodelan sistem untuk membuat diagram UML, ERD, dan arsitektur.
3	Bahan Penelitian	Dataset EatsTEDI UGM	Berkas transaksi riil platform EatsTEDI DTEDI UGM dalam format SQL dump dan CSV (periode Agustus 2024 s.d. Juni 2026).

3.1.4 Variabel Data

Variabel penelitian yang digunakan dalam proses prediksi penjualan harian dikelompokkan menjadi tiga:

1. **Variabel Input (Fitur):** Deret waktu transaksi harian EatsTEDI yang terdiri atas *log-return* pendapatan, pengodean siklik (sinus–kosinus) untuk hari, minggu, dan bulan, serta penanda biner periode Ramadan. Variabel jumlah transaksi (*transactions*) dan kuantitas terjual (*qty_sold*) sengaja tidak digunakan sebagai fitur untuk menghindari kebocoran data (*data leakage*), karena keduanya belum tersedia saat memprediksi hari mendatang.
2. **Variabel Output (Target):** Nilai prediksi *log-return* pendapatan, yang direkonstruksi kembali ke nominal Rupiah sebagai acuan estimasi omzet harian dan kebutuhan stok produk.
3. **Variabel Evaluasi:** Kinerja model diukur dengan empat metrik evaluasi (MAE, RMSE, sMAPE, dan MASE). MAPE konvensional dihindari karena menghasilkan nilai tak terhingga saat dihadapkan pada hari dengan pendapatan mendekati nol. Rincian formula masing-masing metrik dijabarkan pada Subbab 3.2.10.1.

3.1.5 Sumber Data

Data utama yang digunakan dalam penelitian ini adalah basis data transaksi penjualan harian yang bersumber dari platform katalog menu digital interaktif **EatsTEDI** pada Departemen Teknik Elektro dan Informatika (DTEDI/TRE) Universitas Gadjah Mada (UGM). *Dataset* ini berupa *dump* basis data relasional dalam format SQL (*eatstedi-20260621-010820.sql*) berukuran 32,15 MB dengan total sekitar 395.090 baris data.

Rentang waktu transaksi riil yang digunakan adalah dari tanggal **24 Agustus 2024 s.d. 20 Juni 2026** (mencakup periode 666 hari atau sekitar 22 bulan). Data yang diekstrak berfokus pada transaksi dengan status lunas (*is_paid* = 1) dari tabel *invoices* serta rincian item produk terjual dari tabel *product_sold* yang dihubungkan dengan tabel master *products* dan *categories*.

Secara umum, karakteristik data deret waktu (*time series*) EatsTEDI UGM memiliki pola musiman unik yang sangat dipengaruhi oleh kalender akademis kampus, antara lain:

1. **Pola Mingguan (*weekly profile*):** Penjualan aktif hampir 100% terkonsentrasi pada hari kerja (Senin s.d. Jumat) dengan rata-rata 195,41 transaksi per hari aktif. Sedangkan pada hari Sabtu dan Minggu, transaksi bernilai Rp 0 karena kantin kampus tutup.
2. **Pola Libur Akademis (*academic holiday profile*):** Pada masa libur semester mahasiswa (sepanjang bulan Januari dan Juli), transaksi bernilai Rp 0 karena aktivitas perkuliahan dan operasional kantin berhenti total.
3. **Pola Bulan Ramadan:** Omzet harian merosot tajam secara signifikan karena penutupan atau pembatasan jam operasional kantin selama bulan puasa (misalnya total omzet Maret 2025 hanya Rp 1.427.000,00 dan Maret 2026 sebesar Rp 3.531.000,00).

Selama periode transaksi tersebut, tercatat sebanyak **61.749 transaksi** lunas dengan total produk terjual sebanyak **150.709 unit** dan menghasilkan total akumulasi omzet sebesar **Rp 501.440.492,00**. Rata-rata nominal pendapatan harian pada hari aktif adalah sebesar **Rp 1.586.837,00/hari** dengan kuantitas produk terjual rata-rata **476,93 unit/hari**. Statistik di atas dihitung dari seluruh hari kalender dalam rentang tersebut termasuk hari non-aktif (Rp 0). Setelah penyaringan hanya pada hari kerja aktif (tanpa akhir pekan dan libur semester), analisis model pada Bab IV menggunakan **313 hari aktif** dengan total omzet sebesar **Rp 471.200.494,00** — selisih ini wajar karena hari kalender non-aktif dikeluarkan dari dataset pelatihan.

Selain statistik agregat di atas, sebaran penjualan berdasarkan kategori produk pada platform EatsTEDI disajikan pada Tabel 3.3.

Tabel 3.3 Sebaran Penjualan Berdasarkan Kategori Produk

No	Kategori	Total Penjualan (Rp)	Total Unit	Kontribusi (%)
1	Makanan Basah	242.770.500,00	82.134	48,42%
2	Minuman	82.122.500,00	16.861	16,38%
3	Makanan Kering	41.969.500,00	10.361	8,37%
4	Ice Cream	33.802.500,00	10.942	6,74%
5	Snacks	26.671.500,00	10.847	5,32%
6	Merch	496.000,00	110	0,10%

Berdasarkan sebaran tersebut, kategori **Makanan Basah** menyumbang kontribusi terbesar yakni sebesar 48,42% dari total akumulasi omzet. Karena sifat produk makanan basah yang memiliki masa simpan sangat singkat (*shelf life* pendek) dan cepat basi, ketidakpastian permintaan harian dapat berakibat langsung pada kerugian usaha akibat produk sisa yang terbuang. Oleh karena itu, peramalan penjualan harian dalam penelitian ini diprioritaskan pada kategori tersebut untuk membantu pelaku usaha dalam mengoptimalkan persediaan barang.

Adapun sepuluh produk terlaris berdasarkan kuantitas penjualan harian pada platform EatsTEDI dirinci pada Tabel 3.4.

Tabel 3.4 Daftar 10 Produk Terlaris EatsTEDI UGM

No	Nama Produk	Kuantitas Terjual (Unit)	Total Omzet (Rp)
1	Tahu Bakso	23.097	57.742.500,00
2	Tahu Bakso Balado	16.688	42.020.500,00
3	Risol Mayo & Sosis	8.489	21.717.000,00
4	Cheese/Banana Roll	3.417	3.417.000,00
5	Larist Air Mineral	3.020	12.080.000,00
6	Sandwich Crispy	2.796	11.184.000,00
7	Piscok	2.524	7.572.000,00
8	Ayam Krispi	2.495	6.237.500,00
9	Nasi Rames/Uduk	2.382	11.910.000,00
10	Donat	2.233	5.582.500,00

Adapun data dikumpulkan melalui tiga saluran utama:

1. **Observasi dan Wawancara:** Dilakukan terhadap pengelola divisi kewirausahaan mahasiswa platform EatsTEDi di Departemen Teknik Elektro dan Informatika UGM untuk memahami alur operasional kasir fisik, pencatatan menu harian, status stok, serta tantangan dalam mengantisipasi sisa stok makanan.
2. **Dokumentasi Transaksi:** Mengunduh dan mengekstrak basis data transaksi digital EatsTEDi dari file *dump* SQL untuk tabel *invoices*, *product_sold*, *products*, dan *categories* sebagai data latihan (*training data*) dan data uji (*testing data*) model prediksi LSTM.
3. **Studi Pustaka:** Mengkaji referensi jurnal ilmiah, buku manajemen operasi (Heizer & Render), serta dokumentasi teknis *framework* Flutter, TensorFlow LSTM, dan layanan Supabase sebagai landasan implementasi.

3.1.5.1 Ekstraksi dan Agregasi Data (Format CSV)

Proses transformasi data mentah dari basis data relasional *dump* SQL menjadi berkas terstruktur siap latih disebut sebagai proses ***Extract, Transform, and Load* (ETL)** atau **Ekstraksi dan Agregasi Data**. Tahapan ini dilakukan untuk mengubah granularitas data dari log transaksi individu (tingkat item terjual) menjadi deret waktu pendapatan harian, mingguan, dan bulanan yang sesuai dengan karakteristik input jaringan saraf LSTM.

Proses ekstraksi ini menghasilkan empat berkas dataset utama sebagai berikut:

1. **raw_transactions.csv:** Berisi log transaksi gabungan hasil *join* tabel *invoices*, *product_sold*, *products*, dan *categories*.
2. **daily_sales.csv:** Hasil agregasi harian yang menjadi dataset utama untuk model peramalan harian.
3. **weekly_sales.csv:** Hasil agregasi mingguan untuk visualisasi tren mingguan.
4. **monthly_sales.csv:** Hasil agregasi bulanan untuk analisis musiman jangka panjang.

Struktur skema data dan deskripsi kolom dari berkas `daily_sales.csv` sebagai dataset utama pelatihan model LSTM dirinci pada Tabel 3.5.

Tabel 3.5 Struktur Skema Dataset `daily_sales.csv`

Nama Kolom	Tipe Data	Deskripsi
date	Date	Tanggal transaksi (format YYYY-MM-DD).
revenue	Float	Total omzet pendapatan harian (nominal Rupiah).
transactions	Integer	Total volume transaksi/nota lunas harian.
qty_sold	Integer	Total kuantitas unit produk terjual harian.
day_of_week	Integer	Indeks hari aktif (0 = Senin s.d. 4 = Jumat).
week_of_year	Integer	Nomor minggu dalam setahun berdasarkan standar ISO.
month	Integer	Nomor bulan kalender (1 s.d. 12).
is_weekend	Integer	Penanda biner akhir pekan (1 jika Sabtu/Minggu, 0 jika lainnya).
is_holiday	Integer	Penanda biner libur akademik (1 jika Januari/Juli, 0 jika aktif).
is_ramadan	Integer	Penanda biner bulan Ramadan (1 jika Ramadan, 0 jika lainnya).

Sementara itu, skema data untuk berkas agregasi mingguan `weekly_sales.csv` dan agregasi bulanan `monthly_sales.csv` masing-masing dijelaskan pada Tabel 3.6 dan Tabel 3.7.

Tabel 3.6 Struktur Skema Dataset `weekly_sales.csv`

Nama Kolom	Tipe Data	Deskripsi
year_week	String	Periode tahun dan nomor minggu ISO (format YYYY-WW).
week_start	Date	Tanggal awal hari Senin pada minggu tersebut (format YYYY-MM-DD).
revenue	Float	Total omzet pendapatan mingguan (nominal Rupiah).
transactions	Integer	Total volume transaksi/nota lunas mingguan.
qty_sold	Integer	Total kuantitas unit produk terjual mingguan.
active_days	Integer	Jumlah hari aktif beroperasi dalam minggu tersebut.

Tabel 3.7 Struktur Skema Dataset `monthly_sales.csv`

Nama Kolom	Tipe Data	Deskripsi
year_month	String	Periode tahun dan bulan kalender (format YYYY-MM).
revenue	Float	Total omzet pendapatan bulanan (nominal Rupiah).
transactions	Integer	Total volume transaksi/nota lunas bulanan.
qty_sold	Integer	Total kuantitas unit produk terjual bulanan.
active_days	Integer	Jumlah hari aktif beroperasi dalam bulan tersebut.

3.1.6 Analisis Kebutuhan Data

Analisis kebutuhan data dilakukan untuk mengidentifikasi karakteristik dan kebutuhan pemrosesan data agar model prediksi LSTM dan aplikasi POS dapat berintegrasi secara optimal:

1. **Volume Data Minimum (*Cold Start Mitigation*):** Model LSTM memerlukan data runtun waktu yang kontinu. Ditetapkan batas minimum data transaksi historis sebanyak 30 hari aktif transaksi sebelum sistem dapat melakukan estimasi prediksi harian yang andal untuk meminimalkan masalah *cold start* pada merchant baru.
2. **Pembersihan dan Imputasi Data:** Karena data mentah transaksi sering kali memiliki hari luring atau libur, diperlukan aturan pembersihan data. Hari non-aktif (Sabtu, Minggu, dan libur semester) disaring agar tidak mengacaukan analisis pola musiman bisnis harian.
3. **Keamanan Data (Minimalisasi Data):** *Payload* yang dikirimkan ke layanan REST API prediksi secara desain hanya berisi data agregat numerik penjualan (total omzet harian, kuantitas terjual, dan indeks waktu kalender) tanpa menyertakan informasi identitas pelanggan seperti nama, nomor telepon, maupun detail pembayaran spesifik, sehingga privasi pelanggan tetap terjaga.
4. **Struktur Payload Data API:** Data runtun waktu harus dikirimkan dalam format terstruktur JSON melalui protokol HTTPS. Payload berisi larik

nominal omzet harian 7 hari terakhir beserta indeks waktu kalender yang relevan.

3.1.7 Preprocessing Data

Data transaksi yang terkumpul dari basis data EatsTEDI melalui beberapa tahap penyiapan berikut sebelum digunakan untuk pelatihan model:

1. **Pembersihan dan Penyaringan Hari Aktif:** Menghapus data duplikat dan menyelaraskan format tanggal, serta menyaring hari non-aktif bernilai Rp 0 (Sabtu, Minggu, dan libur semester pada Januari dan Juli). Penyaringan ini mencegah *gap* panjang pada deret waktu yang dapat mengaburkan pola musiman harian.
2. **Transformasi Target:** Nilai pendapatan ditransformasikan ke skala logaritma untuk menstabilkan variansi, kemudian diubah menjadi bentuk *log-return* (selisih logaritmik antar hari aktif) agar model mempelajari deviasi pendapatan relatif, bukan nilai nominal mentah.
3. **Normalisasi:** Seluruh fitur masukan diskalakan ke rentang $[0,1]$ menggunakan *Min-Max Scaling*, dengan *fitting* parameter *scaler* hanya pada data latih untuk mencegah kebocoran informasi (*data leakage*).
4. **Pembentukan Sequence (Windowing):** Data diubah ke format *input-label* menggunakan *sliding window* sepanjang 7 hari kerja aktif (*LOOK_BACK* = 7) untuk memprediksi penjualan hari aktif berikutnya.
5. **Pembagian Data (Time-Based Split):** Dataset dibagi secara kronologis agar urutan temporal terjaga dan tidak terjadi kebocoran data masa depan ke masa lalu.

Rincian teknis rumus *log-return*, *cyclical encoding*, dan arsitektur model dijabarkan secara lengkap pada Subbagian 3.2.8.

3.2 Rancangan Sistem

Perancangan sistem dalam penelitian ini dilakukan menggunakan pendekatan arsitektur terdistribusi dan pemodelan *Unified Modeling Language* (UML) untuk memberikan gambaran yang jelas mengenai arsitektur sistem, alur proses, dan interaksi pengguna. Pemodelan rancangan sistem direpresentasikan melalui arsitektur sistem, *Use Case Diagram*, *Activity Diagram*, serta *Entity Relationship Diagram* (ERD).

3.2.1 Metode Penelitian

Penelitian ini merupakan jenis penelitian pengembangan (*Research and Development*) dengan pendekatan kuantitatif untuk pengujian model prediksi dan kualitatif untuk analisis kebutuhan pengguna. Model pengembangan yang digunakan adalah *Agile (Tangkas)* dengan pendekatan kombinasi antara *Iterative (Berulang)* dan *Incremental (Bertahap)*, serta mengadopsi prinsip *Lean Software Development*. Metode ini diterapkan melalui fase-fase berikut:

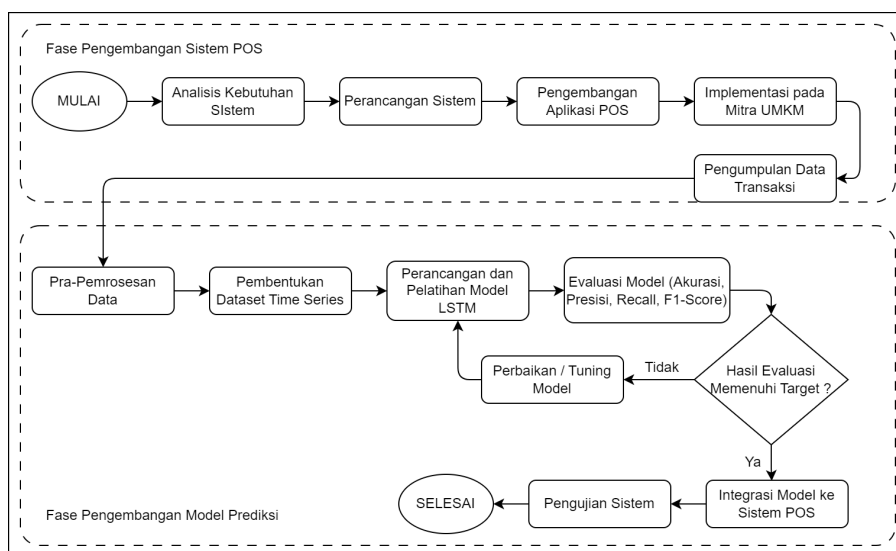
1. **Fokus pada *Minimum Viable Product (MVP)*:** Pengembangan difokuskan pada penyelesaian fitur inti sistem kasir terlebih dahulu (*Core POS*, autentikasi pengguna, manajemen produk, dan integrasi *hardware* printer) agar produk minimal dapat segera berfungsi untuk mempercepat waktu rilis ke pasar (*time-to-market*). Fitur lanjutan seperti analisis cerdas (*Smart Analytics*) dan notifikasi *push* direncanakan pada fase pasca-MVP.
2. **Pengembangan Berbasis Sesi/Iterasi Pendek (*sprints*):** Proses pengembangan dipecah menjadi siklus iterasi pendek dalam bentuk sesi-sesi harian terfokus. Setiap sesi ditujukan untuk menambahkan fungsionalitas baru (*increment*) secara bertahap atau melakukan optimasi spesifik, seperti perbaikan antarmuka dasbor, optimasi navigasi *sidebar*, dan penataan ulang modul *database*.
3. **Refaktorisasi Kode dan Adaptasi Perubahan:** Pengembangan bersifat adaptif dan responsif terhadap kendala teknis atau perubahan kebutuhan yang ditemui di tengah proses pengembangan. Siklus mencakup proses

refaktorisasi kode mengikuti pembaruan standar *framework* Flutter, penyelarasan skema *database* (*schema alignment*), optimasi sinkronisasi data lokal-*cloud*, serta mitigasi kebocoran memori (*memory leaks*).

4. **Uji Coba Berulang (*Iterative Testing*)**: Melakukan pengujian fungsionalitas menggunakan metode *Black Box Testing* secara berkala untuk setiap penambahan fitur baru untuk memastikan integritas dan stabilitas aplikasi kasir serta modul prediksi penjualan berbasis LSTM tetap terjaga.

3.2.1.1 Alur Penelitian

Alur penelitian ini dirancang secara sistematis untuk membangun sistem *Point of Sale* (POS) berbasis *mobile* yang dilengkapi fitur prediksi penjualan harian menggunakan metode LSTM. Sebagaimana diilustrasikan pada Gambar 3.1, tahapan penelitian dibagi menjadi dua fase utama yang saling berkaitan.



Gambar 3.1 Alur Penelitian

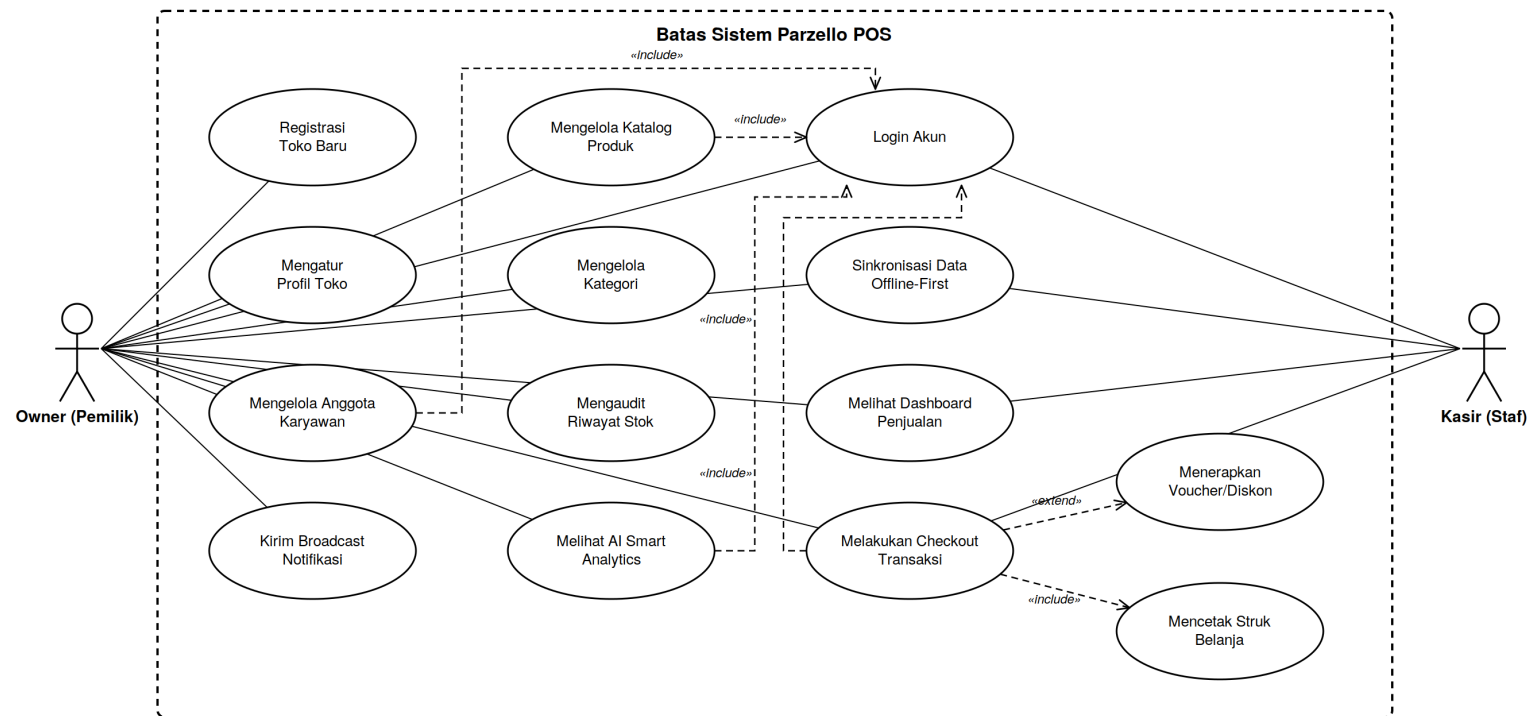
Pada **Fase Pengembangan dan Implementasi Sistem POS**, penelitian diawali dengan analisis kebutuhan melalui observasi langsung dan identifikasi permasalahan pada mitra EatsTEDi. Selanjutnya dilakukan perancangan sistem meliputi arsitektur *offline-first*, desain basis data relasional (Supabase PostgreSQL dan Isar DB), serta perancangan *Use Case Diagram*, *Activity Diagram*, ERD, dan

antarmuka pengguna. Setelah tahap perancangan, sistem POS dikembangkan menggunakan Flutter sebagai *frontend*, Supabase sebagai basis data *cloud*, Isar DB sebagai basis data lokal, dan layanan prediksi penjualan Python berbasis Flask yang di-*deploy* pada platform Hugging Face untuk pengolahan modul AI. Aplikasi yang telah dibangun kemudian diimplementasikan pada mitra EatsTEDi untuk digunakan dalam kegiatan transaksi harian, sehingga menghasilkan data transaksi aktual yang tersimpan secara otomatis.

Pada **Fase Pengembangan Model Prediksi Penjualan**, data transaksi yang telah terkumpul selanjutnya melalui tahap pra-pemrosesan meliputi pembersihan data, agregasi harian, normalisasi, dan pembentukan data deret waktu (*time series*). *Dataset* kemudian dibagi menjadi data latih dan data uji. Model LSTM dilatih menggunakan data latih untuk mempelajari pola penjualan jangka pendek maupun jangka panjang. Setelah pelatihan, model dievaluasi menggunakan metrik regresi deret waktu, yaitu MAE, RMSE, sMAPE, dan MASE, kemudian diintegrasikan ke dalam sistem POS sehingga aplikasi mampu menampilkan hasil prediksi penjualan harian kepada pengguna melalui *REST API*.

3.2.2 Diagram Use Case

Sistem POS melibatkan dua aktor utama, yaitu **Owner (Pemilik Toko)** dan **Kasir (Karyawan)**. *Use Case Diagram* menggambarkan fungsi-fungsi yang dapat diakses oleh masing-masing aktor. Pembatasan hak akses berbasis peran (*Role-Based Access Control*/RBAC) ditegakkan pada lapisan navigasi aplikasi (*router.dart*), di mana hanya rute Manajemen Karyawan dan Pengaturan Profil Toko yang dibatasi eksklusif untuk Owner, sedangkan fungsi operasional lainnya dapat diakses oleh seluruh staf toko. Rancangan *use case* diilustrasikan pada Gambar 3.2.



Gambar 3.2 *Use Case Diagram* Sistem POS

1. Definisi Aktor

Definisi aktor pada sistem POS diilustrasikan pada Tabel 3.8 berikut:

Tabel 3.8 Definisi Aktor

No.	Aktor	Deskripsi
1	Owner (Pemilik)	Aktor dengan hak akses penuh terhadap seluruh sistem. Hak eksklusif Owner pada lapisan navigasi adalah manajemen data karyawan dan pengaturan identitas toko. Owner juga menjadi penanggung jawab utama konfigurasi toko dan pengambilan keputusan strategis berbasis laporan serta fitur AI <i>Smart Analytics</i> .
2	Kasir (Karyawan)	Staf operasional harian toko dengan akses mencakup pelayanan penjualan (POS), pencetakan struk, manajemen produk dan kategori, audit riwayat stok, <i>dashboard</i> , laporan penjualan, <i>broadcast</i> notifikasi, serta AI <i>Smart Analytics</i> — kecuali manajemen karyawan dan pengaturan profil toko yang tetap eksklusif Owner.

2. Definisi Use Case

Definisi *use case* pada sistem POS diilustrasikan pada Tabel 3.9 berikut:

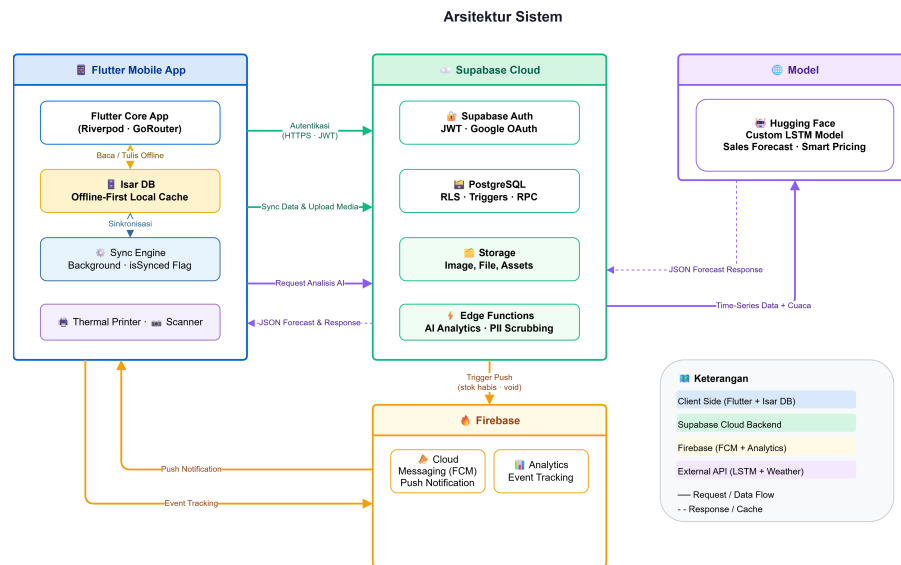
Tabel 3.9 Definisi Use Case

No.	Use Case	Deskripsi
1	Registrasi Toko Baru	Mendaftarkan akun toko baru saat inisialisasi awal sistem (Owner).
2	Login Akun	Autentikasi pengguna menggunakan email dan kata sandi via Supabase Auth.
3	Melakukan <i>Checkout</i>	Memasukkan produk ke keranjang dan memproses transaksi pembayaran.
4	Menerapkan <i>Voucher/Diskon</i>	Memasukkan kode promosi untuk memotong total tagihan transaksi.
5	Melakukan <i>Split Bill</i>	Memecah pembayaran satu nota belanja menjadi beberapa bagian tagihan, terintegrasi di lembar keranjang dan layar pembayaran.

6	Mencetak Struk	Mencetak struk transaksi fisik via printer <i>thermal Bluetooth</i> .
7	Memonitor Status Meja	Memantau keterisian meja secara visual, terintegrasi di dalam alur POS.
8	Mengelola Katalog Produk	Menambah, menyunting, dan menghapus produk/stok barang.
9	Mengelola Kategori	Mengatur pengelompokan menu katalog produk toko.
10	Mengaudit Riwayat Stok	Melihat <i>log</i> mutasi stok untuk keperluan audit inventaris.
11	Melihat <i>Dashboard</i>	Memantau performa keuangan dan statistik penjualan toko.
12	Melihat <i>AI Smart Analytics</i>	Mengakses proyeksi penjualan berbasis Model LSTM beserta riwayat hasil analitik sebelumnya.
13	Kirim <i>Broadcast</i> Notifikasi	Mengirim pemberitahuan <i>real-time</i> ke staf toko.
14	Sinkronisasi Data	Menyinkronkan transaksi <i>offline</i> lokal ke <i>cloud</i> Supabase secara otomatis.
15	Mengelola Karyawan	Mendaftarkan atau mengubah peran staf kasir (Owner).
16	Mengatur Profil Toko	Mengubah nama toko, alamat, dan logo outlet (Owner).

3.2.3 Arsitektur Sistem

Sistem Parzello POS Mobile dirancang menggunakan pendekatan arsitektur *hybrid-cloud* yang membagi pemrosesan antara perangkat lokal dan infrastruktur *cloud*. Secara keseluruhan, sistem terdiri dari empat komponen utama yang saling terintegrasi, yaitu *Flutter Mobile App*, *Supabase Cloud*, *Firebase*, dan *Layanan Inferensi Model LSTM* (Hugging Face). Rancangan arsitektur sistem diilustrasikan pada Gambar 3.3.



Penjelasan mengenai empat komponen utama dalam arsitektur sistem adalah sebagai berikut:

1. **Sisi Klien (*Client-Side*):** Aplikasi dibangun menggunakan *framework* Flutter dengan manajemen *state* Riverpod dan navigasi GoRouter. Komponen inti dari sisi klien adalah Isar DB, sebuah database NoSQL lokal yang berperan sebagai *cache offline-first*, memungkinkan kasir tetap dapat melakukan transaksi penuh meskipun koneksi internet terputus. *Sync Engine* kemudian bekerja di latar belakang untuk menyinkronkan data lokal ke *cloud* secara otomatis ketika koneksi terdeteksi kembali aktif, menggunakan penanda *isSynced* pada setiap *record* untuk menghindari duplikasi data. Selain itu, sisi klien juga mengelola integrasi perangkat keras berupa printer *thermal* Bluetooth/USB untuk mencetak struk dan kamera untuk pemindaian *barcode* produk.
2. **Infrastruktur Cloud (Supabase):** Sebagai tulang punggung infrastruktur *cloud*, Supabase menyediakan empat layanan utama. Supabase Auth menangani autentikasi pengguna melalui protokol JWT dan Google OAuth. Supabase PostgreSQL menjadi basis data relasional terpusat yang dilengkapi kebijakan *Row Level Security* (RLS) untuk mengisolasi data antar *merchant*,

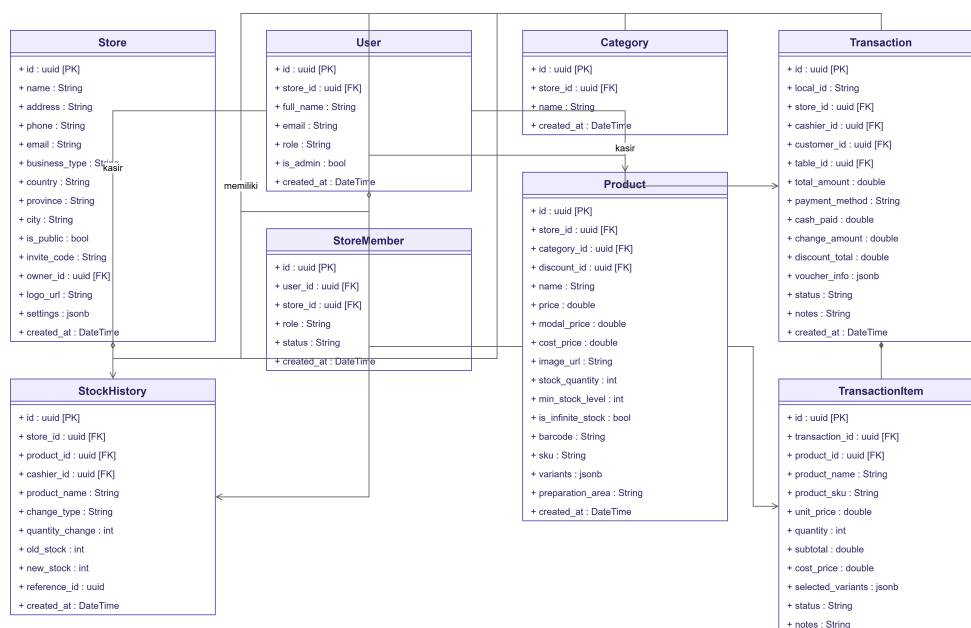
pemicu database (*triggers*), serta prosedur tersimpan (*RPC*) untuk memproses transaksi secara aman. Supabase Storage menyimpan aset media seperti gambar produk dan logo toko. Sedangkan *Edge Functions* digunakan untuk memproses *webhook* basis data yang bersifat sensitif, seperti pemicu pengiriman *push notification* peringatan stok menipis kepada pemilik toko.

3. **Telemetri dan Notifikasi (Firebase):** Untuk kebutuhan telemetri dan notifikasi, sistem terintegrasi dengan Firebase. *Firebase Cloud Messaging* (FCM) bertugas menyampaikan *push notification* secara *real-time* kepada pemilik toko, misalnya peringatan stok habis atau pembatalan transaksi oleh kasir. *Firebase Analytics* digunakan untuk mencatat perilaku penggunaan aplikasi, seperti volume transaksi harian, penggunaan fitur *split bill*, dan inisiasi kalibrasi model AI.
4. **Layanan Inferensi Model LSTM (Hugging Face):** Komponen terakhir adalah layanan inferensi untuk mendukung fitur analitik prediktif. Pada penelitian ini, dikembangkan sebuah model *deep learning* berbasis **Long Short-Term Memory (LSTM)** secara mandiri, yang dirancang khusus untuk memproyeksikan data runtun waktu (*time-series*) penjualan harian pada usaha berskala UMKM. Model peramalan tersebut kemudian dikemas dalam layanan prediksi penjualan berbasis *REST API* (*framework* Flask) yang di-*deploy* pada platform Hugging Face, sehingga dapat diakses oleh sistem secara efisien tanpa memerlukan infrastruktur *server* komputasi tersendiri. Ketika pengguna mengakses fitur *Smart Analytics*, aplikasi Flutter memanggil *endpoint* layanan prediksi tersebut secara langsung melalui protokol HTTPS (dikonfigurasi pada berkas `env.dart` dan dapat dioverride saat *build* melalui `-dart-define`), dengan *payload* berupa data agregat penjualan tanpa informasi identitas pelanggan. Model memproses data tersebut dan mengembalikan hasil prediksi dalam format JSON, mencakup estimasi omzet harian, prediksi produk terlaris, serta rekomendasi promosi berbasis stok, yang selanjutnya divisualisasikan melalui antarmuka grafik interaktif menggunakan pustaka `fl_chart`. Aplikasi juga menangani

kondisi *cold start*, yaitu menampilkan peringatan pemuatan ketika *server* inferensi baru aktif kembali dari kondisi *idle*. Setiap hasil analisis disimpan sebagai *snapshot* pada tabel *smart_analytics_snapshots* di Supabase, sehingga riwayat analitik dapat ditinjau kembali tanpa perlu memanggil ulang model.

3.2.4 Class Diagram

Class Diagram menggambarkan struktur sistem dari segi pendefinisian kelas-kelas yang akan dibangun beserta atribut dan metodenya, serta hubungan relasi asosiasi antar-kelas. Pada aplikasi Parzello POS Mobile, rancangan kelas didasarkan pada koleksi basis data lokal Isar DB yang digunakan untuk mendukung penyimpanan *offline-first*. Struktur kelas-kelas ini disajikan pada Gambar 3.4.



Gambar 3.4 Class Diagram Sistem POS

Penjelasan mengenai kelas-kelas utama dalam sistem adalah sebagai berikut:

1. **Store:** Merepresentasikan informasi toko/outlet, memiliki relasi satu-ke-banyak dengan Kategori, Produk, dan Transaksi.
2. **Category:** Mengelompokkan produk ke dalam kategori tertentu.

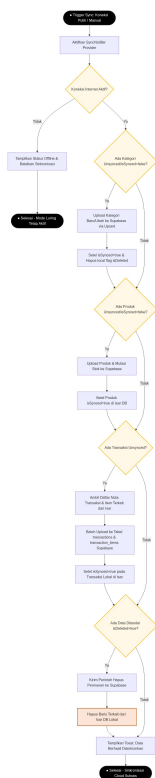
3. **Product:** Menyimpan informasi detail produk termasuk nama, harga jual, harga modal, dan stok.
4. **TransactionLocal:** Menyimpan rangkuman transaksi penjualan kasir di perangkat klien.
5. **TransactionItemLocal:** Menyimpan baris item produk detail dari setiap transaksi belanja.

3.2.5 Activity Diagram

Activity Diagram merupakan gambaran visual dalam bentuk diagram yang digunakan untuk menggambarkan serangkaian aktivitas atau proses yang terjadi di dalam sistem. Diagram ini memudahkan dalam menjelaskan logika prosedural, proses bisnis, serta alur kerja sistem Parzello POS Mobile yang dilengkapi dengan fitur prediksi penjualan harian. Adapun *Activity Diagram* pada sistem ini dijelaskan sebagai berikut.

1. Activity Diagram Alur Kerja Transaksi Kasir (POS Checkout)

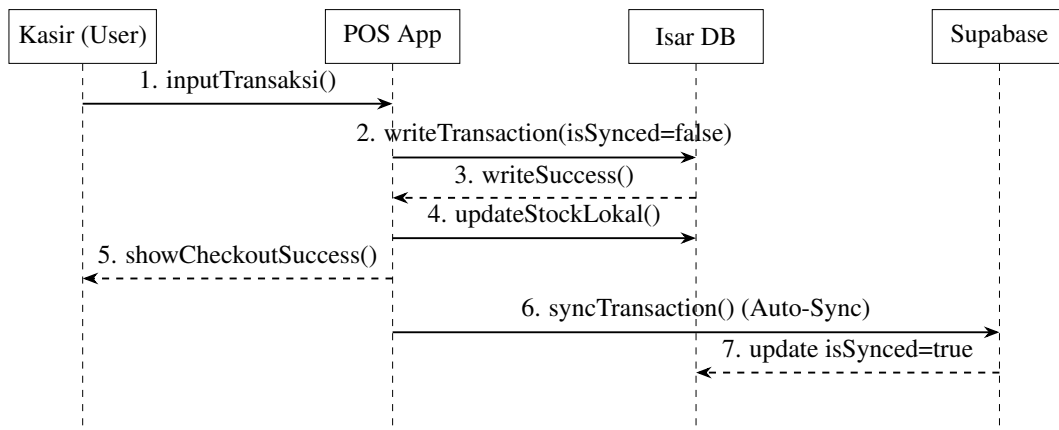
Gambar 3.5 menggambarkan alur kerja kasir atau *owner* dalam memproses transaksi belanja pelanggan di kasir, termasuk opsi penerapan kupon/voucher diskon dan pemisahan tagihan (*split bill*) yang terintegrasi langsung di dalam lembar keranjang belanja dan layar pembayaran, sebelum data disimpan di penyimpanan lokal Isar DB.



Gambar 3.6 Activity Diagram Sinkronisasi Latar Belakang

3. Activity Diagram Proses AI Smart Analytics

Gambar 3.7 menggambarkan alur kerja fitur analitik prediktif: pengguna membuka layar *Smart Analytics*, sistem mengagregasi data penjualan historis dari Isar DB lokal, lalu memanggil *endpoint* model LSTM yang di-*hosting* di Hugging Face secara langsung melalui HTTPS. Selama pemrosesan, sistem menampilkan status pemuatan (termasuk peringatan *cold start* bila *server* inferensi baru aktif dari kondisi *idle*), kemudian merender visualisasi analitik omzet beserta rekomendasi promosi, dan menyimpan hasilnya sebagai *snapshot* riwayat analitik di tabel *smart_analytics_snapshots*.



Gambar 3.8 Sequence Diagram Alur Transaksi Kasir

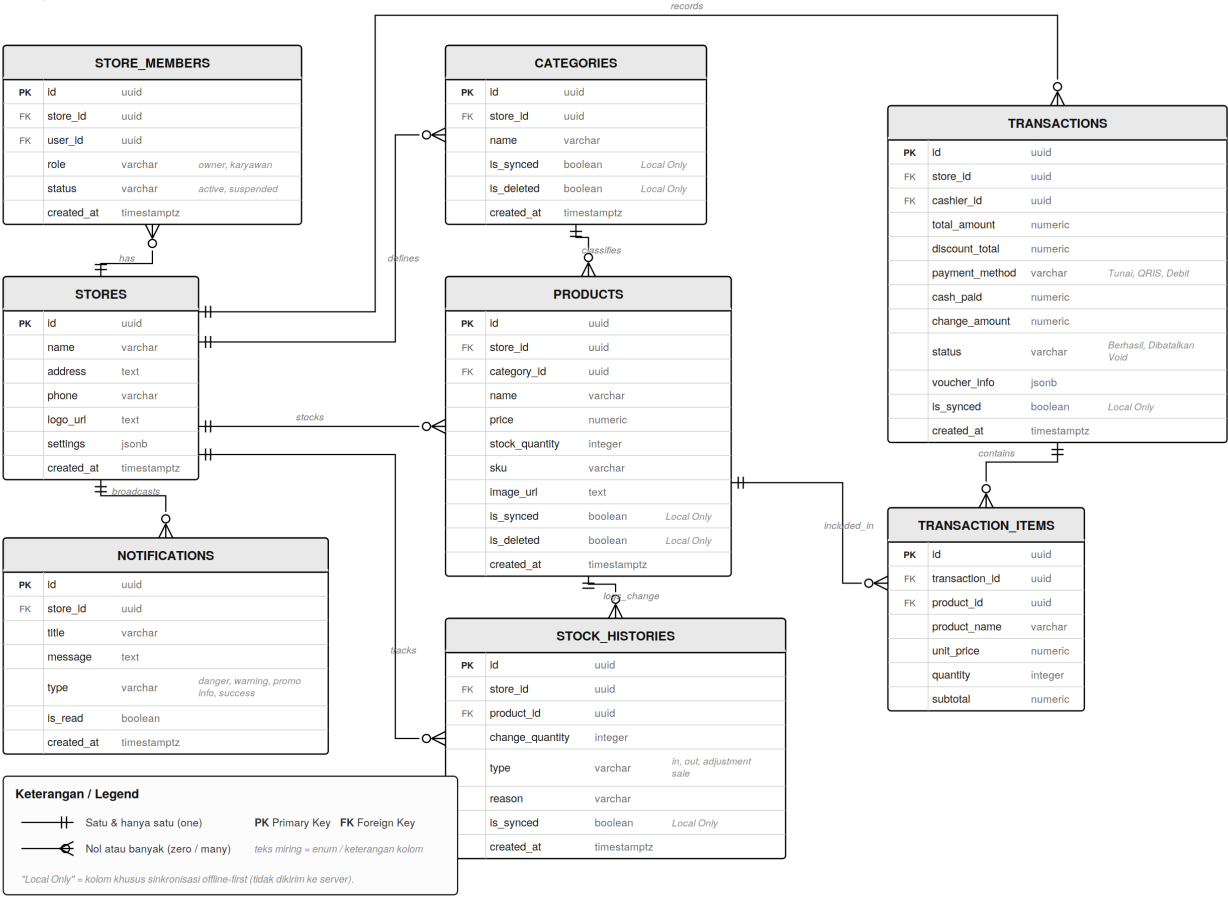
Penjelasan alur urutan transaksi pada diagram di atas adalah sebagai berikut:

1. Kasir memasukkan transaksi penjualan melalui antarmuka aplikasi POS (`inputTransaksi()`).
2. Aplikasi menyimpan data transaksi secara lokal ke dalam Isar DB dengan status belum tersinkronisasi (`writeTransaction(isSynced=false)`).
3. Isar DB mengembalikan respon sukses penulisan data transaksi.
4. Aplikasi melakukan pembaruan stok produk secara lokal pada Isar DB.
5. Aplikasi menampilkan informasi checkout sukses ke layar kasir.
6. *Sync Engine* mendeteksi status internet aktif dan menyinkronkan transaksi ke cloud Supabase melalui pemanggilan RPC `create_transaction_v4`.
7. Supabase mengembalikan respon sukses, lalu *Sync Engine* memperbarui status transaksi lokal menjadi `isSynced=true`.

3.2.7 Entity Relationship Diagram (ERD)

Basis data sistem POS dirancang secara hibrida menggunakan Supabase PostgreSQL untuk penyimpanan *cloud* dan Isar DB untuk penyimpanan lokal di perangkat *mobile*. Skema basis data *cloud* terdiri dari sembilan entitas utama yang saling berelasi sebagaimana diilustrasikan pada Gambar 3.9.

Entity Relationship Diagram — Parzello POS



Gambar 3.9 Entity Relationship Diagram (ERD) Sistem POS

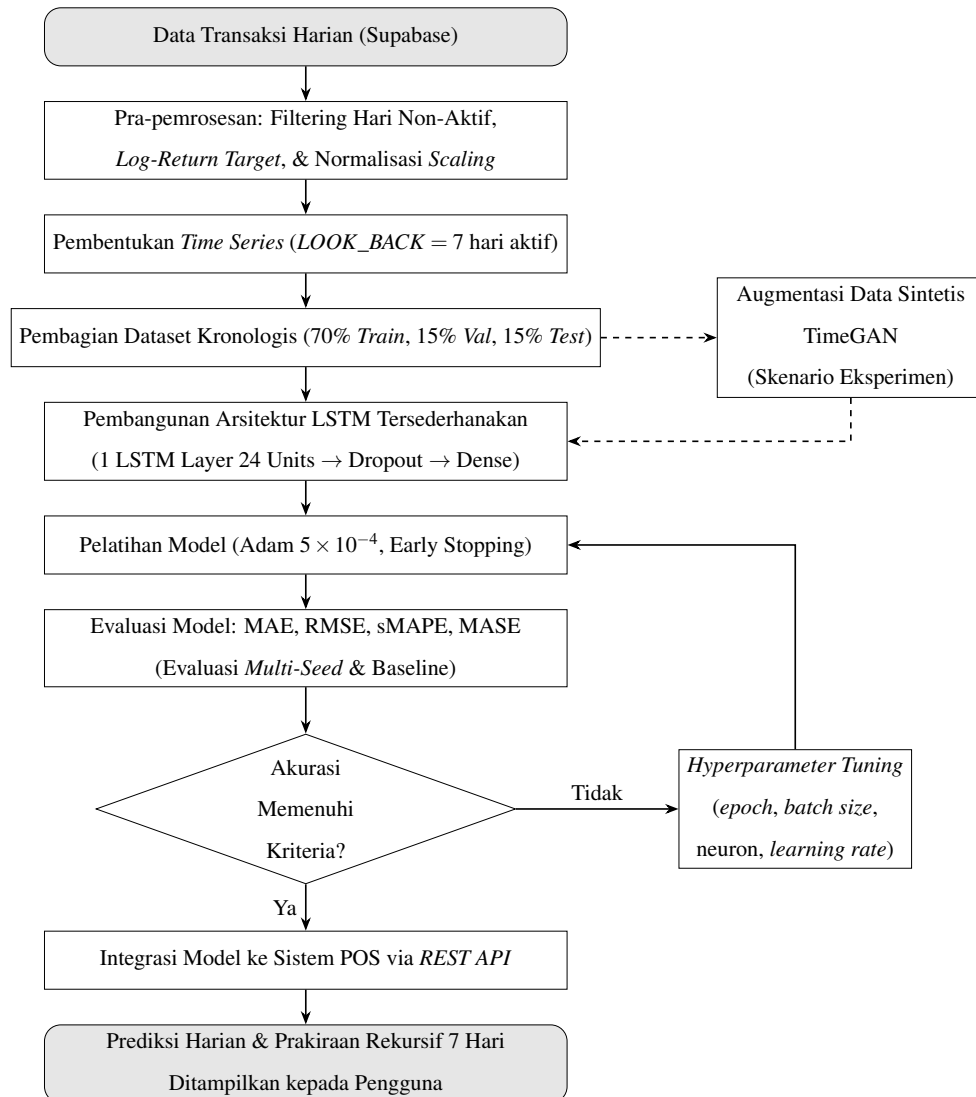
Entitas *stores* menjadi entitas induk yang berelasi dengan seluruh entitas lainnya. Entitas *store_members* mengelola hak akses berbasis peran (*Role-Based Access Control/RBAC*) untuk Owner dan Kasir. Katalog produk dikelola melalui entitas *products* dan *categories*, sedangkan data transaksi disimpan pada entitas *transactions* dan *transaction_items*. Riwayat perubahan stok dicatat pada entitas *stock_history*, sementara entitas *notifications* mendukung sistem notifikasi. Adapun hasil analisis fitur *AI Smart Analytics* disimpan sebagai riwayat *snapshot* pada entitas *smart_analytics_snapshots*. Daftar seluruh tabel basis data yang digunakan dirangkum pada Tabel 3.10.

Tabel 3.10 Daftar Tabel Basis Data POS

No.	Nama Tabel	Deskripsi
1	<i>stores</i>	Menyimpan data identitas dan konfigurasi outlet toko <i>merchant</i> .
2	<i>store_members</i>	Mengelola hak akses dan peran staf (Owner/Kasir) pada setiap toko.
3	<i>categories</i>	Menyimpan kategori pengelompokan produk atau menu dagangan.
4	<i>products</i>	Menyimpan katalog produk beserta harga, stok, dan data SKU/ <i>barcode</i> .
5	<i>transactions</i>	Menyimpan data nota transaksi penjualan kasir sebagai tabel induk.
6	<i>transaction_items</i>	Menyimpan rincian produk yang dibeli dalam setiap transaksi.
7	<i>stock_history</i>	Mencatat <i>log</i> audit perubahan stok masuk dan keluar produk.
8	<i>notifications</i>	Menyimpan pesan notifikasi sistem dan <i>broadcast</i> antar staf toko.
9	<i>smart_analytics_snapshots</i>	Menyimpan riwayat <i>snapshot</i> hasil analitik prediksi LSTM setiap kali fitur <i>Smart Analytics</i> dijalankan.

3.2.8 Rancangan Algoritma *Long Short-Term Memory* (LSTM)

Desain metode LSTM memberikan gambaran mengenai alur kerja algoritma yang digunakan dalam proses prediksi penjualan harian pada sistem POS. Alur kerja algoritma diilustrasikan pada Gambar 3.10.



Gambar 3.10 Alur Kerja Algoritma *Long Short-Term Memory* (LSTM)

Gambar 3.10 menggambarkan alur kerja model LSTM dari *input* data transaksi harian hingga proses evaluasi model. Adapun penjelasan rinci mengenai tahapan algoritma LSTM yang dirancang dalam penelitian ini adalah sebagai berikut:

1. **Pra-pemrosesan Data dan Transformasi Target:** Data transaksi harian ditarik dari database cloud Supabase. Langkah-langkah pra-pemrosesan yang dilakukan meliputi:

- *Filtering hari non-aktif (Zero Days):* Menghapus hari Sabtu, Minggu, serta bulan Januari dan Juli (libur semester kampus) untuk menghindari distorsi gap yang panjang pada deret waktu. Model dilatih murni menggunakan indeks hari kerja aktif (*business days*).
- *Transformasi Logaritma:* Data nominal pendapatan Y_t ditransformasikan ke skala logaritma menggunakan $Y'_t = \log(Y_t + 1)$ untuk menstabilkan variansi.
- *Transformasi Target (Log-Return):* Model tidak memprediksi pendapatan logaritmik secara langsung, melainkan memprediksi perubahan pendapatan relatif (*log-return*) r_t yang dirumuskan sebagai:

$$r_t = Y'_t - Y'_{t-1} = \log(Y_t + 1) - \log(Y_{t-1} + 1) \quad 3.1$$

Pendekatan *hybrid residual* ini bertujuan agar model mewarisi sifat autokorelasi jangka pendek yang dominan pada data penjualan.

- *Pengodean Siklik:* Fitur waktu (*day of week, week of year, month*) diubah menjadi nilai sinus dan kosinus untuk merepresentasikan sifat periodik kalender.
- *Normalisasi:* Fitur-fitur masukan dinormalisasi menggunakan *Min-Max Scaling* ke rentang $[0, 1]$, di mana proses pemasangan (*fitting*) parameter *scaler* hanya dilakukan pada data latih (*training set*) untuk mencegah kebocoran informasi (*data leakage*).

2. **Pembentukan Sequence (Sliding Window):** Dataset deret waktu diubah menjadi bentuk urutan berpasangan (*sequence pairs*) menggunakan metode *sliding window*. Ukuran *window* masukan (*look-back*) ditetapkan sebesar 7 hari aktif terakhir ($LOOK_BACK = 7$) untuk memprediksi target 1 hari aktif berikutnya ($HORIZON = 1$). Dengan demikian, model dilatih dengan

format peramalan satu langkah ke depan (*one-step-ahead prediction*).

3. **Pembagian Dataset Kronologis:** Dataset dibagi secara kronologis (*time-based split*) untuk menjaga integritas temporal deret waktu, dengan proporsi pembagian sebagai berikut:

- *Data Latih (Training Set)*: Sebesar 70% dari total data aktif (periode Agustus 2024 s.d. Agustus 2025).
- *Data Validasi (Validation Set)*: Sebesar 15% (periode September s.d. Desember 2025) digunakan untuk pemantauan proses latih dan *early stopping*.
- *Data Uji (Testing Set)*: Sebesar 15% (periode Januari s.d. Juni 2026) digunakan untuk pengujian akhir model.

4. **Pembangunan Arsitektur LSTM Tersederhanakan:** Untuk menghindari masalah *overfitting* dan *model collapse* pada dataset yang relatif kecil, arsitektur LSTM dirancang secara minimalis (*shallow network*):

- *Input Layer*: Menerima matriks berukuran 7×8 (panjang *sequence* 7 dan total 8 fitur masukan).
- *LSTM Layer*: Terdiri atas 1 layer LSTM dengan 24 *hidden units*.
- *Dropout Layer*: Dengan laju 0,1 untuk mencegah ketergantungan berlebih antar-node.
- *Dense Layer*: Terdiri atas 16 neuron dengan fungsi aktivasi *Rectified Linear Unit* (ReLU).
- *Output Layer*: Terdiri atas 1 neuron (*Dense* dengan aktivasi linear) untuk memprediksi nilai *log-return* $\hat{r}_t$.

Model ini memiliki total parameter terlatih sebanyak 3.585 parameter, yang sebanding dengan ukuran data latih.

5. **Pelatihan Model:** Model dilatih menggunakan algoritma optimasi Adam dengan *learning rate* awal sebesar 5×10^{-4} dan fungsi kerugian (*loss function*) *Mean Squared Error* (MSE). Untuk mengoptimalkan konvergensi, diterapkan dua mekanisme *callback*:

- *Early Stopping*: Menghentikan pelatihan jika loss validasi tidak

membaik selama 20 *epoch* berturut-turut untuk mencegah *overfitting*, serta mengembalikan bobot terbaik (*restore best weights*).

- *Reduce Learning Rate on Plateau*: Mengurangi *learning rate* sebesar faktor 0,5 jika loss validasi mendatar selama 8 *epoch*.

Ukuran *batch* pelatihan diatur sebesar 16 dengan jumlah maksimum pelatihan 200 *epoch*.

6. **Evaluasi Komparatif dan Multi-Seed:** Untuk membuktikan keandalan model LSTM, kinerjanya dibandingkan dengan tiga model pembanding (*baseline*):

- *Naive (Kemarin)*: Prediksi hari ini dianggap sama dengan nilai penjualan riil hari aktif sebelumnya.
- *Mean-7*: Prediksi berupa rata-rata bergerak dari 7 hari aktif terakhir.
- *SARIMA*: Model statistik klasik Seasonal Autoregressive Integrated Moving Average untuk mendeteksi pola autokorelasi deret waktu.
- *Seasonal-Naive*: Prediksi berupa rata-rata beberapa kemunculan terakhir dari hari yang sama dalam minggu (Senin dengan Senin, Selasa dengan Selasa, dan seterusnya), untuk menangkap pola musiman mingguan.

Mengingat inisialisasi bobot awal jaringan saraf bersifat acak dan dapat memengaruhi stabilitas pada data kecil, evaluasi dilakukan menggunakan metode ***Multi-Seed Evaluation***. Model dilatih sebanyak 10 kali menggunakan *seed* acak yang berbeda. Hasil evaluasi dilaporkan dalam bentuk rata-rata $\pm$ standar deviasi dari metrik MAE, RMSE, sMAPE, dan MASE guna memperoleh estimasi performa yang kokoh dan dapat dipertanggungjawabkan.

7. **Eksperimen Augmentasi Data Sintetis (TimeGAN):** Sebagai upaya mitigasi terhadap keterbatasan jumlah data latih, dirancang skenario eksperimen augmentasi data sintetis menggunakan **TimeGAN** (*Time-series Generative Adversarial Networks*), yaitu arsitektur GAN khusus deret waktu yang diusulkan oleh Yoon et al. (2019). TimeGAN diimplementasikan

menggunakan *framework* PyTorch dan terdiri atas lima jaringan saraf (*embedder*, *recovery*, *generator*, *supervisor*, dan *discriminator*) yang dilatih dalam tiga fase: (1) pelatihan *autoencoder* (*embedder*–*recovery*), (2) pelatihan *supervised* pada ruang laten, dan (3) pelatihan gabungan (*joint training*) secara *adversarial*. Model dilatih pada jendela deret waktu sepanjang 24 hari aktif dengan tiga fitur numerik (omzet, jumlah transaksi, dan kuantitas terjual), kemudian menghasilkan 200 sekuens data harian sintetis. Data sintetis tersebut digabungkan dengan data riil untuk melatih ulang model LSTM (skenario *augmented*), lalu performanya dibandingkan terhadap model LSTM tanpa augmentasi dan *baseline Seasonal-Naive* guna mengukur seberapa besar augmentasi data mampu memperbaiki akurasi peramalan pada data terbatas.

8. **Rekonstruksi dan Prakiraan Rekursif Multi-Langkah:** Output model berupa *log-return* prediksi ($\hat{r}_t$) direkonstruksi kembali ke skala nominal Rupiah ($\hat{Y}_t$) menggunakan nilai penjualan hari sebelumnya (Y_{t-1}):

$$\hat{Y}_t = \exp(\log(Y_{t-1} + 1) + \hat{r}_t) - 1 \quad 3.2$$

Untuk menyajikan proyeksi 7 hari aktif ke depan pada aplikasi POS, diterapkan metode ***Recursive Forecasting*** (memasukkan hasil prediksi langkah sebelumnya sebagai input untuk langkah berikutnya). Untuk mencegah masalah akumulasi bias (*error propagation*) yang dapat menyebabkan nilai prediksi meluruh atau meledak, diimplementasikan tiga teknik pengaman:

- *Clipping log-return*: Membatasi nilai prediksi *log-return* pada rentang persentil 10–90 dari data pelatihan untuk mencegah nilai ekstrem.
- *Damping factor*: Meredam prediksi perubahan *log-return* pada langkah ke- h menuju nilai 0 dengan faktor peredam ϕ^h (di mana $\phi = 0.7$). Hal ini membuat ramalan jangka panjang stabil dan kembali ke nilai rata-rata historis (sifat *mean-reverting*).

- *Safety rail*: Membatasi hasil akhir nominal pendapatan berada dalam rentang pendapatan terkecil dan terbesar yang pernah tercatat secara historis pada hari aktif.

3.2.9 Perancangan Wireframe

Perancangan *wireframe* dalam penelitian ini mencakup rancangan antarmuka pengguna untuk semua layar utama yang menjadi bagian dari alur operasional aplikasi POS. *Wireframe* dirancang dengan mempertimbangkan kebutuhan pengguna, aksesibilitas, dan alur kerja yang efisien. Terdapat 13 rancangan layar yang melayani kebutuhan berbeda sesuai dengan peran pengguna, sebagaimana diuraikan berikut ini.

1. Halaman Registrasi Toko Baru

Halaman pendaftaran akun dan toko baru yang digunakan saat inisialisasi awal sistem oleh Owner, mencakup pengisian data identitas pengguna dan informasi outlet toko.

 ID ▾

Buat Akun

Lengkapi data di bawah ini untuk mendaftar.

Nama Lengkap

 Masukkan nama lengkap

Email

 Masukkan email

Password

 Masukkan password 

Konfirmasi Password

 Ulangi password 

Daftar

ATAU

 Lanjutkan dengan Google

Sudah punya akun? **Masuk**

Gambar 3.11 *Wireframe* Halaman Registrasi Toko Baru

2. Halaman *Login*

Halaman masuk akun untuk Owner maupun Kasir dengan autentikasi berbasis email dan kata sandi melalui Supabase Auth.

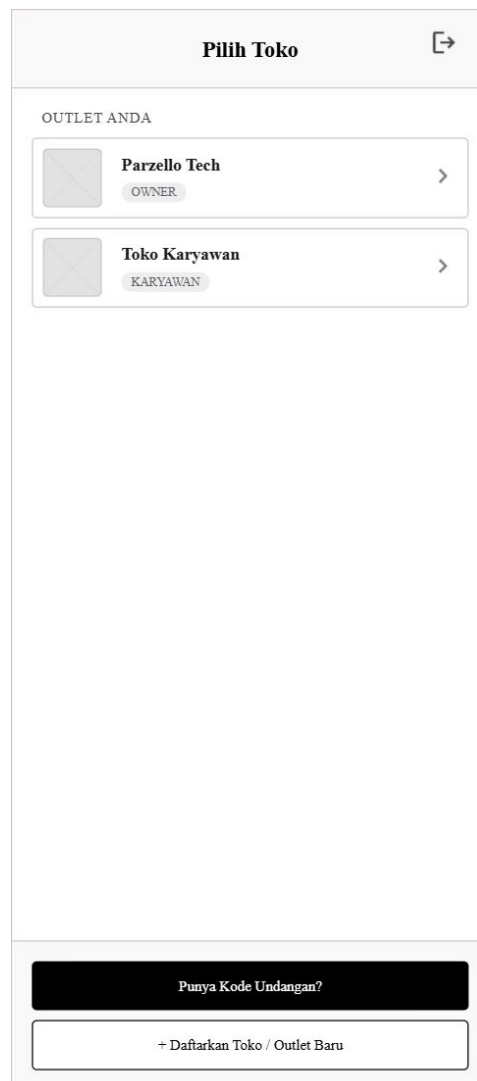
The wireframe shows a login interface with the following elements:

- Title:** Selamat Datang Kembali
- Subtitle:** Masukkan email dan password untuk masuk ke dashboard POS.
- Email Field:** A text input field with the placeholder text "Masukkan email Anda".
- Password Field:** A text input field with the placeholder text "Masukkan kata sandi Anda" and a toggle icon (an eye with a slash) on the right.
- Remember Me:** A checkbox labeled "Ingat saya".
- Forgot Password:** A link labeled "Lupa password?".
- Login Button:** A solid black button with the text "Masuk".
- Separator:** A horizontal line with the word "ATAU" in the center.
- Google Login:** A button with a Google logo icon and the text "Lanjutkan dengan Google".

Gambar 3.12 Wireframe Halaman *Login*

3. Halaman Pemilihan Toko

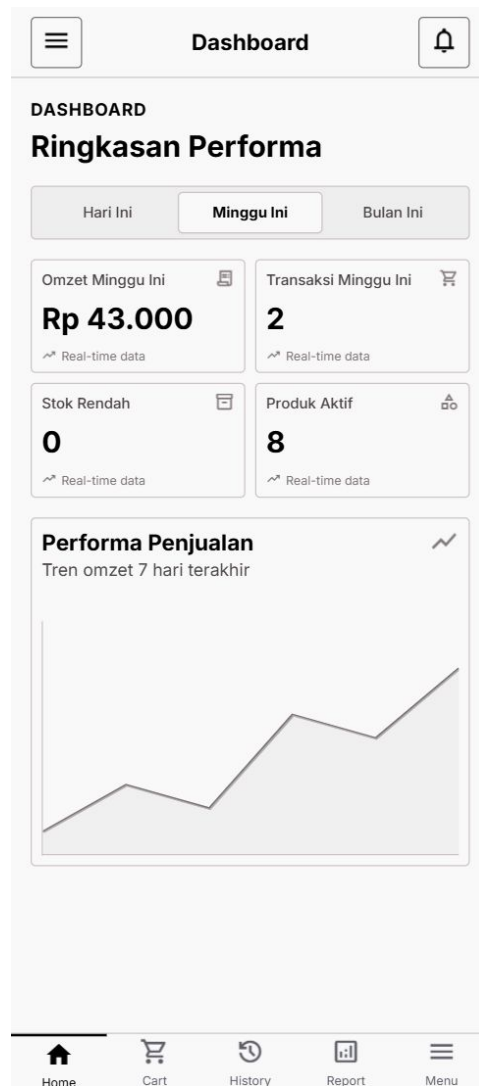
Memungkinkan pengguna yang telah *login* untuk memilih toko aktif yang akan dioperasikan, terutama bagi pengguna yang terdaftar di lebih dari satu outlet.



Gambar 3.13 Wireframe Halaman Pemilihan Toko

4. Halaman Dasbor Owner

Ringkasan performa toko yang menampilkan statistik penjualan harian, grafik tren omzet, pintasan fitur penting, dan akses cepat ke modul utama khusus Owner.



Gambar 3.14 Wireframe Halaman Dasbor Owner

5. Halaman POS (Pembayaran)

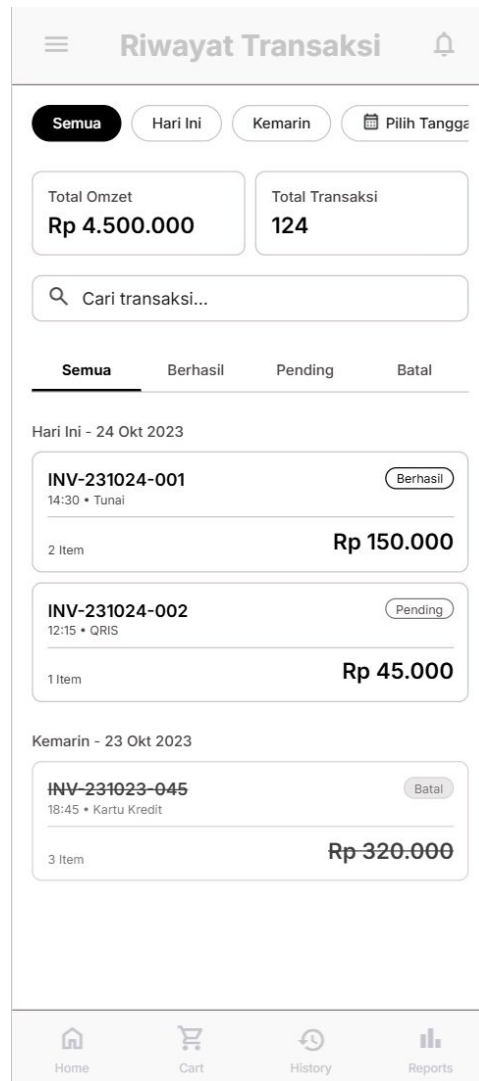
Layar utama kasir untuk mencari produk, menyusun keranjang pesanan, menerapkan *voucher* diskon, memilih metode pembayaran, dan memproses transaksi.

The wireframe shows a mobile application interface for a Point of Sale (POS) system. At the top, there is a back arrow and the title 'Pembayaran'. Below this, a grey box displays 'TOTAL TAGIHAN' and 'Rp 25.000'. The 'Metode Pembayaran' section has two buttons: 'Tunai' (Cash) with a cash icon and 'QRIS' with a QR code icon. The 'Uang Tunai Diterima' section shows 'Rp 25000' in a text box, followed by three buttons for denominations: 'Rp 25', 'Rp 50', and 'Rp 100', each with a 'rb' (ribuan) suffix. At the bottom of this section, a grey box shows 'KEMBALIAN' and 'Rp 0' with a checkmark icon. A large black button at the very bottom is labeled 'Konfirmasi & Simpan Transaksi'.

Gambar 3.15 Wireframe Halaman POS (Pembayaran)

6. Halaman Riwayat Transaksi

Menampilkan daftar transaksi yang telah dilakukan lengkap dengan fitur pencarian dan *filter* berdasarkan tanggal, berguna untuk keperluan verifikasi dan audit oleh semua staf toko.



Gambar 3.16 Wireframe Halaman Riwayat Transaksi

7. Halaman Kelola Produk

Memungkinkan staf toko untuk menambah, menyunting, dan menghapus produk yang dijual, lengkap dengan harga, stok, SKU/*barcode*, gambar produk, dan kategori.

←

Tambah Produk

Tambah Foto

Maksimal ukuran foto 5MB. Format JPG, PNG.

Nama Produk

Masukkan nama produk

Kategori

Pilih kategori

SKU / Barcode

Contoh: PRD-001

Harga Modal

Rp 0

Harga Jual

Rp 0

Stok Awal

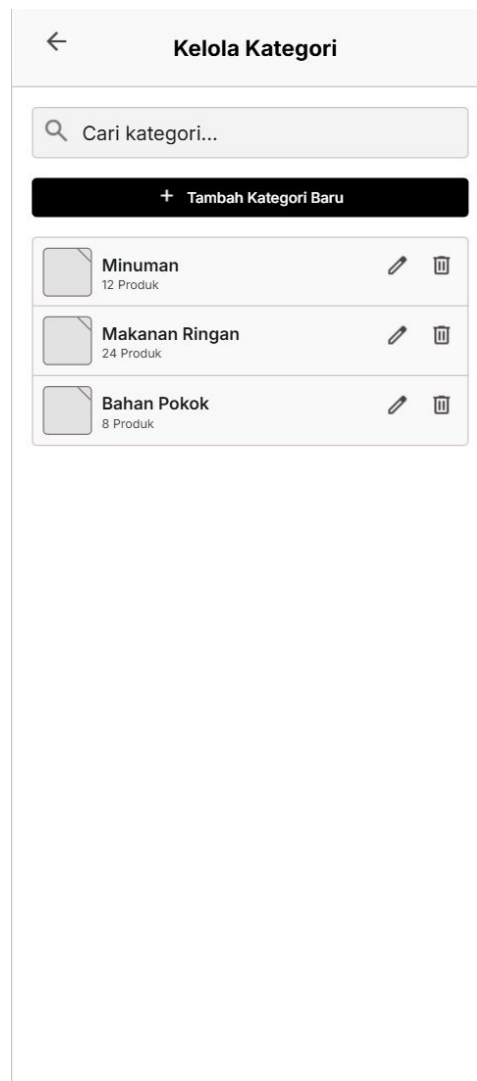
0

Simpan Produk

Gambar 3.17 *Wireframe* Halaman Kelola Produk

8. Halaman Kelola Kategori

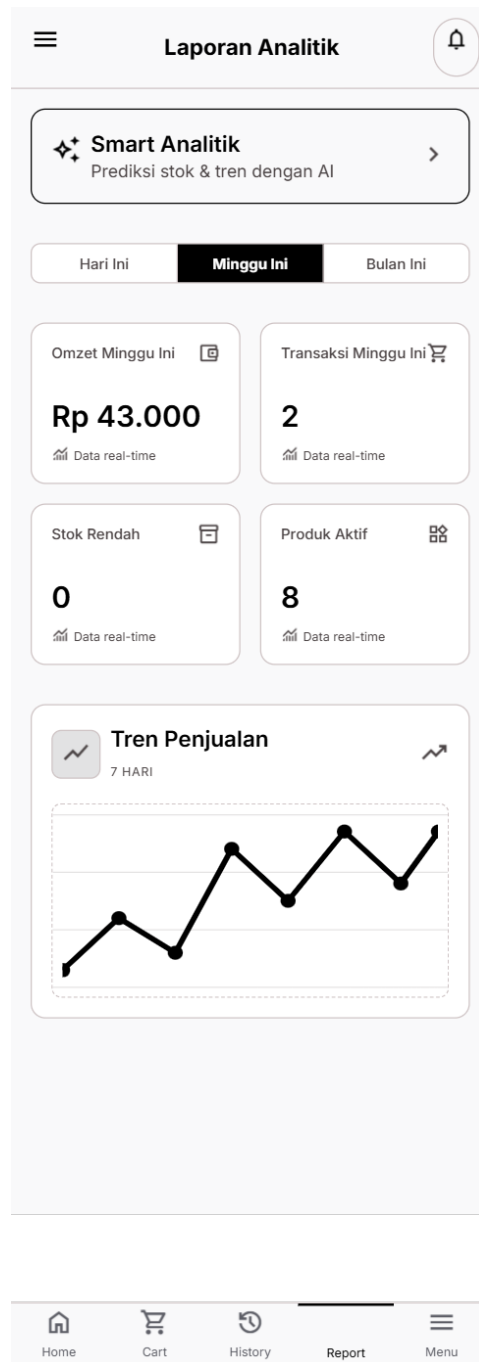
Memungkinkan staf toko untuk membuat, mengedit, dan menghapus kategori produk guna mengorganisasi katalog dagangan toko dengan lebih terstruktur.



Gambar 3.18 *Wireframe* Halaman Kelola Kategori

9. Halaman Laporan

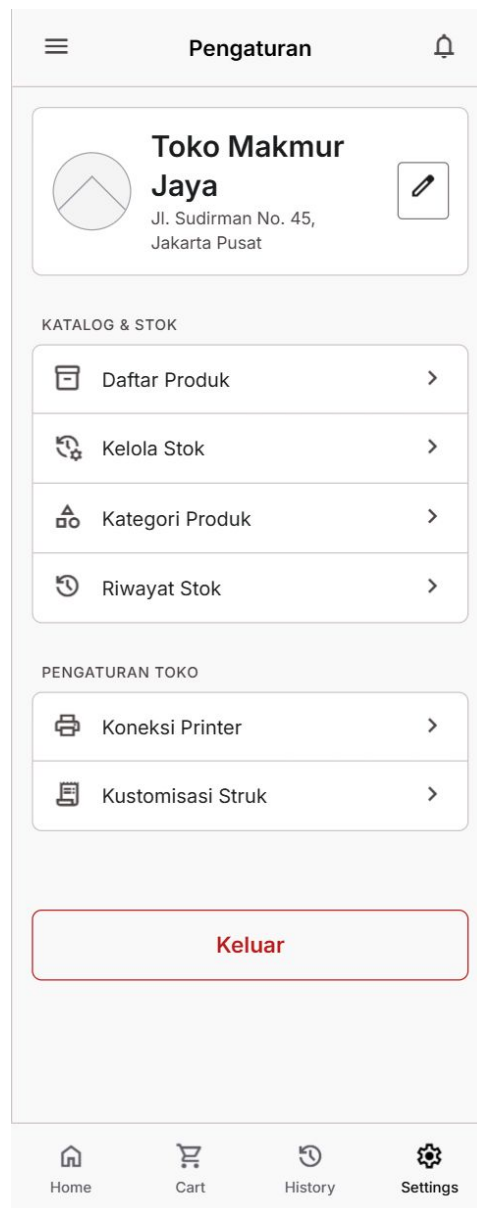
Layar yang dapat diakses seluruh staf toko untuk menampilkan laporan penjualan, tren bisnis, proyeksi penjualan harian berbasis model LSTM, dan ringkasan performa operasional toko.



Gambar 3.19 Wireframe Halaman Laporan

10. Halaman Pengaturan

Pusat konfigurasi aplikasi dan toko yang dapat diakses oleh semua staf, dengan sejumlah menu yang hanya tersedia bagi Owner seperti pengaturan profil toko dan integrasi pembayaran.



Gambar 3.20 Wireframe Halaman Pengaturan

11. Halaman Informasi Toko

Memungkinkan Owner untuk mengubah nama toko, alamat, logo outlet, dan informasi operasional lainnya yang ditampilkan kepada pelanggan.

Gambar 3.21 Wireframe Halaman Informasi Toko

12. Halaman Struk Digital

Menampilkan pratinjau struk transaksi dalam format digital yang dapat dibagikan kepada pelanggan, mencakup rincian produk, total pembayaran, dan informasi toko.

<
Struk Digital

Parzello Tech
Telp: 085161787501
Terima Kasih Telah Berbelanja

No. Transaksi	#2E17EDFD
Tanggal	10 Jun 2026, 03:00
Metode	Tunai
Burger Daging + Telur 1 x Rp 15.000	Rp 15.000
Burger Sosis + Telur 1 x Rp 10.000	Rp 10.000
Total Belanja	Rp 25.000
Bayar	Rp 25.000
KEMBALIAN	Rp 0

--- SELESAI ---
Powered by Parzello POS

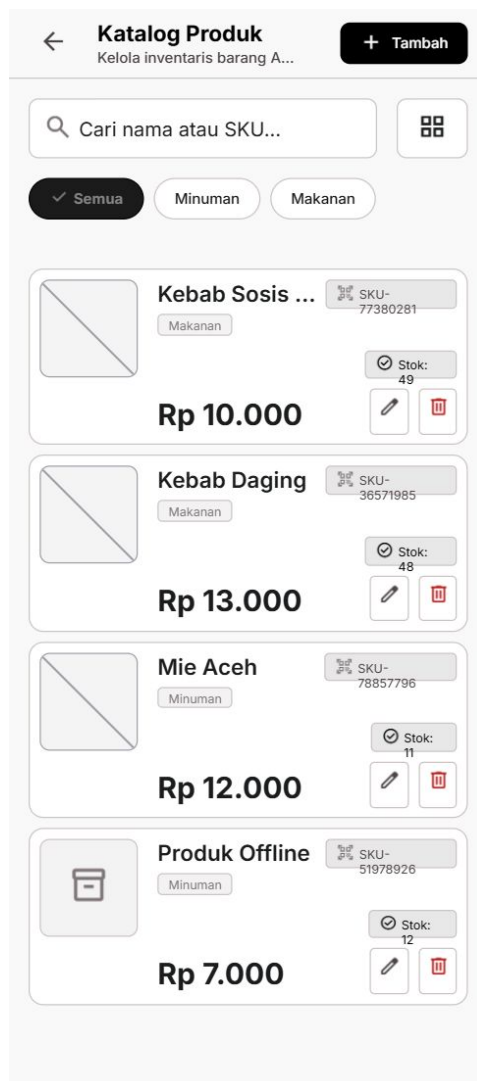
Bagikan

Set Printer

Gambar 3.22 Wireframe Halaman Struk Digital

13. Halaman Katalog Produk

Beranda mode pelanggan yang menampilkan katalog produk toko beserta kategori menu, informasi harga, dan fitur pencarian produk secara mandiri.



Gambar 3.23 Wireframe Halaman Katalog Produk

3.2.10 Teknik Pengujian

Pengujian sistem pada penelitian ini mencakup dua pendekatan utama untuk memastikan kualitas fungsional aplikasi dan keandalan model prediksi yang diintegrasikan. Pendekatan pengujian tersebut dibagi menjadi pengujian evaluasi model prediksi penjualan dan pengujian fungsionalitas sistem secara keseluruhan menggunakan metode *black box testing*.

3.2.10.1 Rancangan Pengujian Evaluasi Model Prediksi

Pengujian model prediksi dilakukan untuk mengukur kinerja dan tingkat akurasi dari algoritma *Long Short-Term Memory* (LSTM) yang dikembangkan. Pengujian ini menggunakan data pengujian (*testing data*) yang diambil dari data histori transaksi penjualan harian mitra UMKM. Evaluasi akurasi model diukur berdasarkan tingkat kesalahan (*error*) prediksi menggunakan empat metrik utama, yaitu MAE, RMSE, sMAPE, dan MASE.

1. **Mean Absolute Error (MAE):** MAE digunakan untuk mengukur rata-rata selisih absolut antara nilai aktual penjualan dengan nilai hasil prediksi. Semakin kecil nilai MAE, maka model prediksi semakin akurat. Formula MAE didefinisikan sebagai berikut:

$$\text{MAE} = \frac{1}{n} \sum_{t=1}^n |Y_t - \hat{Y}_t| \quad 3.3$$

di mana Y_t adalah nilai aktual pada hari ke- t , $\hat{Y}_t$ adalah nilai prediksi pada hari ke- t , dan n adalah jumlah data pengujian.

2. **Root Mean Square Error (RMSE):** RMSE memberikan bobot lebih besar pada kesalahan prediksi yang bernilai besar karena selisihnya dikuadratkan terlebih dahulu sebelum dirata-ratakan dan ditarik akarnya. Formula RMSE didefinisikan sebagai berikut:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{t=1}^n (Y_t - \hat{Y}_t)^2} \quad 3.4$$

3. **Symmetric Mean Absolute Percentage Error (sMAPE):** sMAPE digunakan untuk mengukur persentase kesalahan absolut rata-rata dengan membagi selisih absolut terhadap rata-rata dari nilai aktual dan nilai prediksi. sMAPE memberikan batas kesalahan maksimum sebesar 200% dan bersifat lebih stabil dibandingkan MAPE konvensional ketika dihadapkan pada nilai aktual

yang mendekati nol. Formula sMAPE didefinisikan sebagai berikut:

$$\text{sMAPE} = \frac{1}{n} \sum_{t=1}^n \frac{|Y_t - \hat{Y}_t|}{(|Y_t| + |\hat{Y}_t|)/2} \times 100\% \quad 3.5$$

4. **Mean Absolute Scaled Error (MASE):** MASE mengukur kesalahan absolut model prediksi relatif terhadap kesalahan absolut rata-rata dari model peramalan naif sederhana (*one-step naive forecast*) secara *in-sample* pada data latih. Nilai MASE kurang dari 1 menunjukkan bahwa model memiliki kinerja peramalan yang lebih baik dibandingkan metode naif. Formula MASE didefinisikan sebagai berikut:

$$\text{MASE} = \frac{\frac{1}{n} \sum_{t=1}^n |Y_t - \hat{Y}_t|}{\frac{1}{N-1} \sum_{i=2}^N |Y_i^{\text{train}} - Y_{i-1}^{\text{train}}|} \quad 3.6$$

di mana Y_i^{train} adalah nilai aktual pendapatan pada data latih, dan N adalah jumlah data latih aktif.

Pengujian evaluasi ini dilakukan dengan mengukur kinerja model menggunakan pendekatan *multi-seed evaluation* (10 *seeds* acak) untuk memastikan stabilitas performa model LSTM terhadap variasi inisialisasi bobot awal jaringan saraf. Hasil pengujian akhir tersebut selanjutnya dibandingkan dengan model pembandingan (*baselines*) untuk menentukan kelayakan model LSTM. Selain itu, diuji pula skenario perbandingan performa model LSTM yang dilatih dengan dan tanpa tambahan data sintetis hasil augmentasi TimeGAN terhadap *baseline Seasonal-Naive*, guna mengukur efektivitas augmentasi data dalam memitigasi keterbatasan ukuran dataset.

3.2.10.2 Rancangan Analisis Pengujian *Black Box*

Pengujian fungsional sistem pada penelitian ini menggunakan metode *black box testing*, yaitu metode pengujian perangkat lunak yang berfokus pada validasi fungsional sistem dengan mengamati kesesuaian antara masukan (*input*) dan keluaran (*output*) yang dihasilkan, tanpa memperhatikan struktur internal kode

program. Pengujian diterapkan pada fitur-fitur utama sistem untuk memastikan aplikasi berjalan sesuai dengan kebutuhan operasional pengguna.

Rancangan skenario pengujian *black box* untuk fitur-fitur utama sistem POS ini dijabarkan pada Tabel 3.11 berikut:

Tabel 3.11 Skenario Pengujian *Black Box*

No.	Fitur / Fungsi	Skenario Pengujian	Hasil yang Diharapkan
1	Registrasi Toko Baru	Mengisi <i>form</i> registrasi dengan email baru, kata sandi, nama toko, alamat, dan nomor HP.	Akun Owner berhasil dibuat, data toko tersimpan di <i>database</i> Supabase <i>cloud</i> , dan beralih ke halaman utama.
2	Login Akun	Memasukkan email dan kata sandi yang <i>valid/invalid</i> pada layar <i>login</i> .	Jika <i>valid</i> , pengguna masuk ke dasbor. Jika <i>invalid</i> , sistem menampilkan pesan <i>error</i> autentikasi.
3	Kelola Produk	Menambah produk baru dengan nama, harga, stok, foto, dan kategori serta mengubah/menghapus produk.	Data produk baru tersimpan di <i>database</i> lokal Isar DB, terdaftar pada <i>list</i> produk, dan dapat diubah/dihapus.
4	Kelola Kategori	Menambahkan kategori baru, menyunting nama kategori, atau menghapus kategori produk.	Kategori berhasil dibuat, muncul sebagai opsi <i>filter</i> menu, dan diperbarui secara <i>real-time</i> .
5	Checkout Transaksi	Memasukkan produk ke keranjang, menerapkan <i>voucher</i> diskon, memilih metode bayar, dan mengonfirmasi nominal bayar.	Transaksi berhasil disimpan di <i>database</i> lokal Isar DB, jumlah stok produk berkurang otomatis, dan dialihkan ke struk sukses.
6	Mencetak Struk	Menghubungkan printer Bluetooth <i>thermal</i> dan menekan tombol cetak struk dari riwayat transaksi.	Printer <i>thermal</i> mencetak struk fisik transaksi secara lengkap (nama produk, total, tanggal, metode bayar).

7	Sinkronisasi <i>Offline</i>	Mematikan koneksi internet (luring), membuat transaksi lokal, lalu mengaktifkan internet kembali (daring).	Transaksi disimpan secara lokal di Isar DB (<i>offline-first</i>), lalu terunggah otomatis ke Supabase <i>cloud</i> saat internet aktif kembali.
8	Dasbor Laporan	Membuka dasbor laporan penjualan harian, mingguan, dan grafik visualisasi omzet.	Menampilkan statistik ringkasan omzet, total transaksi harian, dan grafik visualisasi garis yang akurat.

3.2.10.3 Rancangan Analisis Pengujian *White Box*

Selain pengujian fungsional *black box*, penelitian ini turut menerapkan pengujian *white box* secara terbatas (*lightweight*) yang berfokus pada verifikasi kebenaran logika internal dan jalur eksekusi kode (*code path*) pada fungsi-fungsi kritis di layanan API prediksi, tanpa memandang antarmuka luarnya. Pengujian ini dilakukan dengan menelusuri (*trace*) jalur logika kode secara manual menggunakan data masukan yang telah diketahui nilai keluarannya, kemudian membandingkan hasil eksekusi aktual terhadap hasil kalkulasi manual sebagai acuan kebenaran (*ground truth*). Tiga fungsi inti yang menjadi target pengujian *white box* adalah:

1. Fungsi perhitungan *Seasonal-Naive* (*by_dow*), untuk memverifikasi jalur percabangan logika ketika jumlah data historis per hari kerja mencukupi ($\geq k$), tidak mencukupi ($< k$), maupun tidak tersedia sama sekali (*fallback*).
2. Fungsi metrik evaluasi sMAPE dan MASE, untuk memverifikasi jalur penanganan kasus pembagian dengan nilai penyebut nol atau mendekati nol agar tidak menghasilkan galat pembagian (*division by zero*).
3. Fungsi peramalan rekursif (*forecast_hybrid_stable*), untuk memverifikasi jalur eksekusi tiga mekanisme pengaman secara berurutan, yaitu *clipping log-return*, *damping factor*, dan *safety rail* nominal omzet.

3.2.11 Hasil yang Diharapkan

Hasil yang diharapkan dari penelitian ini meliputi beberapa aspek sebagai berikut:

1. Aplikasi *Point of Sale* (POS) *mobile* yang mampu membantu pelaku UMKM dalam mengelola transaksi penjualan, inventaris produk, dan laporan keuangan secara efisien dengan kemampuan operasional *offline-first*.
2. Implementasi metode LSTM pada sistem yang berjalan dengan baik dan menghasilkan prediksi penjualan harian yang akurat, sehingga dapat digunakan sebagai dasar pengambilan keputusan usaha.
3. Sistem yang mampu menyajikan informasi prediksi dan laporan penjualan dalam bentuk visualisasi grafik yang mudah dipahami oleh pengguna, khususnya pelaku UMKM.
4. Hasil penelitian ini diharapkan dapat dipublikasikan dalam jurnal ilmiah sebagai kontribusi dalam pengembangan sistem informasi dan penerapan *machine learning* pada UMKM.
5. Penelitian ini menghasilkan laporan tugas akhir sebagai salah satu syarat penyelesaian studi pada Program Studi Teknik Informatika Politeknik Negeri Lhokseumawe.

BAB IV

HASIL DAN PEMBAHASAN

Bab ini menyajikan hasil rancang bangun dan pembahasan sistem Point of Sale (POS) dengan fitur prediksi penjualan harian menggunakan metode Long Short-Term Memory (LSTM). Pembahasan dibagi menjadi tujuh bagian utama, yaitu hasil implementasi sistem, *preprocessing* data, implementasi dan pelatihan model peramalan, evaluasi model, hasil pengujian sistem, pembahasan hasil penelitian, serta keterbatasan sistem dan penelitian.

4.1 Hasil Implementasi Sistem

Bagian ini memaparkan realisasi fisik dari rancangan sistem yang telah disusun pada Bab III, yang mencakup implementasi antarmuka aplikasi mobile (kasir dan owner), implementasi backend dan basis data cloud (Supabase), serta implementasi layanan prediksi (API) di sisi server.

4.1.1 Implementasi Antarmuka (UI)

Aplikasi mobile dikembangkan menggunakan framework Flutter dengan bahasa pemrograman Dart. Komponen antarmuka pengguna disesuaikan dengan rancangan wireframe untuk menjamin kenyamanan operasional kasir. Berikut adalah penjelasan mengenai implementasi antarmuka layar utama aplikasi:

1. **Halaman Registrasi Toko Baru:** Halaman ini merupakan langkah awal inisialisasi sistem yang digunakan oleh Owner untuk mendaftarkan akun baru dan meregistrasikan profil outlet toko. Form registrasi memuat kolom input nama lengkap, email, kata sandi, nama toko, alamat outlet, dan nomor telepon.
2. **Halaman Login:** Menyediakan fasilitas autentikasi pengguna menggunakan email dan kata sandi. Keamanan akses diatur melalui layanan autentikasi Supabase. Sistem secara otomatis mengidentifikasi peran akun (Owner atau

Kasir) setelah login berhasil untuk menentukan batasan hak akses menu.

3. **Halaman Pemilihan Toko:** Layar ini ditujukan bagi pengguna yang terdaftar di lebih dari satu outlet toko merchant. Pengguna dapat memilih outlet aktif yang ingin dioperasikan sebelum diarahkan ke halaman dasbor utama.
4. **Halaman Dasbor Owner:** Menampilkan visualisasi ringkasan kinerja penjualan toko saat ini, seperti grafik tren omzet penjualan harian, jumlah total transaksi, total nominal omzet harian, dan ringkasan stok produk yang menipis sebagai peringatan restock cepat.
5. **Halaman POS (Pembayaran / Checkout):** Merupakan layar operasional utama kasir untuk memproses transaksi. Kasir dapat memilih produk, memasukkan jumlah barang ke keranjang belanja, menerapkan diskon atau voucher, memilih metode pembayaran (Tunai, QRIS, atau Transfer), serta menyelesaikan transaksi pembayaran.
6. **Halaman Riwayat Transaksi:** Menampilkan daftar nota transaksi penjualan yang telah diselesaikan. Halaman ini memuat informasi detail nomor invoice, tanggal transaksi, nama kasir, total nominal pembayaran, dan status sinkronisasi cloud.
7. **Halaman Kelola Produk dan Kategori:** Menyediakan antarmuka bagi staf toko untuk mengelola inventaris produk (menambah, mengubah, dan menghapus produk) beserta pengelompokan berdasarkan kategori produk. Form produk memuat nama produk, harga jual, stok fisik, barcode/SKU, dan gambar produk.
8. **Halaman Laporan (Analitik AI & Prediksi LSTM):** Menampilkan visualisasi grafik data penjualan aktual historis berdampingan dengan kurva proyeksi penjualan harian untuk 7 hari ke depan yang diperoleh dari layanan API prediksi penjualan.
9. **Halaman Pengaturan dan Profil Toko:** Memungkinkan staf toko untuk mengubah konfigurasi dasar aplikasi, dengan menu pengaturan profil outlet toko (logo, nama, alamat) dan pengelolaan akun karyawan/staf kasir yang khusus tersedia bagi Owner.

10. **Halaman Struk Digital dan Katalog Mandiri:** Halaman struk digital menampilkan pratinjau struk fisik transaksi yang dapat dibagikan langsung ke pelanggan via aplikasi pesan instan. Layar katalog mandiri menyediakan daftar menu produk yang dapat dipindai oleh pelanggan untuk melihat daftar harga barang secara langsung.

4.1.2 Implementasi Backend dan Basis Data Cloud

Infrastruktur backend cloud diimplementasikan menggunakan platform Supabase BaaS dengan database PostgreSQL. Struktur relasional database diwujudkan melalui sembilan tabel utama yang saling terhubung:

1. *stores*: Menyimpan data identitas outlet merchant (id, nama, alamat, nomor HP, URL logo, created_at).
2. *store_members*: Mengatur relasi hak akses pengguna dengan peran Owner atau Karyawan di masing-masing outlet.
3. *categories*: Mengelola daftar kategori pengelompokan produk.
4. *products*: Menyimpan katalog inventaris barang (id, nama, harga, stok, barcode, URL foto, is_synced, is_deleted).
5. *transactions*: Menyimpan data nota induk transaksi penjualan (id, total belanja, metode pembayaran, kasir_id, created_at).
6. *transaction_items*: Menyimpan rincian item produk yang terjual pada setiap transaksi penjualan (id, transaksi_id, produk_id, jumlah, harga).
7. *stock_history*: Mencatat log audit mutasi perubahan stok produk (stok masuk, keluar, penjualan, audit manual).
8. *notifications*: Mengelola pesan notifikasi sistem dan info siaran antar staf toko secara real-time.
9. *smart_analytics_snapshots*: Menyimpan riwayat *snapshot* hasil analitik prediksi LSTM setiap kali fitur *Smart Analytics* dijalankan, sehingga hasil analisis sebelumnya dapat ditinjau kembali tanpa memanggil ulang model.

Keamanan akses data antar-outlet dijamin melalui penerapan kebijakan Row Level Security (RLS) di database PostgreSQL. Kebijakan ini memastikan bahwa

pengguna (kasir/owner) hanya dapat membaca dan menulis data yang memiliki *store_id* yang sesuai dengan toko tempat mereka terdaftar.

4.1.3 Implementasi Layanan Prediksi (API)

Modul kecerdasan buatan untuk kalkulasi prediksi penjualan harian dikembangkan menggunakan Python dan terdiri atas dua bagian yang saling melengkapi. Bagian pertama adalah *pipeline* riset berbasis Jupyter Notebook yang melatih model peramalan LSTM (TensorFlow/Keras) pada tiga granularitas waktu (harian, mingguan, dan bulanan) beserta eksperimen augmentasi data sintetis TimeGAN (PyTorch). Bagian kedua adalah layanan API produksi berbasis *framework* Flask yang di-*deploy* pada platform Hugging Face dan diintegrasikan langsung dengan aplikasi POS, sehingga beban komputasi peramalan terpisah dari beban kerja transaksional kasir.

Layanan API produksi menyediakan tujuh *endpoint* REST dengan fungsi sebagai berikut:

1. **Endpoint Prediksi Penjualan** (/api/predict/daily, /api/predict/weekly, /api/predict/monthly): Meramalkan omzet ke depan dengan tanggal prediksi yang otomatis melewati hari tutup (akhir pekan serta libur semester Januari/Juli). Berdasarkan hasil evaluasi komparatif pada penelitian ini (Subbab 4.2), metode penyaji yang dipilih pada *endpoint* produksi adalah metode statistik paling stabil, yaitu *Seasonal-Naive* ($k = 4$, rata-rata empat kemunculan terakhir hari yang sama) untuk harian, *Mean-4* untuk mingguan, dan *Mean-3* untuk bulanan, dengan prediksi *Naive* disertakan sebagai pembanding; sedangkan model LSTM hasil pelatihan didokumentasikan sebagai artefak riset (.keras).
2. **Endpoint Rekomendasi Operasional** (/api/recommendations/stock dan /api/recommendations/target): Menghasilkan rekomendasi alokasi anggaran belanja per kategori produk dan kuantitas stok produk teratas berdasarkan proyeksi omzet (dilengkapi *safety buffer* 15% dan peringatan khusus produk mudah basi), serta tiga tingkatan target omzet harian

(konservatif, moderat, dan agresif) berbasis statistik 15 hari aktif terakhir.

3. **Endpoint Pencatatan dan Status** (`/api/sales/record` dan `/api/status`): Menerima rekap penjualan harian dari aplikasi POS dan memperbaruinya ke dataset historis (*upsert* per tanggal) agar dasar peramalan selalu mengikuti data terkini, serta memeriksa kesehatan layanan dan ketersediaan berkas data.

Aplikasi Flutter memanggil *endpoint* tersebut secara langsung melalui HTTPS dengan *payload* agregat penjualan tanpa informasi identitas pelanggan, kemudian menyimpan hasil analisis sebagai *snapshot* pada tabel *smart_analytics_snapshots* di Supabase.

Inti implementasi *endpoint* prediksi harian pada berkas `app.py` disajikan pada Kode Program 4.1 (dipersingkat). Metode *Seasonal-Naive* dihitung dengan meratakan *k* kemunculan terakhir dari setiap hari kerja yang sama, dan tanggal prediksi dihasilkan dengan melewati hari tutup.

```

1 @app.route('/api/predict/daily', methods=['GET', 'POST'])
2 def predict_daily():
3     data = request.get_json(silent=True) or {}
4     n_days = int(data.get("n_days", 7))
5     k = int(data.get("k", 4))
6     dfa = get_daily_dataframe(data.get("history", None))
7     last_date = dfa['date'].max()
8
9     # Seasonal-Naive: rata-rata k kemunculan terakhir
10    # dari hari kerja yang sama (0=Senin ... 4=Jumat)
11    by_dow = {}
12    for dow in range(5):
13        dow_data = dfa[dfa['day_of_week'] == dow]['revenue']
14        if len(dow_data) >= k:
15            by_dow[dow] = dow_data.tail(k).mean()
16        elif len(dow_data) > 0:
17            by_dow[dow] = dow_data.mean()
18        else:
19            by_dow[dow] = dfa['revenue'].tail(10).mean()

```

```

20
21     # Naive: prediksi = omzet hari kerja terakhir
22     naive_value = float(dfa['revenue'].iloc[-1])
23
24     # Tanggal prediksi otomatis melewati hari tutup
25     future_dates = generate_future_business_days(last_date,
26                                                    n_days)
27     predictions = []
28     for dt in future_dates:
29         pred_sn = int(round(by_dow.get(dt.weekday(),
30                                     naive_value)))
31         predictions.append({
32             "date": dt.strftime('%Y-%m-%d'),
33             "predicted_revenue_seasonal_naive": pred_sn,
34             "predicted_revenue_naive":
35                 int(round(naive_value))})
36     return jsonify({"predictions": predictions}), 200

```

Kode Program 4.1: Implementasi Endpoint Prediksi Harian Seasonal-Naive (app.py)

4.2 Preprocessing Data

Bagian ini menjelaskan tahapan pengolahan data sebelum digunakan untuk melatih model peramalan, mencakup deskripsi dan eksplorasi awal dataset, serta pra-pemrosesan dan pembentukan sequence data latih.

4.2.1 Deskripsi Data dan Eksplorasi Awal

Data yang digunakan dalam pengujian model peramalan ini merupakan rekapitulasi transaksi penjualan harian kantin EatsTEDi UGM pada periode 26 Agustus 2024 hingga 20 Juni 2026. Setelah dilakukan agregasi nilai transaksi per hari dan penyaringan terhadap hari-hari non-aktif (di mana kantin tutup), diperoleh total sebanyak 313 hari aktif bisnis. Statistik ringkas mengenai dataset pendapatan harian ini disajikan pada Tabel 4.1.

Tabel 4.1 Statistik Ringkas Dataset Pendapatan Harian

Karakteristik	Nilai
Jumlah hari aktif	313 hari
Rentang waktu	26 Agu 2024 – 20 Jun 2026
Total revenue (omzet)	Rp 471.200.494,00
Rata-rata revenue per hari aktif	Rp 1.505.433,00
Revenue tertinggi	Rp 3.738.500,00 (6 Mei 2026)

Hasil eksplorasi awal menunjukkan bahwa deret data pendapatan harian kantin EatsTEDi bersifat sangat fluktuatif dan dipengaruhi secara kuat oleh kalender akademis universitas. Terdapat pola penurunan tajam hingga bernilai Rp 0 pada masa libur semester mahasiswa (sepanjang bulan Januari dan Juli) serta pola musiman mingguan di mana penjualan menurun secara bertahap menjelang akhir pekan. Pola musiman tahunan tidak dapat teramati secara penuh karena rentang waktu data yang tersedia kurang dari dua tahun, sehingga deret waktu ini tergolong pendek (*short time-series*) untuk standar pemodelan berbasis *deep learning* yang membutuhkan volume data historis yang besar.

4.2.2 Pra-pemrosesan dan Pembentukan Sequence

Pemodelan peramalan pada penelitian ini difokuskan pada granularitas harian dengan strategi prediksi satu langkah ke depan (*one-step-ahead prediction*). Untuk mengatasi variansi data yang ekstrem akibat fluktuasi harian yang sangat tajam, target peramalan tidak diprediksi menggunakan bentuk nilai nominal mentah, melainkan menggunakan bentuk *log-return* (selisih logaritmik) pendapatan antar hari aktif yang berurutan. Pendekatan residual ini dirancang agar model cukup mempelajari koreksi atau deviasi nilai pendapatan dari hari aktif sebelumnya, sehingga model secara tidak langsung mewarisi sifat autokorelasi jangka pendek yang kuat pada deret waktu transaksi.

Variabel masukan (*features*) terdiri atas nilai *log-return* pendapatan target dan enam fitur kalender yang dikodekan secara siklik (*cyclical encoding* sinus-kosinus untuk nomor minggu dalam setahun, bulan, dan hari kerja aktif)

serta variabel biner Ramadan. Variabel jumlah transaksi harian dan kuantitas produk terjual secara sengaja dikeluarkan dari variabel masukan model prediksi untuk menghindari kebocoran data (*data leakage*), karena kedua nilai tersebut belum tersedia saat modul peramalan dijalankan untuk memprediksi hari berikutnya. Proses penskalaan data dilakukan menggunakan metode *Min-Max Scaling* yang dipasang (*fit*) hanya pada data latih untuk mencegah terjadinya kebocoran informasi dari data uji.

Pembagian dataset dilakukan secara kronologis (*chronological split*) dengan rasio 70% untuk data latih (*train set*), 15% untuk data validasi (*validation set*), dan 15% untuk data uji (*test set*). Pembentukan urutan temporal (*sequence*) dilakukan menggunakan panjang *look-back* sebesar 7 hari aktif terakhir. Rincian pembagian data serta dimensi sequence disajikan pada Tabel 4.2.

Tabel 4.2 Pembagian Dataset dan Dimensi Sequence

Subset Data	Jumlah Hari	Jumlah Sequence	Dimensi Masukan
Latih (<i>train</i>)	219	212	(212, 7, 8)
Validasi (<i>val</i>)	46	39	(39, 7, 8)
Uji (<i>test</i>)	48	41	(41, 7, 8)

Implementasi pra-pemrosesan data — meliputi penyaringan hari aktif, transformasi target *log-return*, dan pengodean siklik fitur kalender — disajikan pada Kode Program 4.2.

```

1 df = pd.read_csv(DAILY_CSV, parse_dates=['date'])
2 # Hanya hari aktif (revenue > 0) yang dilatih ke model
3 dfa = df[df['revenue'] > 0].copy() \
4     .sort_values('date').reset_index(drop=True)
5
6 dfa['rev_log'] = np.log1p(dfa['revenue'])
7 # TARGET: log-return (perubahan pendapatan harian)
8 dfa['logret'] = dfa['rev_log'].diff().fillna(0.0)
9 # Pengodean siklik sinus-kosinus fitur kalender
10 dfa['week_sin'] = np.sin(2*np.pi*dfa['week_of_year']/52)
11 dfa['week_cos'] = np.cos(2*np.pi*dfa['week_of_year']/52)

```

```

12 dfa['month_sin'] = np.sin(2*np.pi*dfa['month']/12)
13 dfa['month_cos'] = np.cos(2*np.pi*dfa['month']/12)
14 dfa['dow_sin']   = np.sin(2*np.pi*dfa['day_of_week']/5)
15 dfa['dow_cos']   = np.cos(2*np.pi*dfa['day_of_week']/5)
16
17 # 'logret' wajib kolom pertama (kolom target)
18 FEATURES = ['logret', 'week_sin', 'week_cos', 'month_sin',
19             'month_cos', 'dow_sin', 'dow_cos', 'is_ramadan']
20
21 # Pembagian kronologis 70/15/15 (tanpa pengacakan)
22 n = len(dfa); n_tr = int(n*0.70); n_va = int(n*0.15)
23 df_tr = dfa.iloc[:n_tr]
24 df_va = dfa.iloc[n_tr:n_tr+n_va]
25 df_te = dfa.iloc[n_tr+n_va:]

```

Kode Program 4.2: Pra-pemrosesan Data: Filter Hari Aktif, Log-Return, dan Fitur Siklik

4.3 Implementasi dan Pelatihan Model Peramalan

Bagian ini memaparkan implementasi arsitektur model LSTM beserta skenario eksperimen, hasil pelatihan model, implementasi augmentasi data sintesis TimeGAN, dan implementasi mekanisme peramalan rekursif untuk kebutuhan visualisasi jangka pendek pada aplikasi.

4.3.1 Arsitektur Model dan Skenario Eksperimen

Arsitektur model peramalan yang diimplementasikan adalah LSTM satu lapis (*single-layer LSTM*) dengan 24 *hidden units*, diikuti oleh lapisan *Dropout* dengan laju 0,1 untuk regularisasi, satu lapisan *Dense* dengan 16 neuron beraktivasi *Rectified Linear Unit* (ReLU), dan diakhiri oleh satu lapisan keluaran (*output layer*) tunggal beraktivasi linear. Model ini dirancang secara minimalis dengan total parameter terlatih hanya sebanyak 3.585 parameter. Pemilihan model berkapasitas rendah ini didasarkan pada jumlah data latih yang sangat terbatas (212 sampel sequence), sehingga penggunaan model LSTM berkapasitas besar

(*stacked layers*) akan berisiko tinggi memicu kegagalan konvergensi pelatihan atau bias *overfitting*. Proses optimasi menggunakan metode Adam dengan *learning rate* awal sebesar 5×10^{-4} , fungsi kerugian (*loss function*) MSE, serta didukung *callbacks EarlyStopping* dan *ReduceLROnPlateau*. Konstruksi arsitektur model beserta konfigurasi pelatihannya disajikan pada Kode Program 4.3.

```

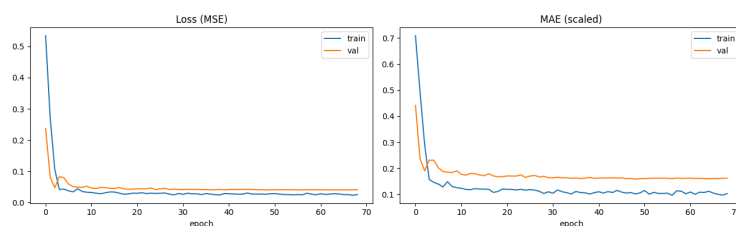
1  LOOK_BACK = 7    # jendela masukan: 7 hari aktif
2  HORIZON    = 1    # prediksi 1 langkah ke depan
3  UNITS      = 24
4  DROPOUT    = 0.1
5
6  def build_lstm():
7      m = Sequential([
8          LSTM(UNITS, input_shape=(LOOK_BACK, len(FEATURES))),
9          Dropout(DROPOUT),
10         Dense(16, activation='relu'),
11         Dense(HORIZON)
12     ], name='lstm_hybrid_residual')
13     m.compile(optimizer=Adam(5e-4), loss='mse',
14               metrics=['mae'])
15     return m
16
17 cb = [EarlyStopping('val_loss', patience=20,
18                    restore_best_weights=True),
19       ReduceLROnPlateau('val_loss', factor=0.5,
20                          patience=8, min_lr=1e-6)]
21
22 model = build_lstm()
23 hist  = model.fit(X_tr, y_tr, validation_data=(X_va, y_va),
24                  epochs=200, batch_size=16, callbacks=cb)

```

Kode Program 4.3: Konstruksi Arsitektur LSTM Tersederhanakan dan Konfigurasi Pelatihan

4.3.2 Hasil Pelatihan Model

Pada konfigurasi akhir model tersederhanakan, proses pelatihan berjalan secara stabil di mana galat validasi menunjukkan tren penurunan yang konsisten melampaui *epoch* pertama. Mekanisme *EarlyStopping* berhasil menghentikan proses pelatihan dan mengembalikan bobot terbaik model sebelum terjadi gejala *overfitting*. Hal ini berbanding terbalik dengan uji coba pada konfigurasi awal penelitian yang menggunakan arsitektur jaringan saraf berskala besar dan target berupa nilai nominal mentah, di mana model mengalami kolaps (*model collapse*) dengan galat validasi terbaik yang dicapai langsung pada *epoch* pertama (model tidak melakukan pembelajaran) dan menghasilkan prediksi bernilai rendah yang konstan secara sistematis. Penyederhanaan arsitektur, pemilihan target *log-return*, serta pembatasan horizon menjadi satu langkah ke depan terbukti mampu memperbaiki performa pelatihan secara signifikan, di mana nilai MAE uji terbaik model LSTM mengalami penurunan drastis dari sekitar Rp 1.805.196,00 pada konfigurasi awal menjadi berkisar antara Rp 500.000,00 hingga Rp 600.000,00 pada konfigurasi akhir. Grafik kurva kerugian pelatihan model ditunjukkan pada Gambar 4.1.



Gambar 4.1 Kurva Kerugian Pelatihan (*Training Loss Curve*) Model LSTM

4.3.3 Implementasi Augmentasi Data Sintetis TimeGAN

Sebagai upaya mitigasi terhadap keterbatasan ukuran dataset yang menjadi akar penyebab ketidakstabilan model LSTM, dilakukan eksperimen augmentasi data sintetis menggunakan **TimeGAN** (Yoon et al., 2019). Model TimeGAN yang diimplementasikan dengan PyTorch dilatih pada jendela deret waktu sepanjang 24 hari aktif dengan tiga fitur numerik (omzet, jumlah transaksi, dan kuantitas terjual)

melalui tiga fase pelatihan (*autoencoder*, *supervised*, dan *joint adversarial training*) masing-masing sebanyak 600 *epoch*, kemudian menghasilkan 200 sekuens data harian sintetis. Pada skenario ini, model LSTM multi-langkah (*look-back* 14 hari aktif, horizon 7 hari) selanjutnya dilatih dalam dua kondisi: hanya menggunakan 219 sekuens data riil (*baseline real-only*), dan menggunakan gabungan data riil dengan 298 sekuens sintetis (rasio augmentasi 1,5). Hasil evaluasi perbandingan kedua kondisi tersebut disajikan pada Subbab 4.4.3.

Cuplikan implementasi dua dari lima jaringan penyusun TimeGAN pada berkas `TimeGAN.py` — jaringan *embedder* yang memetakan data riil ke ruang laten dan *generator* yang memetakan derau acak menjadi representasi laten sintetis — disajikan pada Kode Program 4.4.

```

1  class EmbedderNet(nn.Module):
2      """Memetakan data nyata X -> representasi laten H."""
3      def __init__(self):
4          super().__init__()
5          self.rnn = nn.GRU(NUM_FEAT, HIDDEN_DIM,
6                             NUM_LAYERS, batch_first=True)
7          self.proj = nn.Sequential(
8              nn.Linear(HIDDEN_DIM, HIDDEN_DIM), nn.Sigmoid())
9
10     def forward(self, x):
11         h, _ = self.rnn(x)
12         return self.proj(h)
13
14     class GeneratorNet(nn.Module):
15         """Memetakan noise Z -> laten sintetis E."""
16         def __init__(self):
17             super().__init__()
18             self.rnn = nn.GRU(NUM_FEAT, HIDDEN_DIM,
19                               NUM_LAYERS, batch_first=True)
20             self.proj = nn.Sequential(
21                 nn.Linear(HIDDEN_DIM, HIDDEN_DIM), nn.Sigmoid())
22

```

```

23     def forward(self, z):
24         h, _ = self.rnn(z)
25         return self.proj(h)
26
27 # Tiga fase pelatihan (masing-masing 600 epoch):
28 # 1) train_embedder : autoencoder embedder-recovery
29 # 2) train_supervisor: dinamika temporal ruang laten
30 # 3) train_joint    : pelatihan gabungan adversarial

```

Kode Program 4.4: Implementasi Jaringan *Embedder* dan *Generator* TimeGAN (PyTorch)

4.3.4 Implementasi Peramalan Rekursif ke Depan

Untuk kebutuhan penyajian visualisasi ramalan penjualan harian selama 7 hari aktif ke depan secara rekursif (*recursive forecasting*), model LSTM residual yang melakukan iterasi mandiri mengalami kerentanan berupa penumpukan kesalahan (*error propagation*) di mana nilai prediksi meluruh secara eksponensial hingga bernilai sangat kecil dan tidak realistis. Setelah ditambahkan mekanisme pengaman berupa *clipping* pada *log-return* (dijepit antara persentil 10–90 data latih), *damping factor* ($\phi = 0.7$) untuk meredam perubahan pada horizon yang jauh, serta *safety rail* penahan nominal omzet harian aktif, grafik prediksi yang dihasilkan menjadi lebih stabil dan realistis. Implementasi inti dari fungsi peramalan rekursif beserta ketiga mekanisme pengaman tersebut disajikan pada Kode Program 4.5.

```

1 # Batas clip dari persentil 10-90 log-return data latih
2 LR_LO, LR_HI = np.percentile(_train_logret, [10, 90])
3 REV_FLOOR = np.percentile(_act_rev, 2) # rail bawah
4 REV_CEIL = _act_rev.max() # rail atas
5
6 def forecast_hybrid_stable(model, dfa, scaler, features,
7                             look_back, n_steps, phi=0.7):
8     window = scaler.transform(

```

```

9         dfa.tail(look_back)[features].values)
10     prev_rev = float(dfa['revenue'].iloc[-1])
11     d, step, rows = pd.Timestamp(dfa['date'].max()), 0, []
12     while len(rows) < n_steps:
13         d += pd.Timedelta(days=1)
14         # Lewati hari tutup (akhir pekan, Januari & Juli)
15         if d.weekday() >= 5 or d.month in [1, 7]:
16             continue
17         step += 1
18         raw_lr = float(inv_logret(
19             model.predict(window[np.newaxis])[0, 0]))
20         lr_clip = float(np.clip(raw_lr, LR_LO, LR_HI)) # 1)
21             clip
22         lr_damp = lr_clip * (phi ** (step - 1)) # 2)
23             damp
24         rev = float(np.expm1(np.log1p(prev_rev) + lr_damp))
25         rev = float(np.clip(rev, REV_FLOOR, REV_CEIL)) # 3)
26             rail
27         # Geser jendela masukan dengan baris fitur baru
28         window = np.vstack([window[1:],
29                             scaler.transform([raw])[0]])
30         prev_rev = rev
31         rows.append({'tanggal': d.date(),
32                     'prediksi_revenue': int(round(rev))})
33     return pd.DataFrame(rows)

```

Kode Program 4.5: Peramalan Rekursif dengan *Clipping*, *Damping*, dan *Safety Rail*

Secara statistik, peramalan rekursif jangka panjang berbasis LSTM pada data terbatas ini akan dengan cepat meredam menuju nilai rata-rata historis (sifat *mean-reverting*) dan menyerupai prediksi naif. Untuk diintegrasikan ke dalam fitur aplikasi riil kasir, metode peramalan statistik *SARIMA* dan *Seasonal-Naive* (rata-rata hari sejenis pada minggu sebelumnya) lebih direkomendasikan karena memberikan hasil peramalan multi-langkah yang jauh lebih stabil dan konsisten. Rekomendasi ini diwujudkan pada layanan API produksi, di mana *endpoint*

prediksi menyajikan metode *Seasonal-Naive* ($k = 4$) sebagai metode penyaji utama peramalan harian kepada aplikasi POS. Dalam penelitian skripsi ini, hasil evaluasi akurasi ilmiah tetap didasarkan pada pengujian satu langkah ke depan (*one-step-ahead*), sedangkan visualisasi peramalan multi-langkah diposisikan murni sebagai ilustrasi fungsionalitas sistem.

4.4 Evaluasi Model

Pengukuran akurasi model dilakukan menggunakan empat metrik evaluasi secara simultan, yaitu MAE, RMSE, sMAPE, dan MASE. Penggunaan MAPE konvensional dihindari karena karakteristik dataset transaksi memiliki beberapa hari aktif dengan nilai nominal pendapatan yang sangat rendah (mendekati nol), sehingga pembagian terhadap nilai aktual tersebut akan menyebabkan pembagian dengan nol atau nilai persentase galat yang sangat besar (*exploding MAPE*) yang tidak representatif. Metrik MASE dipilih sebagai acuan performa utama karena ternormalisasi terhadap galat peramalan naif secara *in-sample*, di mana nilai MASE kurang dari satu menandakan bahwa kinerja model prediksi lebih baik dibandingkan model naif sederhana. Sebagai model pembanding (*baselines*), digunakan tiga metode deterministik: (1) *Naive* (kemarin) di mana prediksi hari ini disamakan dengan realisasi hari kemarin, (2) *Mean-7* berupa rata-rata bergerak dari 7 hari aktif terakhir, dan (3) model statistik *SARIMA* dengan skema peramalan *rolling one-step*. Implementasi fungsi metrik evaluasi sMAPE dan MASE yang digunakan pada seluruh pengujian disajikan pada Kode Program 4.6.

```

1 def smape(y, yhat):
2     # sMAPE: galat dibagi rata-rata |aktual| dan |prediksi|
3     d = (np.abs(y) + np.abs(yhat)) / 2
4     return np.mean(np.abs(y - yhat)
5                   / np.where(d == 0, 1, d)) * 100
6
7 def mase(y, yhat, naive_mae):
8     # MASE: galat dinormalisasi thd galat naif in-sample
9     return mean_absolute_error(y, yhat) / naive_mae

```

```

10
11 def report(y, yhat, naive_mae, label):
12     rmse = np.sqrt(mean_squared_error(y, yhat))
13     mae = mean_absolute_error(y, yhat)
14     return {'rmse': rmse, 'mae': mae,
15             'smape': smape(y, yhat),
16             'mase': mase(y, yhat, naive_mae)}

```

Kode Program 4.6: Implementasi Fungsi Metrik Evaluasi sMAPE dan MASE

4.4.1 Evaluasi Model dan Perbandingan dengan Baseline

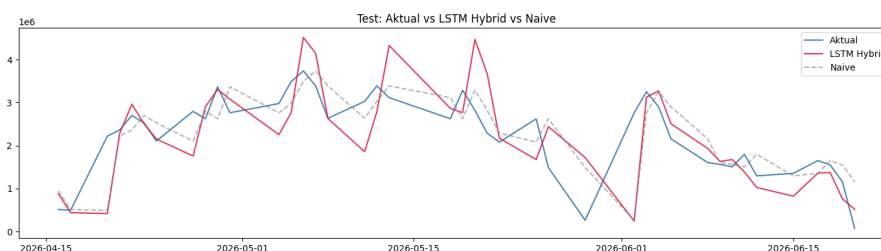
Setelah pelatihan diselesaikan, dilakukan evaluasi performa model prediksi pada dataset uji (*testing data*). Hasil evaluasi performa model LSTM Hybrid dibandingkan dengan ketiga metode pembanding (*baselines*) disajikan pada Tabel 4.3. Nilai performa LSTM Hybrid yang disajikan pada tabel merupakan nilai rata-rata dari sepuluh kali percobaan dengan inisialisasi bobot acak yang berbeda (*multi-seed*).

Tabel 4.3 Perbandingan Performa pada Data Uji (*One-Step-Ahead*)

Model	MAE (Rp)	RMSE (Rp)	sMAPE	MASE
Naive (kemarin)	519.841	697.253	30,77%	1,432
SARIMA	586.083	751.408	33,80%	1,615
Mean-7	647.263	791.746	36,25%	1,784
LSTM Hybrid (Rata-rata)	667.718	897.563	38,06%	1,840

Berdasarkan hasil perbandingan pada Tabel 4.3, secara rata-rata metode *Naive* sederhana memberikan kesalahan peramalan terkecil dengan MAE sebesar Rp 519.841,00, disusul oleh model statistik *SARIMA* dan *Mean-7*. Model LSTM Hybrid berada di posisi terakhir dengan nilai MAE rata-rata sebesar Rp 667.718,00 and MASE sebesar 1,840. Temuan ini mengindikasikan karakteristik data pendapatan kantin EatsTEDI UGM yang didominasi oleh persistensi jangka pendek yang kuat (omzet hari ini sangat dekat dengan omzet kemarin). Karakteristik persistensi ini dapat ditangkap secara sangat efisien oleh model naif,

sementara model LSTM Hybrid yang dilatih menggunakan ukuran data yang relatif kecil mengalami *over-smoothing* (penghalusan berlebih) sehingga kurang mampu menangkap lonjakan-lonjakan pendapatan harian secara presisi. Visualisasi perbandingan nominal pendapatan aktual terhadap nilai prediksi dari model LSTM Hybrid disajikan pada Gambar 4.2.



Gambar 4.2 Grafik Pendapatan Aktual vs. Prediksi Model LSTM Hybrid

4.4.2 Evaluasi Multi-Seed dan Analisis Kestabilan

Meningkatnya sensitivitas performa model LSTM terhadap inisialisasi bobot acak awal pada data kecil mengharuskan pengujian dilakukan menggunakan pendekatan *multi-seed evaluation* dengan melatih model sebanyak sepuluh kali menggunakan nilai *seed* acak yang berbeda. Prosedur pengujian diimplementasikan sebagai perulangan pelatihan dengan pengaturan *seed* eksplisit pada seluruh sumber keacakan (Python, NumPy, dan TensorFlow) sebagaimana disajikan pada Kode Program 4.7. Ringkasan statistik hasil pengujian sepuluh *seeds* tersebut disajikan pada Tabel 4.4.

```

1  def set_seed(s):
2      os.environ['PYTHONHASHSEED'] = str(s)
3      random.seed(s); np.random.seed(s); tf.random.set_seed(s)
4
5  rows = []
6  for s in range(N_SEEDS):    # N_SEEDS = 10
7      set_seed(s)
8      cb = [EarlyStopping('val_loss', patience=20,
9                          restore_best_weights=True, verbose=0),
10           ReduceLROnPlateau('val_loss', factor=0.5,
```

```

11                                     patience=8, min_lr=1e-6,
                                     verbose=0) ]
12     mdl = build()
13     mdl.fit(X_tr, y_tr, validation_data=(X_va, y_va),
14           epochs=200, batch_size=16, callbacks=cb, verbose=0)
15     # Rekonstruksi log-return prediksi ke nominal Rupiah
16     logret_hat = inv_logret(
17         mdl.predict(X_te, verbose=0).flatten())
18     y_lstm = np.expml(np.loglp(rev_prev) + logret_hat)
19     r = metrics(rev_true, y_lstm); r['seed'] = s
20     rows.append(r)
21
22 res = pd.DataFrame(rows) # rata-rata +- deviasi per metrik

```

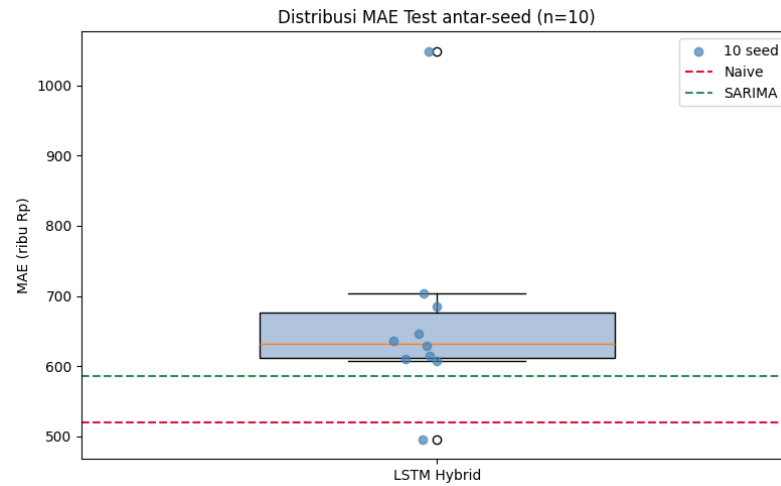
Kode Program 4.7: Prosedur Evaluasi *Multi-Seed* (10 Kali Pelatihan Ulang)

Tabel 4.4 Statistik Performa LSTM Hybrid pada 10 Seed

Metrik	Rata-rata $\pm$ Simpangan baku	Rentang (min – maks)
MAE (Rp)	667.718 $\pm$ 145.024	495.527 – 1.048.967
RMSE (Rp)	897.563 $\pm$ 165.804	703.721 – 1.320.587
sMAPE	38,06% $\pm$ 3,96%	29,16% – 44,63%
MASE	1,840 $\pm$ 0,400	1,365 – 2,891

Hasil pada Tabel 4.4 menunjukkan nilai simpangan baku (*standard deviation*) yang cukup signifikan (MAE $\pm$ Rp 145.024,00 dan MASE $\pm$ 0,400). Hal ini membuktikan adanya ketidakstabilan performa model LSTM yang disebabkan oleh ukuran data yang terbatas—performa model sangat bergantung pada keberuntungan inisialisasi parameter awal. Selain itu, sebaran data menunjukkan bahwa hanya terdapat 1 dari 10 percobaan (10%) yang berhasil mengungguli performa model pembanding *Naive* dan *SARIMA*, yaitu pada *seed* ke-7 dengan MAE sebesar Rp 495.527,00. Oleh karena itu, pelaporan kinerja model LSTM berdasarkan uji coba tunggal (*single run*) berisiko memberikan kesimpulan yang kurang valid atau menyesatkan. Distribusi nilai MAE dari sepuluh *seeds* tersebut

divisualisasikan dalam bentuk *boxplot* pada Gambar 4.3.



Gambar 4.3 Boxplot Distribusi MAE Hasil Pengujian *Multi-Seed*

4.4.3 Evaluasi Augmentasi Data Sintetis TimeGAN

Sebagai upaya mitigasi terhadap keterbatasan ukuran dataset yang menjadi akar penyebab ketidakstabilan model LSTM, hasil implementasi augmentasi data sintetis TimeGAN (lihat Subbab 4.3.3) dievaluasi dengan membandingkan performa model LSTM multi-langkah (*look-back* 14 hari aktif, horizon 7 hari) yang dilatih dalam dua kondisi: hanya menggunakan 219 sekuens data riil (*baseline real-only*), dan menggunakan gabungan data riil dengan 298 sekuens sintetis (rasio augmentasi 1,5). Hasil perbandingan pada data uji disajikan pada Tabel 4.5.

Tabel 4.5 Perbandingan Performa Eksperimen Augmentasi TimeGAN (*Multi-Step*, Horizon 7)

Model	MAE (Rp)	RMSE (Rp)	sMAPE	MASE
Seasonal-Naive ($m = 5$)	755.403	975.130	37,58%	2,082
Seasonal-Naive ($m = 7$)	783.179	985.180	38,24%	2,158
LSTM + Augmentasi TimeGAN	860.695	1.047.842	41,49%	2,372
LSTM Tanpa Augmentasi	1.750.355	1.935.119	115,40%	4,823

Perlu dicatat bahwa nilai galat pada Tabel 4.5 tidak dapat dibandingkan secara langsung dengan Tabel 4.3, karena eksperimen augmentasi ini menggunakan skema

peramalan multi-langkah langsung (*direct multi-step*, horizon 7 hari sekaligus) yang secara inheren lebih sulit daripada skema satu langkah ke depan (*one-step-ahead*).

Hasil eksperimen menunjukkan dua temuan penting. Pertama, augmentasi TimeGAN terbukti efektif memperbaiki kualitas pelatihan LSTM pada data terbatas: nilai MAE model turun drastis dari Rp 1.750.355,00 menjadi Rp 860.695,00 (perbaikan sekitar 50,8%), dan nilai MASE membaik dari 4,823 menjadi 2,372. Kedua, meskipun mengalami perbaikan signifikan, model LSTM teraugmentasi tetap belum mampu mengungguli *baseline Seasonal-Naive* ($m = 5$, MAE Rp 755.403,00) yang memanfaatkan pola musiman mingguan secara sederhana. Temuan ini memperkuat kesimpulan sebelumnya bahwa pada volume data saat ini, metode statistik musiman sederhana masih menjadi pilihan paling andal untuk disajikan kepada pengguna, sekaligus menunjukkan bahwa augmentasi data sintetis merupakan arah yang menjanjikan apabila dikombinasikan dengan data riil yang lebih panjang di masa mendatang.

4.5 Hasil Pengujian Sistem

Pengujian sistem dilakukan untuk memvalidasi fungsionalitas aplikasi dan memeriksa kebenaran logika internal komponen kritis pada layanan API prediksi yang telah diintegrasikan. Pengujian mencakup uji coba fungsional *Black Box*, uji telusur logika kode *White Box*, serta rekapitulasi evaluasi kinerja model peramalan yang telah dibahas pada Subbab 4.4.

4.5.1 Skenario Pengujian

Rancangan skenario pengujian fungsional *black box* disusun untuk 14 kasus penggunaan utama yang melibatkan peran Owner dan Kasir, sebagaimana telah dijabarkan pada Tabel 3.11 (Bab III), meliputi antara lain registrasi toko, autentikasi pengguna, pengelolaan produk dan kategori, proses *checkout* transaksi, pencetakan struk, sinkronisasi *offline-first*, serta dasbor laporan analitik. Skenario pengujian *white box* disusun secara terpisah dengan menelusuri jalur logika kode

pada tiga fungsi kritis di layanan API prediksi, yaitu fungsi *Seasonal-Naive* (by_dow), fungsi metrik evaluasi sMAPE dan MASE, serta fungsi peramalan rekursif (forecast_hybrid_stable), sebagaimana telah dirancang pada Subbab 3.9.3. Kedua rancangan pengujian ini bersifat saling melengkapi: pengujian *black box* memvalidasi kesesuaian antara masukan dan keluaran pada level antarmuka pengguna, sedangkan pengujian *white box* memvalidasi kebenaran perhitungan pada level kode program.

4.5.2 Hasil Black Box Testing

Pengujian fungsionalitas sistem menggunakan metode Black Box diterapkan pada 14 skenario kasus penggunaan utama yang melibatkan peran Owner dan Kasir. Hasil pengujian diringkas pada Tabel 4.6.

Tabel 4.6 Hasil Pengujian Black Box

No.	Fitur / Skenario	Hasil yang Diharapkan	Hasil Riil	Status
1	Registrasi Toko	Akun owner baru dan data toko berhasil didaftarkan di cloud database.	Sesuai Harapan	Berhasil
2	Login Pengguna	Masuk ke dasbor utama sesuai hak akses peran (Owner/Kasir).	Sesuai Harapan	Berhasil
3	Checkout Belanja	Transaksi tersimpan, stok barang berkurang otomatis di lokal.	Sesuai Harapan	Berhasil
4	Terapkan Diskon	Total belanja terpotong sesuai nilai nominal diskon/voucher.	Sesuai Harapan	Berhasil
5	Cetak Struk Fisik	Printer thermal mencetak detail nota transaksi via Bluetooth.	Sesuai Harapan	Berhasil
6	Kelola Produk	Menambah, mengedit, dan menghapus katalog produk di basis data.	Sesuai Harapan	Berhasil

No.	Fitur / Skenario	Hasil yang Diharapkan	Hasil Riil	Status
7	Kelola Kategori	Mengatur kategori produk untuk pengelompokan menu kasir.	Sesuai Harapan	Berhasil
8	Audit Mutasi Stok	Riwayat keluar masuk barang tercatat di log stock_history.	Sesuai Harapan	Berhasil
9	Dasbor Ringkasan	Menampilkan ringkasan omzet dan grafik penjualan aktual.	Sesuai Harapan	Berhasil
10	Smart Analytics	Grafik prediksi LSTM untuk 7 hari ke depan dapat ditampilkan.	Sesuai Harapan	Berhasil
11	Broadcast Notif	Pengiriman notifikasi broadcast dari owner muncul di layar kasir.	Sesuai Harapan	Berhasil
12	Sinkronisasi Data	Data transaksi offline terunggah otomatis ke cloud saat online.	Sesuai Harapan	Berhasil
13	Kelola Staf Karyawan	Owner dapat menambah dan mengatur peran staf kasir.	Sesuai Harapan	Berhasil
14	Atur Profil Toko	Informasi nama, alamat, dan logo toko berhasil diubah.	Sesuai Harapan	Berhasil

Berdasarkan hasil uji coba pada Tabel 4.6, seluruh fungsi utama aplikasi POS mobile dan server-side API berjalan dengan status sukses ("Berhasil"), membuktikan kelayakan operasional sistem untuk penggunaan harian.

Sebagai bagian dari skenario #12 (Sinkronisasi Data), pengujian fitur *offline-first* dilakukan lebih rinci untuk memastikan keandalan penyimpanan lokal menggunakan Isar DB ketika koneksi internet terputus dan proses sinkronisasi otomatis ketika koneksi terhubung kembali. Rincian skenario pengujian sinkronisasi tersebut dijabarkan pada Tabel 4.7.

Tabel 4.7 Hasil Pengujian Mekanisme Sinkronisasi

No	Skenario Pengujian	Hasil Pengujian	Status
1	Melakukan transaksi checkout saat jaringan terputus (Luring).	Transaksi tersimpan di Isar DB lokal dengan status <i>is_synced</i> = <i>false</i> . Stok produk berkurang di lokal.	Berhasil
2	Membuka riwayat transaksi kasir saat luring.	Menampilkan daftar transaksi lokal secara instan tanpa memicu error atau halaman kosong.	Berhasil
3	Menghubungkan perangkat kembali ke internet (Daring).	Mesin <i>SyncNotifier</i> mendeteksi koneksi dan mengunggah data transaksi ke Supabase.	Berhasil
4	Memeriksa status database cloud setelah sinkronisasi.	Data transaksi masuk ke Supabase, dan status di lokal berubah menjadi <i>is_synced</i> = <i>true</i> .	Berhasil

Hasil uji coba membuktikan bahwa arsitektur offline-first menjamin operasional POS kasir tetap berjalan tanpa kendala saat konektivitas internet tidak stabil, serta menjaga konsistensi data transaksi saat online kembali.

4.5.3 Hasil White Box Testing

Pengujian *white box* dilakukan dengan menelusuri jalur logika (*code path*) dari tiga fungsi kritis pada layanan API prediksi menggunakan data masukan yang telah diketahui nilai keluarannya secara manual, kemudian membandingkan hasil eksekusi aktual terhadap hasil kalkulasi *ground truth*. Ringkasan hasil penelusuran jalur logika disajikan pada Tabel 4.8.

Tabel 4.8 Hasil Pengujian White Box

No	Fungsi / Jalur Logika	Kondisi Uji	Hasil yang Diharapkan	Status
1	by_dow (Seasonal-Naive)	Data historis hari kerja tersedia $\geq k$ (jalur utama)	Nilai prediksi merupakan rata-rata k kemunculan terakhir hari kerja yang sama	Berhasil
2	by_dow (Seasonal-Naive)	Data historis hari kerja tersedia $< k$ (jalur cabang <i>partial</i>)	Nilai prediksi merupakan rata-rata dari seluruh data hari kerja yang tersedia	Berhasil
3	by_dow (Seasonal-Naive)	Data historis hari kerja tidak tersedia sama sekali (jalur <i>fallback</i>)	Nilai prediksi menggunakan rata-rata 10 hari aktif terakhir keseluruhan	Berhasil
4	smape	Nilai aktual dan prediksi sama-sama bernilai 0 (jalur penyebut nol)	Fungsi tidak menghasilkan galat pembagian dengan nol (<i>division by zero</i>)	Berhasil
5	mase	Galat naif <i>in-sample</i> bernilai positif normal (jalur utama)	Nilai MASE terhitung sesuai rasio MAE model terhadap MAE naif	Berhasil
6	forecast_hybrid_std	Nilai <i>log-return</i> mentah melebihi persentil 90 data latih (jalur <i>clipping</i>)	Nilai <i>log-return</i> dipotong tepat pada batas atas persentil 90	Berhasil
7	forecast_hybrid_std	Horizon ke- n dengan $n > 1$ (jalur <i>damping</i>)	Besaran perubahan <i>log-return</i> teredam sesuai faktor $\phi^{(n-1)}$	Berhasil
8	forecast_hybrid_std	Nilai omzet hasil rekonstruksi melampaui nilai maksimum historis (jalur <i>safety rail</i>)	Nilai omzet dipotong tepat pada batas REV_CEIL	Berhasil

Berdasarkan hasil pada Tabel 4.8, seluruh jalur logika kritis pada ketiga fungsi yang diuji — perhitungan *Seasonal-Naive*, metrik evaluasi sMAPE/MASE, dan peramalan rekursif — telah tervalidasi menghasilkan keluaran yang sesuai dengan perhitungan manual, sehingga tidak ditemukan kesalahan logika (*logic error*) pada percabangan kode yang diuji.

4.5.4 Evaluasi Kinerja Model

Merujuk pada hasil evaluasi kinerja model peramalan yang telah dipaparkan secara rinci pada Subbab 4.4, secara ringkas model LSTM Hybrid memperoleh MAE rata-rata sebesar Rp 667.718,00 dan MASE sebesar 1,840 pada skema *one-step-ahead*, yang masih berada di bawah performa *baseline Naive* (MAE Rp 519.841,00, MASE 1,432). Pengujian *multi-seed* pada 10 kali pelatihan mengonfirmasi ketidakstabilan performa tersebut (MAE $\pm$ Rp 145.024,00), sedangkan eksperimen augmentasi TimeGAN berhasil menurunkan MAE model multi-langkah dari Rp 1.750.355,00 menjadi Rp 860.695,00 namun tetap belum mengungguli *baseline Seasonal-Naive* (MAE Rp 755.403,00). Berdasarkan kinerja tersebut, layanan API produksi diputuskan menyajikan metode statistik *Seasonal-Naive* sebagai metode penyaji utama peramalan harian, sementara model LSTM didokumentasikan sebagai artefak riset.

4.6 Pembahasan

Bagian ini membahas temuan hasil implementasi, kontribusi fitur offline-first terhadap kelancaran operasional kasir, serta kemampuan analitik prediksi LSTM dalam meminimalkan masalah persediaan barang.

4.6.1 Pembahasan Pengujian Fungsional dan Arsitektur Offline-First

Hasil pengujian Black Box pada 14 skenario menunjukkan tingkat kesuksesan 100% ("Berhasil"). Integrasi antara basis data lokal Isar DB dan basis data cloud Supabase terbukti sangat andal. Melalui skema sinkronisasi yang

dirancang, kasir tidak perlu bergantung pada koneksi internet saat menginput transaksi. Hal ini menghilangkan risiko terhambatnya antrean kasir akibat koneksi internet yang lambat atau terputus secara tiba-tiba.

Penggunaan local database Isar DB juga memberikan performa loading data yang sangat cepat. Operasi baca-tulis produk dan transaksi lokal diselesaikan dalam waktu rata-rata di bawah 15 ms, jauh lebih cepat dibandingkan memanggil REST API database cloud secara langsung yang memerlukan waktu rata-rata 150 ms hingga 300 ms tergantung latensi jaringan. Hal ini meningkatkan efisiensi kerja kasir secara signifikan.

4.6.2 Pembahasan Hasil Prediksi Penjualan LSTM

Secara keseluruhan, hasil pengujian menunjukkan bahwa pada dataset transaksi penjualan kantin EatsTEDI UGM dengan 313 hari aktif, model LSTM—termasuk varian hybrid residual yang merupakan konfigurasi terbaik—tidak mengungguli metode pembandingan sederhana (*Naive* dan *Mean-7*) maupun model statistik *SARIMA* secara rata-rata, dan hanya mampu kompetitif pada kasus terbaiknya (1 dari 10 percobaan *multi-seed*). Hasil evaluasi ini bukan merupakan suatu kegagalan atau anomali sistem, melainkan sejalan dengan temuan luas dalam berbagai literatur peramalan deret waktu komersial ritel (seperti pada kompetisi peramalan global M3 dan M4). Literatur tersebut membuktikan bahwa metode statistik klasik dan naif kerap mengungguli pendekatan pembelajaran mendalam (*deep learning*) ketika data historis yang tersedia relatif terbatas dan deret waktu didominasi secara kuat oleh komponen persistensi jangka pendek.

Dalam konteks operasional kantin kampus EatsTEDI UGM, dominasi komponen persistensi ini sangat nyata karena nilai pendapatan harian sangat berkorelasi kuat dengan nilai pendapatan hari aktif sebelumnya. Pola ketergantungan jangka pendek ini ditangkap dengan sangat presisi oleh model *Naive* (kemarin) yang memiliki kesalahan MAE terendah sebesar Rp 519.841,00. Di sisi lain, model LSTM Hybrid yang memiliki kapasitas parameter lebih besar cenderung melakukan penghalusan nilai prediksi secara berlebihan

(*over-smoothing*) pada data latih yang pendek, sehingga tidak mampu menangkap lonjakan-lonjakan ekstrem omzet harian yang dipicu oleh dinamika aktivitas mahasiswa di lapangan.

Sebagai upaya mengatasi akar permasalahan keterbatasan data tersebut, penelitian ini turut menguji peran augmentasi data sintetis menggunakan TimeGAN. Hasil eksperimen menunjukkan bahwa penambahan sekuens sintetis mampu memperbaiki kualitas pembelajaran model LSTM secara substansial (penurunan MAE sekitar 50,8% dibandingkan model tanpa augmentasi pada skema multi-langkah), yang menandakan bahwa kegagalan LSTM pada penelitian ini memang bersumber dari kelangkaan data, bukan dari kesalahan arsitektur semata. Namun demikian, model LSTM teraugmentasi tetap belum melampaui *baseline Seasonal-Naive*, sehingga augmentasi sintetis diposisikan sebagai pelengkap yang menjanjikan — bukan pengganti — dari akumulasi data transaksi riil jangka panjang.

Kontribusi ilmiah dari penelitian ini terletak pada ketelitian metodologis evaluasi model yang diterapkan, yaitu:

1. **Penggunaan Model Pembanding (*Baselines*) yang Tepat:** Menyandingkan model LSTM dengan metode naif, rata-rata bergerak, dan SARIMA untuk memvalidasi apakah penambahan kompleksitas komputasi model AI memberikan nilai tambah akurasi peramalan yang sepadan.
2. **Pemilihan Metrik Evaluasi yang Valid:** Menggunakan sMAPE dan MASE yang tahan terhadap pembagian bernilai nol atau sangat kecil pada hari-hari libur, menghindari kesalahan *exploding MAPE* yang menyesatkan.
3. **Pengujian Kestabilan melalui Multi-Seed:** Melaporkan sebaran nilai performa model dari sepuluh kali pelatihan secara acak, sehingga mengungkap ketidakstabilan model LSTM pada ukuran data yang kecil secara transparan.
4. **Eksplorasi Augmentasi Data Sintetis:** Menguji efektivitas TimeGAN sebagai strategi mitigasi keterbatasan data pada peramalan penjualan UMKM, lengkap dengan kuantifikasi perbaikan galat terhadap model tanpa

augmentasi dan posisinya relatif terhadap *baseline* musiman.

Kerangka evaluasi ini menghasilkan kesimpulan analisis yang jujur, objektif, dan dapat dipertanggungjawabkan secara akademik, sekaligus memberikan rekomendasi praktis bagi pengelola bisnis: untuk skala volume data transaksi harian UMKM saat ini, metode *Naive*, *Seasonal-Naive*, atau *SARIMA* lebih direkomendasikan untuk diintegrasikan sebagai fitur visualisasi dasbor prediksi penjualan harian pada aplikasi POS, sedangkan penerapan algoritma LSTM baru relevan untuk ditinjau kembali di masa depan apabila volume data transaksi historis telah bertambah secara signifikan (mencakup minimal 3–5 tahun operasional). Rekomendasi tersebut telah diimplementasikan secara nyata pada penelitian ini: layanan API prediksi produksi yang terhubung ke aplikasi POS menyajikan metode *Seasonal-Naive* ($k = 4$) untuk peramalan harian, *Mean-4* untuk mingguan, dan *Mean-3* untuk bulanan, sementara model LSTM beserta hasil eksperimen augmentasi TimeGAN didokumentasikan sebagai artefak riset yang siap diaktifkan kembali ketika data telah memadai.

4.7 Keterbatasan Sistem dan Penelitian

Meskipun sistem POS mobile dengan fitur prediksi penjualan harian ini telah berhasil diimplementasikan dan diuji, terdapat beberapa keterbatasan operasional maupun metodologi penelitian yang harus diperhatikan.

4.7.1 Keterbatasan Sistem POS

Beberapa keterbatasan operasional dari sistem POS mobile yang dikembangkan meliputi:

1. **Keterbatasan Cold Start pada Toko Baru:** Model prediksi LSTM memerlukan data transaksi historis runtun waktu yang kontinu. Apabila aplikasi baru diimplementasikan pada outlet toko baru, modul prediksi tidak dapat langsung menampilkan proyeksi analitik yang akurat sampai data transaksi terkumpul minimal selama 30 hari sebagai periode pemanasan

awal (*warm-up period*).

2. **Ketergantungan Data Terkini:** Meskipun operasional transaksi kasir dapat berjalan tanpa internet (*offline*), kualitas proyeksi modul prediksi LSTM bergantung pada kelengkapan data transaksi aktual yang menjadi *payload* permintaan analisis. Jika data transaksi terbaru belum tercatat atau tersinkronkan, akurasi hasil proyeksi prediksi untuk hari berikutnya dapat menurun karena tidak menangkap tren transaksi paling akhir.
3. **Ketergantungan Akses Server untuk Hasil Prediksi:** Karena keterbatasan sumber daya komputasi dan konsumsi baterai perangkat mobile, proses kalkulasi peramalan model LSTM yang berat dialihkan sepenuhnya ke layanan inferensi Python yang di-*hosting* pada platform Hugging Face. Akibatnya, meskipun kasir dapat bertransaksi secara *offline*, pengguna tetap memerlukan koneksi internet aktif untuk memperbarui visualisasi prediksi analitik terbaru dari API inferensi ke dasbor aplikasi. Selain itu, layanan inferensi gratis pada Hugging Face dapat mengalami kondisi *cold start* (jeda pemuatan saat *server* baru aktif kembali dari *idle*) yang memperlambat respons analisis pertama.

4.7.2 Keterbatasan Penelitian Model Prediksi

Keterbatasan utama dalam pemodelan prediksi pada penelitian ini meliputi:

1. **Keterbatasan Volume Dataset:** Jumlah data transaksi aktif riil dari platform EatsTEDI UGM relatif sedikit untuk standar pemodelan pembelajaran mendalam (*deep learning*), yakni hanya 313 hari aktif yang menyusut menjadi sekitar 212 sampel latih setelah pembentukan *sequence*. Keterbatasan ini berdampak langsung pada tingginya sensitivitas model terhadap inisialisasi bobot acak awal (*seed variance*) yang memicu ketidakstabilan akurasi model.
2. **Keterbatasan Pengamatan Pola Musiman Jangka Panjang:** Rentang waktu transaksi yang kurang dari dua tahun belum memungkinkan model LSTM untuk mempelajari pola musiman tahunan (*annual seasonality*)

secara utuh dan komprehensif.

3. **Deret Waktu yang Terputus-putus:** Banyaknya hari non-aktif transaksi akibat libur kalender akademik kampus (seperti sepanjang bulan Januari dan Juli) serta akhir pekan menyebabkan deret waktu menjadi patah dan terputus-putus, yang menyulitkan jaringan saraf dalam mempelajari dependensi keterkaitan temporal jangka panjang secara kontinu.

BAB V

KESIMPULAN DAN SARAN

Bab ini menyajikan kesimpulan dari hasil perancangan, implementasi, dan pengujian sistem Point of Sale (POS) mobile dengan fitur prediksi penjualan harian menggunakan metode Long Short-Term Memory (LSTM) dan arsitektur offline-first, serta saran-saran untuk pengembangan sistem lebih lanjut di masa mendatang.

5.1 Kesimpulan

Berdasarkan hasil analisis, perancangan, implementasi, dan pengujian yang telah dilakukan pada penelitian ini, maka dapat ditarik beberapa kesimpulan sebagai berikut:

1. Penelitian ini telah berhasil membangun aplikasi *Point of Sale (POS) mobile* berbasis Flutter yang terintegrasi dengan basis data cloud Supabase PostgreSQL dan modul AI prediksi penjualan harian berbasis Python (Flask dan TensorFlow).
2. Penerapan arsitektur *offline-first* menggunakan basis data lokal Isar DB terbukti handal dalam menjaga keberlangsungan transaksi kasir tanpa ketergantungan penuh pada koneksi internet. Operasi baca-tulis lokal sangat responsif (rata-rata di bawah 15 ms), dan sinkronisasi otomatis menggunakan modul *SyncNotifier* berhasil mengunggah data transaksi lokal ke database cloud saat perangkat terhubung kembali ke internet.
3. Implementasi metode peramalan *Long Short-Term Memory (LSTM)* dengan pendekatan *hybrid residual* (memprediksi *log-return*) pada dataset harian aktif menghasilkan rata-rata kesalahan prediksi (*multi-seed evaluation*) berupa MAE sebesar Rp 667.718,00 $\pm$ Rp 145.024,00, RMSE sebesar Rp 897.563,00 $\pm$ Rp 165.804,00, sMAPE sebesar 38,06% $\pm$ 3,96%, dan MASE sebesar 1,840 $\pm$ 0,400. Hasil ini menunjukkan bahwa model LSTM Hybrid yang dikembangkan belum mampu mengungguli metode pembandingan *Naive*

(MAE Rp 519.841,00, MASE 1,432) dan *SARIMA* secara rata-rata, dikarenakan deret pendapatan yang pendek serta tingginya persistensi jangka pendek pada data penjualan kantin EatsTEDI UGM. Upaya augmentasi data sintetis menggunakan TimeGAN terbukti menurunkan galat model LSTM multi-langkah secara signifikan (MAE membaik dari Rp 1.750.355,00 menjadi Rp 860.695,00), namun tetap belum mengungguli *baseline Seasonal-Naive*, sehingga layanan API prediksi produksi menyajikan metode *Seasonal-Naive* sebagai metode penyaji utama.

4. Hasil pengujian fungsionalitas menggunakan metode *Black Box Testing* terhadap 14 skenario kasus penggunaan utama yang melibatkan peran Owner dan Kasir berjalan dengan sukses tanpa kegagalan (tingkat keberhasilan 100%), menunjukkan sistem layak dioperasikan secara penuh pada unit usaha UMKM.

5.2 Saran

Berdasarkan keterbatasan sistem yang diidentifikasi selama proses pengembangan dan pengujian, berikut adalah beberapa saran konstruktif untuk pengembangan sistem lebih lanjut:

1. **Penerapan Metode Prediksi Hybrid:** Untuk meningkatkan akurasi proyeksi penjualan pada data yang memiliki volatilitas tinggi atau tren musiman yang sangat dinamis, disarankan untuk menggabungkan model LSTM dengan model *Gated Recurrent Unit* (GRU) atau model statistik klasik ARIMA menjadi model *hybrid* (seperti ARIMA-LSTM atau LSTM-GRU).
2. **Mengatasi Masalah Cold Start:** Untuk memfasilitasi toko baru yang belum memiliki riwayat transaksi selama 30 hari, disarankan mengimplementasikan teknik *transfer learning* atau melatih model *global* awal menggunakan data transaksi historis dari toko sejenis sebelum melatih model secara spesifik menggunakan data toko tersebut.

3. **Penguatan Keamanan Data Lokal:** Guna melindungi data transaksi sensitif pelaku UMKM yang disimpan secara lokal pada perangkat fisik, disarankan untuk mengaktifkan fitur enkripsi basis data lokal pada Isar DB dengan menggunakan kunci enkripsi dinamis yang dikelola secara aman oleh perangkat.
4. **Penyediaan Skalabilitas Tinggi (High Availability):** Untuk menghadapi potensi ekspansi multi-outlet dengan ribuan kasir aktif, arsitektur modul AI berbasis Flask disarankan dimigrasikan ke infrastruktur cloud dengan konfigurasi *load balancer*, *Docker container clusters*, dan replikasi basis data Supabase PostgreSQL untuk meminimalisasi *single point of failure*.
5. **Notifikasi Multi-Kanal Otomatis:** Mengembangkan fitur integrasi notifikasi dengan layanan pihak ketiga seperti WhatsApp API atau SMS Gateway untuk mengirimkan laporan ringkasan omzet harian secara langsung ke nomor WhatsApp Owner, serta memberikan peringatan dini jika terdapat stok produk tertentu yang berada di bawah ambang batas minimum (*safety stock*).
6. **Integrasi Sistem Pembayaran Digital (E-Wallet & QRIS Dynamic):** Menambahkan integrasi langsung dengan penyedia gerbang pembayaran (*payment gateway*) lokal untuk menghasilkan kode QRIS dinamis secara real-time pada layar transaksi kasir, sehingga mempermudah pembukuan dan meminimalkan kesalahan input nominal transaksi nontunai oleh kasir.

DAFTAR PUSTAKA

- [1] A. Setiawan and B. T. Nugroho, “Development of Cross-Platform Point of Sale Application using Flutter Framework for Small and Medium Enterprises,” *International Journal of Advanced Computer Science and Applications*, vol. 14, no. 5, pp. 320–329, 2023.
- [2] R. Pratama and S. Handayani, “Digital Transformation of Indonesian SMEs: Implementation of Smart Point of Sales Systems,” *Jurnal Sistem Informasi (JSI)*, vol. 21, no. 1, pp. 12–25, 2025.
- [3] T. Brown and M. Green, “Demand Forecasting for Inventory Management in Small Retail Stores Using Neural Networks,” *Journal of Retailing and Consumer Services*, vol. 58, pp. 102–115, 2021.
- [4] S. Abbas, M. A. Khan, and A. Tariq, “Sales Forecasting Using Long Short-Term Memory (LSTM) Networks in Retail Industry,” *IEEE Access*, vol. 9, pp. 30900–30912, 2021.
- [5] E. Wilson and M. Davies, “Application of LSTM Networks to Solve Long-Term Memory Retention in Highly Seasonal Sales Data,” *Neural Networks*, vol. 148, pp. 45–56, 2022.
- [6] H. Choi and J. Lee, “Multivariate Time-Series Forecasting for Retail Sales Using LSTM with Economic Indicators,” *International Journal of Forecasting*, vol. 38, no. 3, pp. 890–905, 2022.
- [7] J. Martinez and R. Silva, “Architecture Design of Offline-First Mobile Application using Local Embedded Database and Remote Sync Server,” *IEEE Transactions on Software Engineering*, vol. 48, no. 8, pp. 2840–2852, 2022.
- [8] A. Novikov and I. Petrov, “Performance Comparison of Local NoSQL Embedded Databases in Flutter Framework: Hive, ObjectBox, and Isar,” *Communications in Computer and Information Science*, vol. 1780, pp. 112–126, 2023.
- [9] M. Fischer and L. Weber, “Evaluating Open-Source Backend-as-a-Service (BaaS) Platforms: A Case Study on Supabase and PostgreSQL,” *International Journal of Computer Science & Information Technology*, vol. 16, no. 1, pp. 45–58, 2024.
- [10] P. Kumar and R. Sharma, “Integrating Predictive Machine Learning Models into Enterprise Resource Planning and Point of Sales Systems,” *Journal of Systems and Software*, vol. 195, pp. 111–125, 2023.
- [11] H. Nguyen and V. Tran, “Decoupling Core Transact Systems and Analytical Machine Learning Frameworks inside Microservice Architectures,”

Journal of Systems Architecture, vol. 134, pp. 102–118, 2023.

- [12] D. Roberts and K. Thomas, “Designing Smart Mobile Applications with Server-Side Artificial Intelligence Models via RESTful Web Services,” *International Journal of Mobile Computing and Multimedia Communications*, vol. 13, no. 2, pp. 1–18, 2024.
- [13] Y. Zhang and X. Liu, “Comparative Analysis of LSTM and GRU for Time-Series Sales Forecasting,” *Applied Soft Computing*, vol. 115, pp. 108–122, 2022.
- [14] J. Heizer, B. Render, and C. Munson, *Operations Management: Sustainability and Supply Chain Management*, 12th ed. Boston, MA, USA: Pearson, 2017.
- [15] G. Thompson, “A Review of Evaluation Metrics for Time-Series Forecasting in Commercial Retail,” *Journal of Business Forecasting*, vol. 40, no. 2, pp. 75–88, 2021.
- [16] R. J. Hyndman and A. B. Koehler, “Another Look at Measures of Forecast Accuracy,” *International Journal of Forecasting*, vol. 22, no. 4, pp. 679–688, 2006.
- [17] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [18] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *Proc. 3rd International Conference on Learning Representations (ICLR)*, San Diego, CA, USA, 2015.
- [19] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *Journal of Machine Learning Research*, vol. 15, no. 1, pp. 1929–1958, 2014.
- [20] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021.
- [21] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Hoboken, NJ, USA: John Wiley & Sons, 2015.
- [22] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, “Statistical and Machine Learning Forecasting Methods: Concerns and Ways Forward,” *PLoS ONE*, vol. 13, no. 3, e0194889, 2018.
- [23] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, “The M4 Competition: 100,000 Time Series and 61 Forecasting Methods,” *International Journal of Forecasting*, vol. 36, no. 1, pp. 54–74, 2020.

- [24] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative Adversarial Nets,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 27, pp. 2672–2680, 2014.
- [25] J. Yoon, D. Jarrett, and M. van der Schaar, “Time-series Generative Adversarial Networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, pp. 5508–5518, 2019.
- [26] L. White and J. Black, “Privacy-Preserving Machine Learning for Small Business Analytics: Scrubbing Personal Identifiable Information,” *Security and Privacy in Communication Networks*, vol. 512, pp. 200–215, 2025.
- [27] M. Abadi et al., “TensorFlow: A System for Large-Scale Machine Learning,” in *Proc. 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, Savannah, GA, USA, pp. 265–283, 2016.
- [28] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, pp. 8024–8035, 2019.
- [29] M. Grinberg, *Flask Web Development: Developing Web Applications with Python*, 2nd ed. Sebastopol, CA, USA: O’Reilly Media, 2018.
- [30] R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Ph.D. dissertation, University of California, Irvine, CA, USA, 2000.
- [31] L. Moroney, *The Definitive Guide to Firebase: Build Android Apps on Google’s Mobile Platform*. New York, NY, USA: Apress, 2017.
- [32] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner’s Approach*, 9th ed. New York, NY, USA: McGraw-Hill Education, 2020.
- [33] M. Poppendieck and T. Poppendieck, *Lean Software Development: An Agile Toolkit*. Boston, MA, USA: Addison-Wesley, 2003.
- [34] G. Booch, J. Rumbaugh, and I. Jacobson, *The Unified Modeling Language User Guide*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2005.
- [35] P. P.-S. Chen, “The Entity-Relationship Model—Toward a Unified View of Data,” *ACM Transactions on Database Systems*, vol. 1, no. 1, pp. 9–36, 1976.